

Data Architecture

System design for data platforms: batch and streaming, scaling, lakehouse patterns and data-mesh trade-offs

Nine parts, five depth levels, and annotated diagrams throughout. Each part goes deeper and more technical than the last.

Consume

BI · ML · applications · partners



Serve

SQL engines · semantic layer · caches



Process

Batch · streaming · ML pipelines



Store

Object storage · open table formats



Ingest

CDC · events · files · APIs

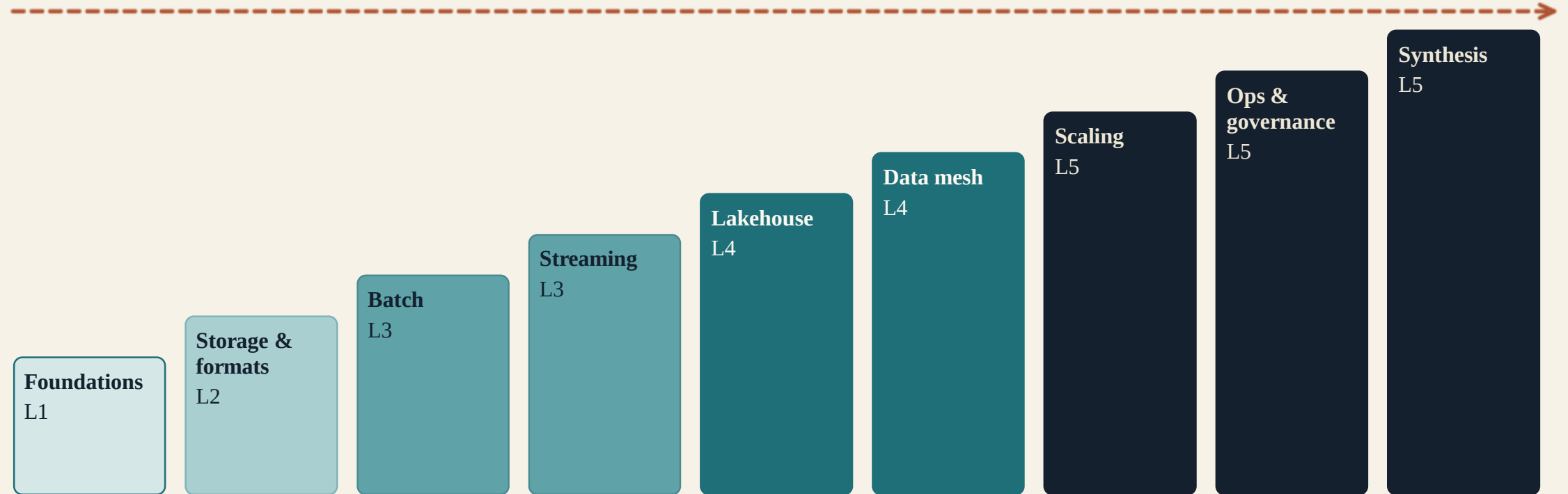


Sources

Databases · SaaS · devices · logs

How this tutorial is built: depth increases part by part

Depth and quantitative rigor increase from left to right



Key idea. L1 concepts · L2 structures · L3 mechanisms · L4 internals · L5 quantitative systems. Slide numbers in the footer tell you how far in you are.

What you will be able to do

Reason from requirements

Translate freshness, scale, correctness and cost targets into architecture choices you can defend with numbers.

Design storage and layout

Choose file formats, partitioning and clustering so that scans skip data instead of reading it.

Build reliable batch pipelines

Idempotent, incremental, backfillable ELT with explicit critical-path and skew handling.

Master streaming semantics

Logs, partitions, event time, watermarks, state, checkpoints and exactly-once end to end.

Operate a lakehouse

Table-format internals, ACID on object stores, compaction, catalogs and medallion modelling.

Weigh mesh and scale limits

Data-mesh trade-offs plus Amdahl, USL, sharding, tail latency and cost engineering.

Key idea. Every part ends in decisions: what to choose, what it costs, and how to know when it is failing.

Reading the diagrams: colours, arrows and notation

Teal

Storage and data at rest

Rust

Compute, processing and risk

Sand

Sources, consumers, neutral items

Gold

Metadata, state and control information

Ink

Governance and cross-cutting controls

Arrows and lines

Solid slate: data flow



Dashed: control, metadata or lineage



Rust: failure or problem path



Notation

≈ means approximate. Numbers marked illustrative give intuition, not benchmarks. Formulas use plain symbols. Boxed key ideas summarise each slide; speaker notes carry extra detail.

Key idea. Consistent colour and arrow conventions let you read any diagram in the deck at a glance.

Agenda: nine parts, 150 slides

1	Foundations	L1	Slides 6–27	Requirements, workloads, sizing, reference layers
2	Storage and formats	L2	Slides 28–42	Object stores, Parquet, partitioning, evolution
3	Batch processing	L3	Slides 43–57	Idempotency, Spark, shuffles, skew, cost
4	Streaming	L3	Slides 58–73	Logs, event time, watermarks, state, exactly-once
5	Lakehouse	L4	Slides 74–89	Iceberg and Delta internals, ACID, compaction
6	Data mesh	L4	Slides 90–103	Data products, contracts, platform, trade-offs
7	Scaling and capacity	L5	Slides 104–119	Amdahl, USL, queues, tail latency, capacity plans
8	Operations and governance	L5	Slides 120–133	Observability, security, privacy, FinOps, recovery
9	Synthesis	L5	Slides 134–150	Decision frameworks, case studies, review kit

PART 1 OF 9

Foundations

Requirements, workloads, the value chain and the shape of a data platform

- What architecture decides
- Workloads, freshness and sizing
- Reference layers and trade-off thinking

DEPTH LADDER · THIS PART IS LEVEL 1

L1 · Concepts

L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

What data architecture is

Logical architecture

What data exists and how it flows: domains, entities, pipelines, contracts. Technology-neutral.

Physical architecture

Where and how it runs: storage engines, formats, compute, networks, regions and failure domains.

Organisational architecture

Who owns, operates and pays: team boundaries, interfaces, on-call and governance. Conway's law applies.

Cheap to change

Query engines, BI tools, individual pipelines, SQL logic

Expensive to change

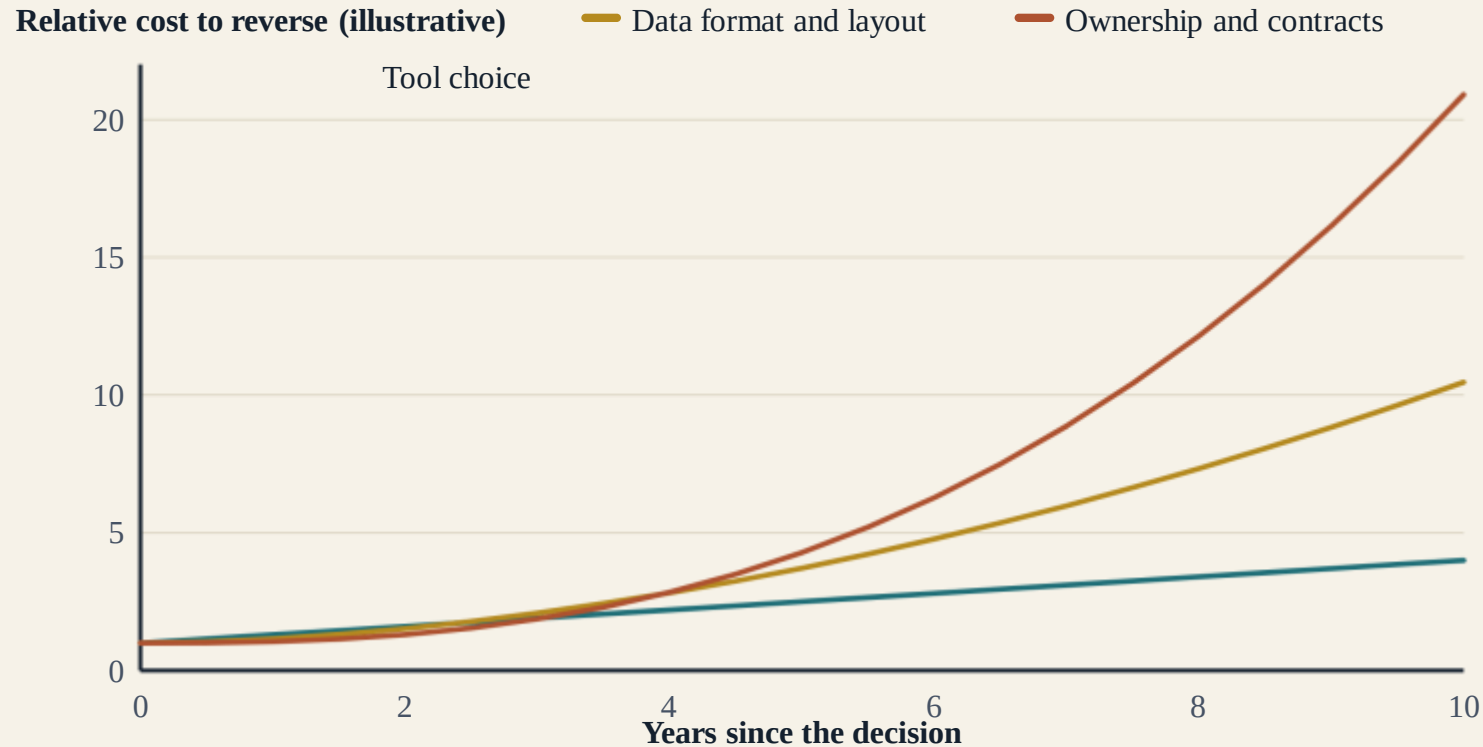
File and table formats, identifiers and keys, ownership boundaries, consistency guarantees, public contracts

Cost to reverse rises as you move down



Key idea. Architecture is the set of decisions that are expensive to reverse. Formats, ownership boundaries and guarantees outlive tools.

Why decisions become expensive to reverse



Tools

Swap in weeks. The data and contracts stay put, so cost grows slowly.

Formats and layout

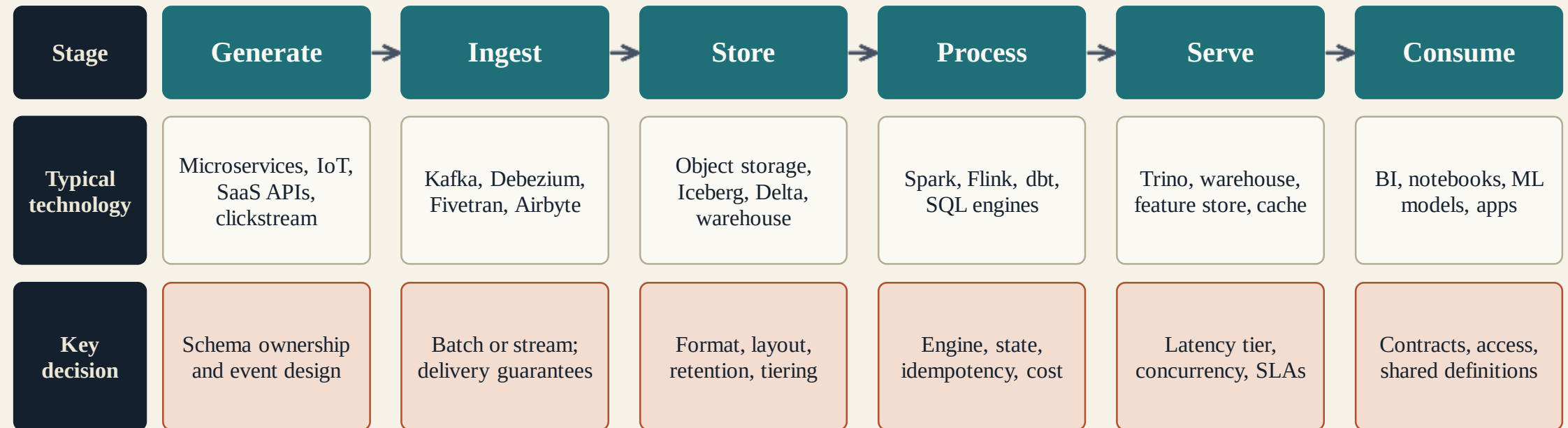
Rewriting petabytes is possible but slow and costly, and every reader is affected.

Ownership and contracts

Every downstream consumer depends on them. Cost rises steeply with adoption.

Key idea. Invest early in the right-hand curves: contracts, keys and formats. Choose tools last and keep them replaceable.

The data value chain, stage by stage



Cross-cutting: catalog · lineage · data quality · security · orchestration · cost control

Key idea. Every stage hides a decision that later stages inherit. Name the owner and the guarantee at each hand-off.

Seven workloads, seven access patterns

Workload	Access pattern	Latency	Data shape	Typical engines
OLTP	Point reads and writes, small transactions	ms	Row-oriented, normalised	PostgreSQL, MySQL, DynamoDB
BI / OLAP	Scans, aggregations and joins over many rows	seconds	Columnar, star schema	Warehouses, Trino, Spark SQL
Exploration	Unpredictable filters over wide tables	seconds–minutes	Wide, semi-structured	Notebooks, DuckDB, Spark
Stream analytics	Continuous windows over event streams	ms–seconds	Append-only events	Flink, Kafka Streams, Spark SS
ML training	Large sequential scans, repeated epochs	minutes–hours	Feature tables, tensors	Spark, Ray, GPU clusters
Feature serving	Key lookups at high query rates	under 10 ms	Key–value	Redis, DynamoDB, feature stores
Operational analytics	Low-latency aggregates behind applications	sub-second	Pre-aggregated, columnar	Druid, Pinot, ClickHouse

Key idea. One platform rarely serves all seven well. Decide which workloads matter, then choose storage and engines per workload.

OLTP and OLAP: two different jobs

Dimension	OLTP	OLAP
Unit of work	One transaction touching a few rows	One query scanning millions of rows
Query shape	Predictable, keyed lookups and updates	Ad hoc filters, joins and aggregations
Storage layout	Row-oriented pages	Column-oriented files or segments
Scaling	Scale up first; shard for writes	Scale out with parallel scans
Consistency	Strong ACID per transaction	Snapshot reads over batch-loaded data
Success metric	Latency and availability	Throughput and cost per query

Key idea. Different physics, different systems: mixing OLTP and OLAP on one engine trades away the strengths of both.

Quality attributes: turn wishes into numbers

Attribute	Question to ask	Typical metric
Freshness	How old may data be when a consumer reads it?	Age of newest row; end-to-end lag
Latency	How fast must a query or lookup return?	p50, p95 and p99 response time
Throughput	How much data per second or per day must move?	MB/s, events/s, TB/day
Correctness	May results be duplicated, lost or reordered?	Delivery semantics; reconciliation diffs
Availability	How long may the platform be down or degraded?	Uptime %, RTO, RPO
Scalability	What growth must be absorbed without redesign?	Headroom at 10× volume or users
Cost efficiency	What is the budget per TB, query or run?	Dollars per TB scanned or per run
Governance	Who may see what, and how is it audited?	Policy coverage; lineage completeness

Key idea. An attribute without a number is a wish. Ask for the metric, the target and the cost of missing it.

The freshness spectrum

Cost and operational complexity rise

Daily	Hourly	1–5 minutes	Seconds	Sub-second
Typical pattern Scheduled batch ETL; full or incremental loads	Typical pattern Incremental batch and partition swaps	Typical pattern Micro-batch or streaming into table formats	Typical pattern Streaming ETL and materialised views	Typical pattern Stateful stream processing and serving stores
Typical use Finance close, regulatory reports, model retraining	Typical use Operational dashboards, inventory reports	Typical use Product analytics, marketing attribution	Typical use Personalisation, monitoring, alerting	Typical use Fraud scoring, bidding, safety systems

Key idea. Pick the slowest freshness the business can tolerate. Each step to the right multiplies cost and operational burden.

SLIs, SLOs and SLAs for data

SLI — service level indicator

A measured quantity: the age of the newest row in orders_gold, sampled every 5 minutes.

SLO — service level objective

A target for the SLI: age ≤ 15 minutes in 99% of samples over 30 days.

SLA — service level agreement

A contract with consequences: credits, escalation or penalties when the SLO is missed.

Error budget = (1 – SLO) × window

(1 – 0.99) × 720 h = 7.2 hours of allowed breach per 30 days

SLI	Example SLO
Freshness	≤ 15 min for 99% of checks
Completeness	≥ 99.9% of source rows landed
Accuracy	Reconciliation delta ≤ 0.01%
Availability	Query endpoint ≥ 99.9% monthly

Key idea. SLOs make data quality operational: when the error budget is spent, feature work stops and reliability work starts.

Back-of-envelope sizing: rate × size × time

Events per second	Event size	Throughput	Per day	30 days raw
1,000	1 KB	1 MB/s	86.4 GB	2.6 TB
10,000	1 KB	10 MB/s	864 GB	25.9 TB
100,000	1 KB	100 MB/s	8.64 TB	259 TB
1,000,000	500 B	500 MB/s	43.2 TB	1.3 PB

Conversion

1 TB per day $\approx$ 11.6 MB/s sustained (10^{12} bytes $\div$ 86,400 s).

Peak versus average

Traffic is diurnal. Size for the peak, often several times the mean, and budget for the mean.

Compression

Columnar files with zstd are commonly several times smaller than raw JSON or CSV. Measure on your data.

Key idea. Sizing is arithmetic: rate × size × time. Do it before choosing technology; it tells you whether you have a small, medium or very large problem.

Six data models and their sweet spots

Relational

Strong schemas, joins and ACID. Watch out: horizontal write scaling needs sharding.

Document

Nested, flexible records that map to objects. Watch out: cross-document joins and consistency.

Key-value

Lowest-latency point access at huge scale. Watch out: no ad hoc queries beyond the key.

Wide-column

Massive write throughput with tunable consistency. Watch out: query patterns are fixed at design time.

Graph

Relationship traversals stay cheap as depth grows. Watch out: harder to shard; niche tooling.

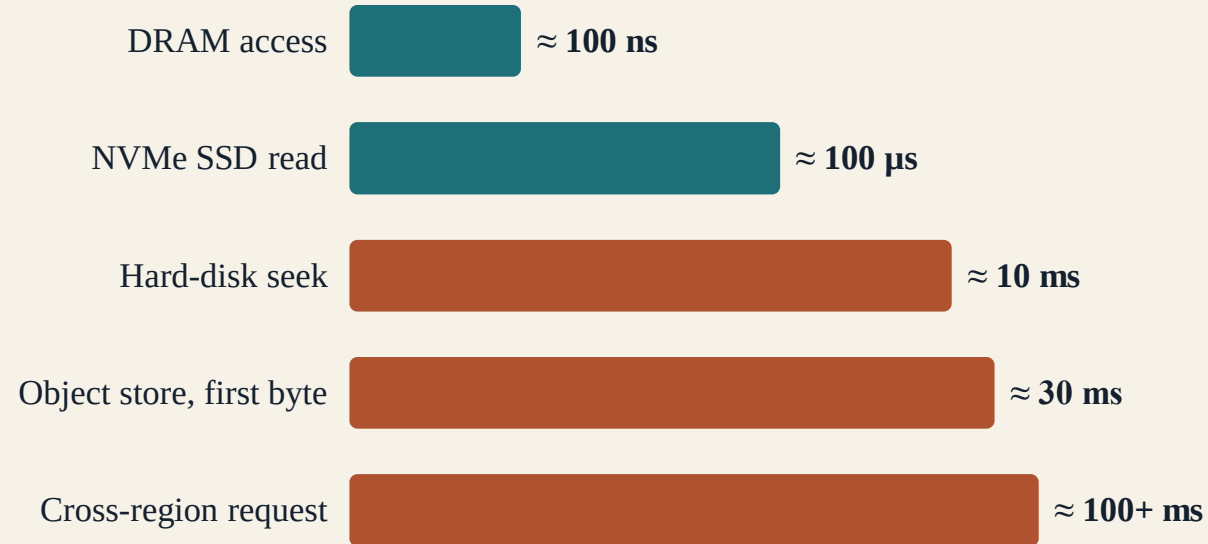
Time-series

Compression and retention built for timestamped metrics. Watch out: high-cardinality tags.

Key idea. The model follows the access pattern. Start from the queries you must answer, not from the data you happen to hold.

The storage hierarchy in numbers

Latency to first byte, log scale (illustrative orders of magnitude)



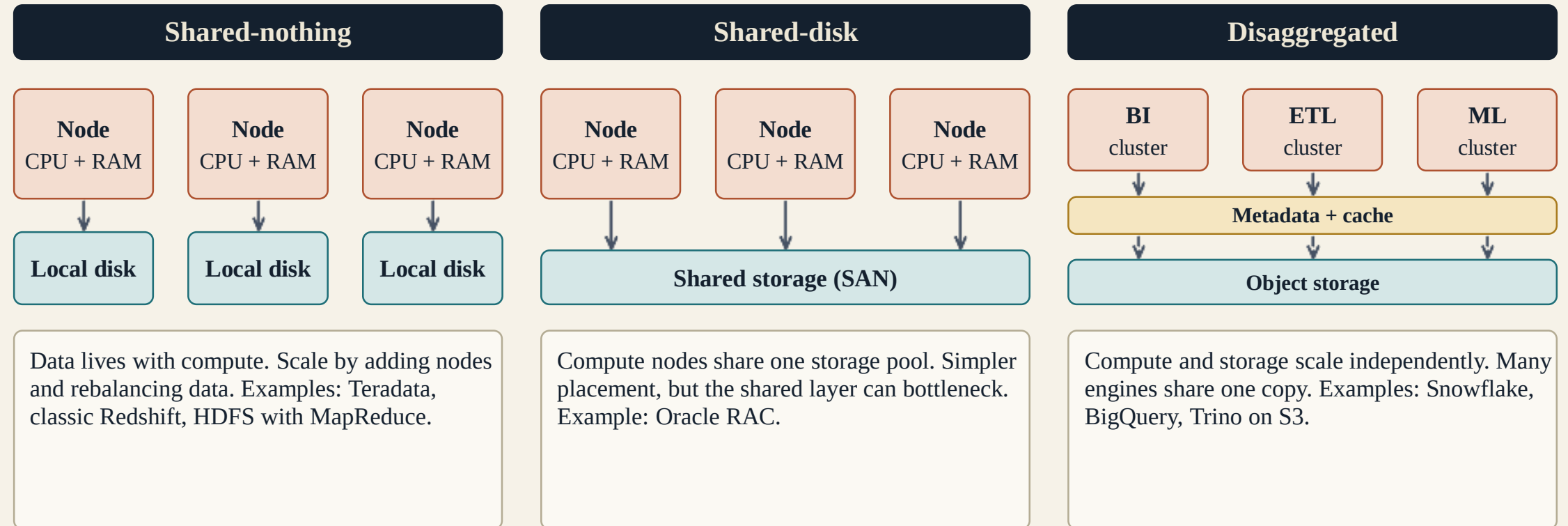
Six orders of magnitude
from memory to a remote object store.

Design consequence
Read big sequential ranges, cache hot data, prefetch, and avoid chatty small reads.

Latency is not throughput
Object stores deliver huge aggregate throughput through parallelism.

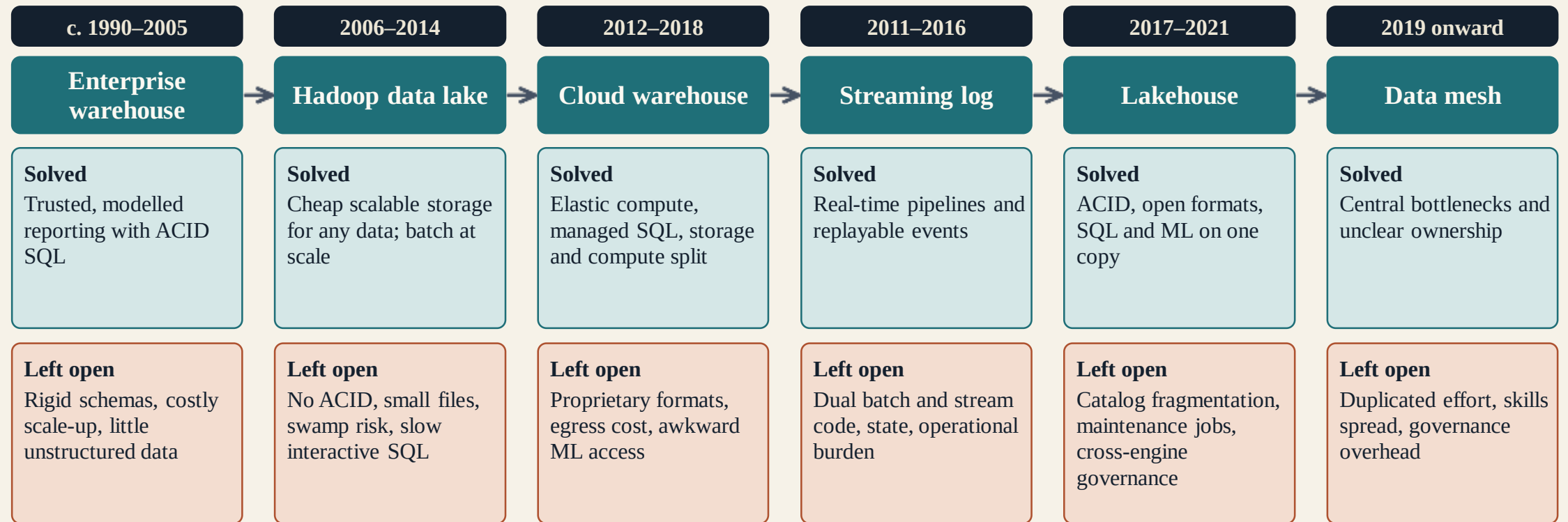
Key idea. Cheap, durable storage is slow per request. Data layout exists to turn many slow requests into a few large ones.

Three ways to couple compute and storage



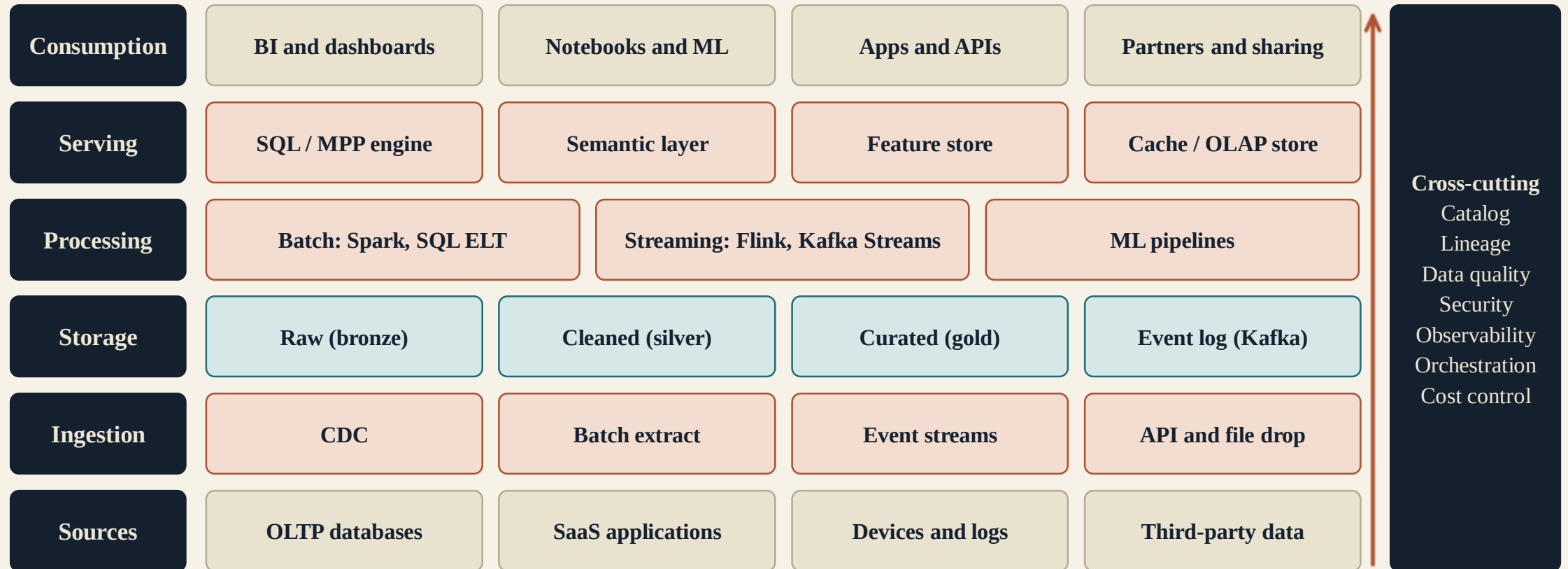
Key idea. The industry moved from shared-nothing to disaggregated storage: elasticity and one shared copy outweigh the loss of data locality.

Six generations of data platforms



Key idea. Each generation fixed the previous one's pain and created its own. Dates are approximate and eras overlap in practice.

A layered reference architecture

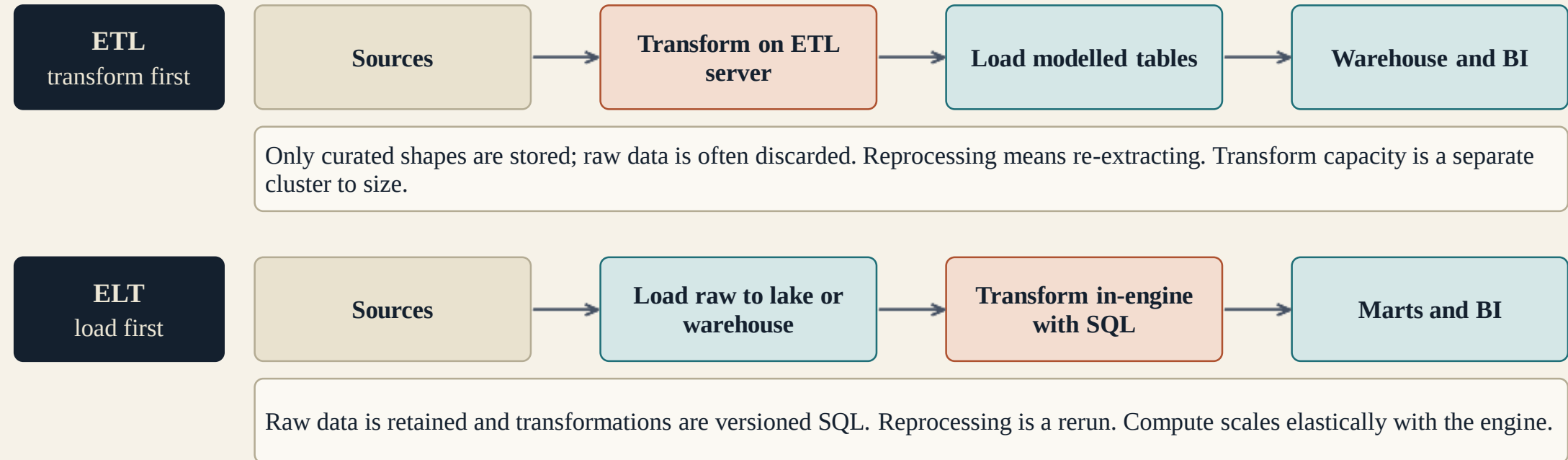


Read bottom to top: data moves up through the layers while metadata and policy apply across all of them.

Six architecture styles compared

Style	Core idea	Strength	Watch out for
Warehouse-centric	A curated SQL warehouse is the hub	Fast BI, strong governance	Cost at scale; poor fit for unstructured data
Data lake	Raw files on object storage	Cheap, flexible, any format	Swamp risk; no ACID; small files
Lakehouse	Open table formats add ACID and metadata to the lake	One copy for SQL and ML; open formats	Maintenance jobs; catalog dependence
Streaming-first (Kappa)	The log is the source of truth; reprocess by replay	Low latency; one code path	Retention cost; state and operations complexity
Data mesh	Domains own data products; federated governance	Scales organisations	Duplication; needs platform maturity
Federated query	Query data in place across sources	No copies; fast start	Performance, source load, weak history

ETL versus ELT



Key idea. ELT won because storage became cheap and engines elastic. ETL still fits when PII must be masked before anything lands.

Six ingestion patterns

Batch extract

Minutes to days. Full or incremental SELECT over JDBC. Gotcha: load on the source and missed deletes.

Log-based CDC

Seconds. Reads the database WAL or binlog. Gotcha: initial snapshots, schema changes, log retention.

Event streaming

Milliseconds to seconds. Producers publish to a log. Gotcha: schema governance and duplicates.

API polling and webhooks

Minutes. SaaS sources with rate limits. Gotcha: pagination, backfills, idempotency.

File drop

Hourly to daily. SFTP or object-store landing zones. Gotcha: partial files and late arrivals.

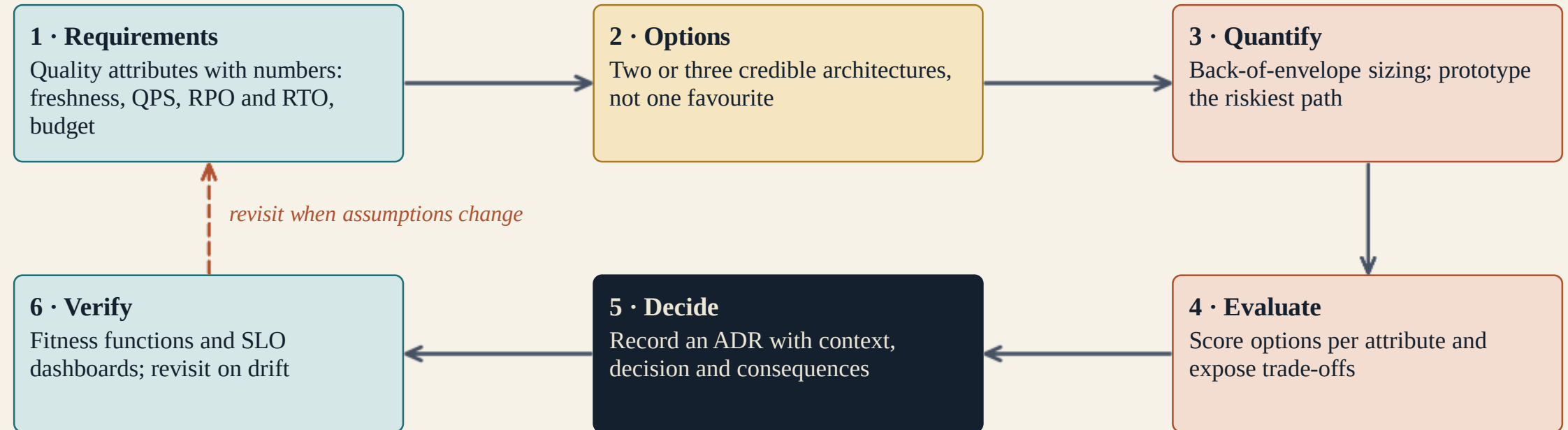
Reverse ETL

Minutes. Warehouse tables synced back into applications. Gotcha: loops and overwriting user edits.

Key idea. Choose the pattern from the source's capabilities and the freshness target. Log-based CDC is usually the best default for operational databases.

How architects decide: the trade-off loop

A repeatable loop for architecture decisions



Key idea. Decide with numbers and write the decision down. Revisit deliberately when the assumptions behind it change.

Architecture decision records: template and example

ADR-NNN: Title

Status: proposed | accepted | superseded

Context

Forces, constraints and numbers

Decision

What we will do, in one paragraph

Options considered

A, B, C with trade-offs

Consequences

Positive, negative and follow-ups

Review date

When to revisit

ADR-014: Iceberg for the curated layer

Status: accepted

Context

Spark, Trino and Flink must read the same tables; row deletes needed for erasure

Decision

Iceberg v2 tables, Parquet with zstd, REST catalog

Options

Delta with UniForm; Hudi; plain Parquet

Consequences

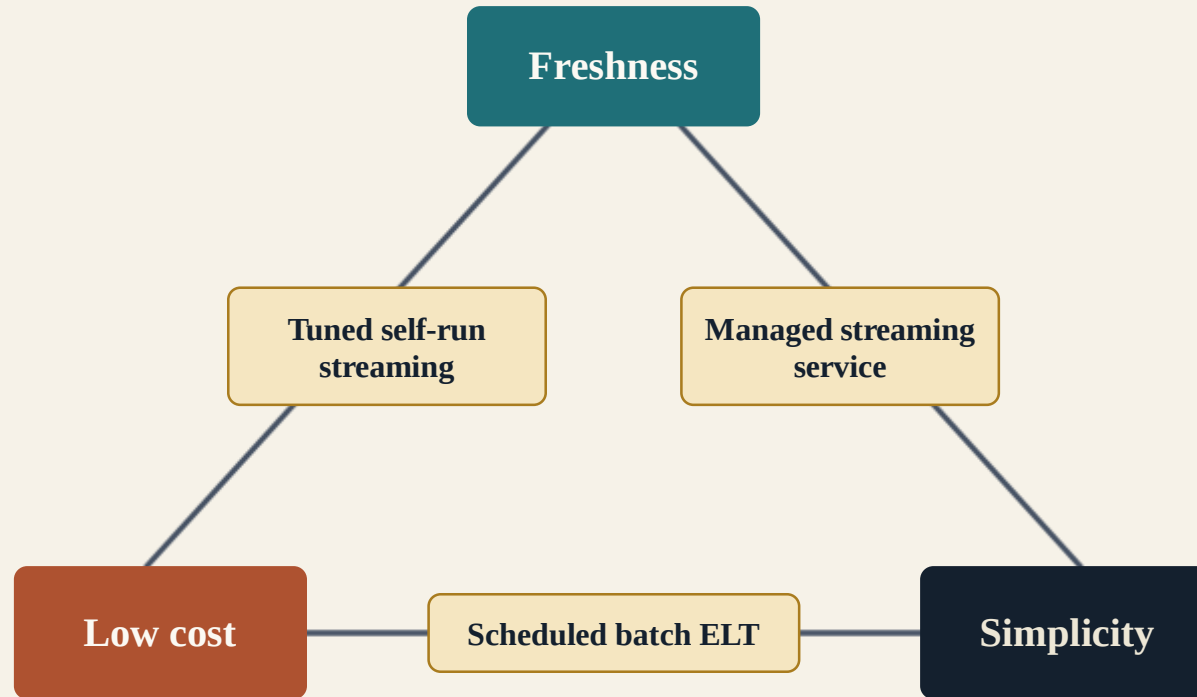
- + engine-neutral; hidden partitioning
- we own compaction and snapshot expiry

Review

After the first year of production

Key idea. A one-page ADR captures why, not just what. It turns a hallway decision into something the next team can challenge.

The freshness, cost and simplicity triangle



How to read it

Each corner is a goal. Any real design sits nearer two corners and pays in the third.

Pick deliberately

Most analytics needs minutes, not milliseconds. Paying for seconds you never use is the classic failure.

Revisit as the business changes

Freshness needs grow. Keep the batch path replaceable by a streaming one.

Key idea. Ask what the consumer does with a fresher result. If nothing changes, the extra freshness is pure cost.

Foundations checkpoint

A 500 MB/s ingest stream is how many TB per day?	→	43.2 TB per day
Why did ELT displace ETL on cloud platforms?	→	Cheap storage, elastic compute, raw retention
What does a 99% freshness SLO budget over 30 days?	→	7.2 hours of breach
Which style separates storage from compute?	→	Cloud warehouses and lakehouse engines
Name three decisions that are expensive to reverse	→	Formats, ownership boundaries, consistency guarantees
What is an ADR for?	→	Recording context, decision and consequences

Key idea. If any answer felt uncertain, revisit slides 13–15 before moving on to storage.

PART 2 OF 9

Storage and formats

Object stores, columnar files, partitioning and the layout decisions that set scan cost

- Object storage semantics and pricing
- Parquet internals, encodings and statistics
- Partitioning, clustering, lifecycle and evolution

DEPTH LADDER · THIS PART IS LEVEL 2

L1 · Concepts

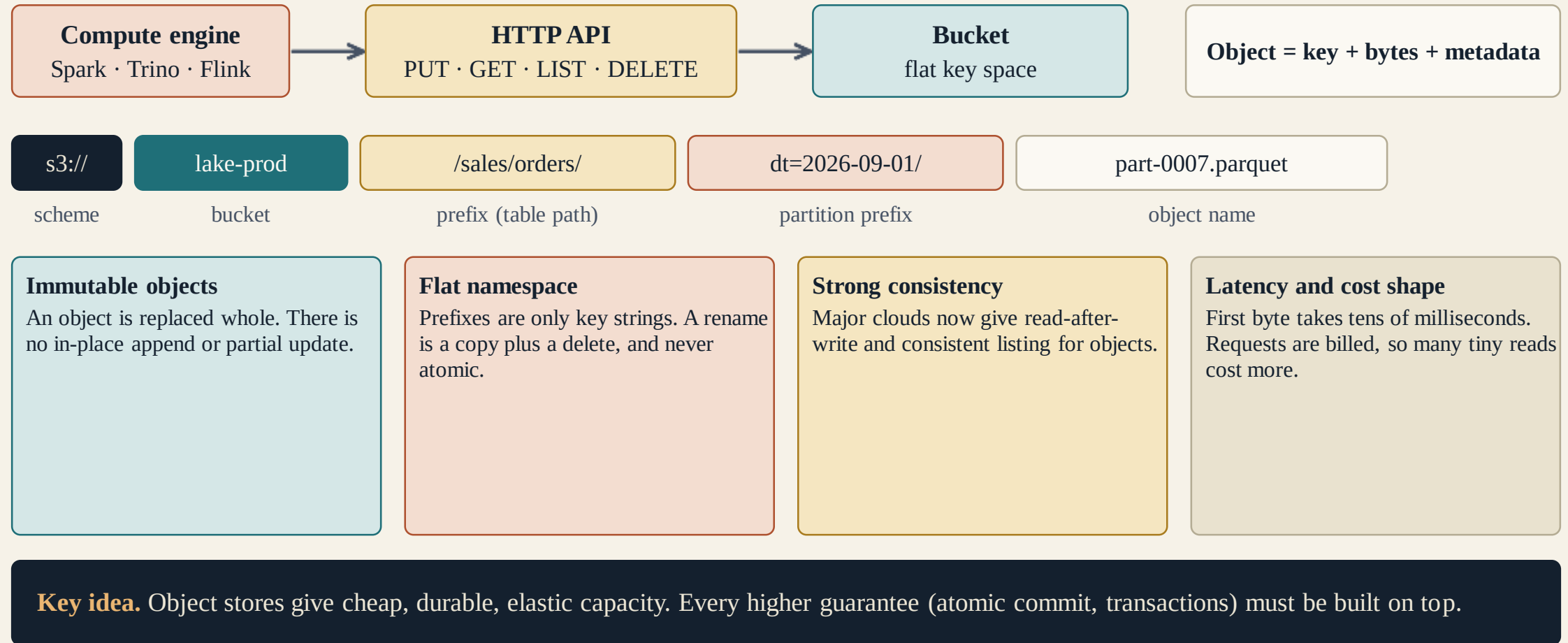
L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

Object storage: the substrate under every modern platform



Row layout versus column layout

Logical table: 4 rows, 4 columns

id	ct	amt	ts
id1	ct1	amt1	ts1
id2	ct2	amt2	ts2
id3	ct3	amt3	ts3
id4	ct4	amt4	ts4

Row layout: bytes of each row sit together

id1	ct1	amt1	ts1	id2	ct2	amt2	ts2	id3	ct3	amt3	ts3	id4	ct4	amt4	ts4
-----	-----	------	-----	-----	-----	------	-----	-----	-----	------	-----	-----	-----	------	-----

SUM(amt) must read all 16 cells to use 4 values: 100% of bytes

Column layout: bytes of each column sit together

id1	id2	id3	id4	ct1	ct2	ct3	ct4	amt1	amt2	amt3	amt4	ts1	ts2	ts3	ts4
-----	-----	-----	-----	-----	-----	-----	-----	------	------	------	------	-----	-----	-----	-----

SUM(amt) reads 4 contiguous cells: 25% of bytes, and similar values compress well

Row store

Best for OLTP: read or write a whole record with one seek. Used by PostgreSQL and MySQL heaps.

Column store

Best for OLAP: scan few columns over many rows, with vectorised execution and strong compression.

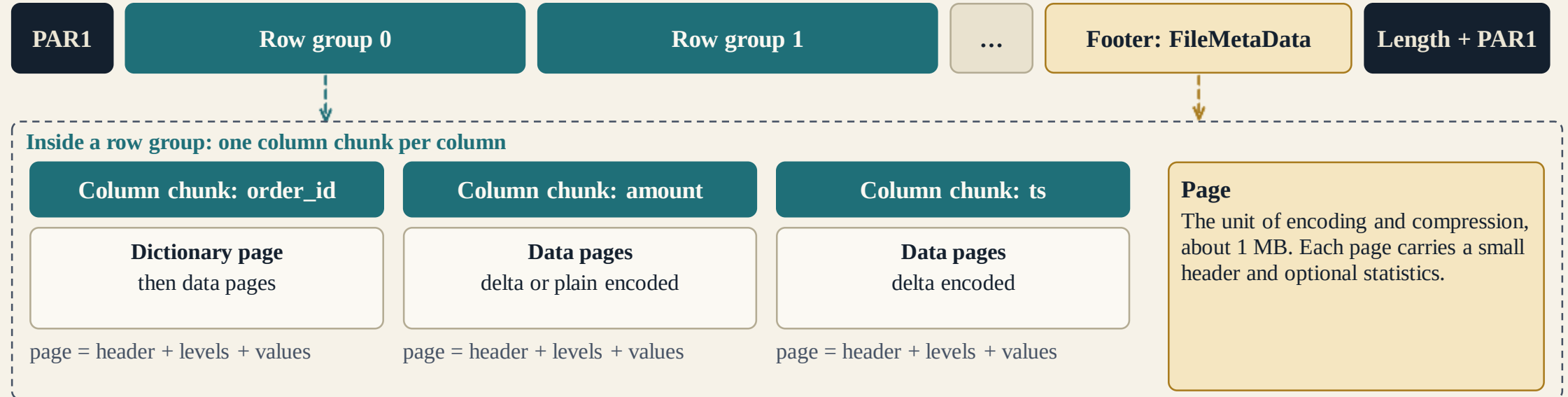
Hybrid (PAX, Parquet)

Rows are grouped into row groups, then columns are laid out inside each group. Balances both.

Key idea. Analytical queries touch few columns and many rows. Columnar layout reads only what the query needs.

Inside a Parquet file: row groups, column chunks and pages

Parquet file, left to right on disk



Key idea. Read the footer first: one small range read reveals schema, row-group offsets and per-column min/max, so the engine fetches only the chunks it needs.

Encodings and compression: how columns shrink

Dictionary

Store each distinct value once. NL, US, US, NL becomes dictionary {0:NL, 1:US} plus ids 0,1,1,0.

Run-length

Repeated values collapse into pairs. AAAABBB becomes (A,4)(B,3). Best on sorted columns.

Delta

Store differences. 1000, 1002, 1005 becomes 1000, +2, +3. Ideal for timestamps and counters.

Bit-packing

Values below 8 need 3 bits, not 32. Often combined with dictionary ids.

Encoding choice is automatic per column chunk in most writers.

Compression codecs applied after encoding

Codec	Speed	Ratio	Typical use
Snappy	Very fast	Low	Hot data
LZ4	Very fast	Low	Hot data
ZSTD	Fast	High, tunable	Default choice
Gzip	Slow	High	Cold, legacy
None	n/a	1x	Already compressed

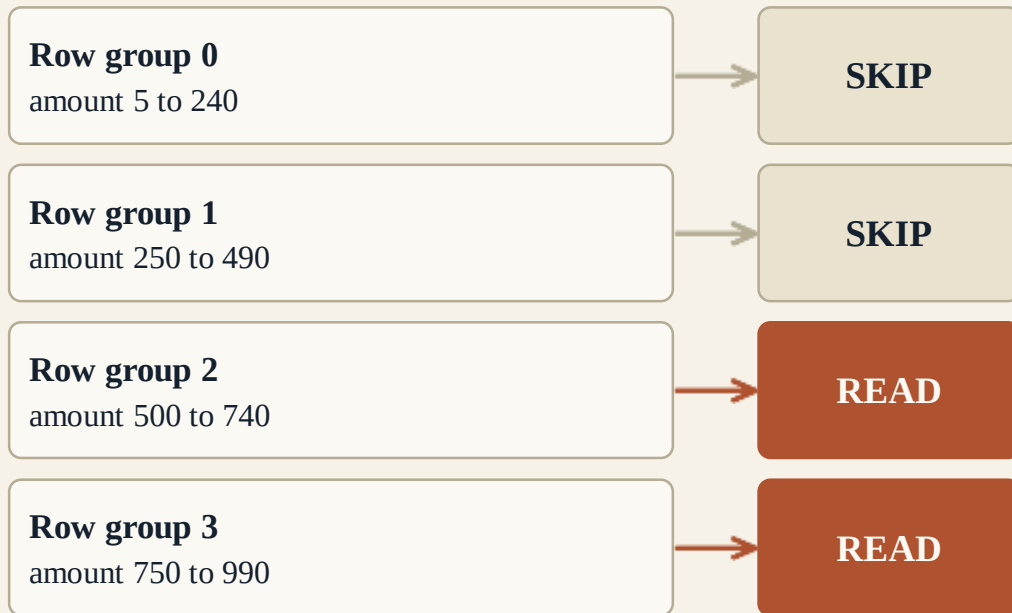
Two stages

Encoding removes redundancy within a column. A general codec then squeezes what remains. Columnar order helps both.

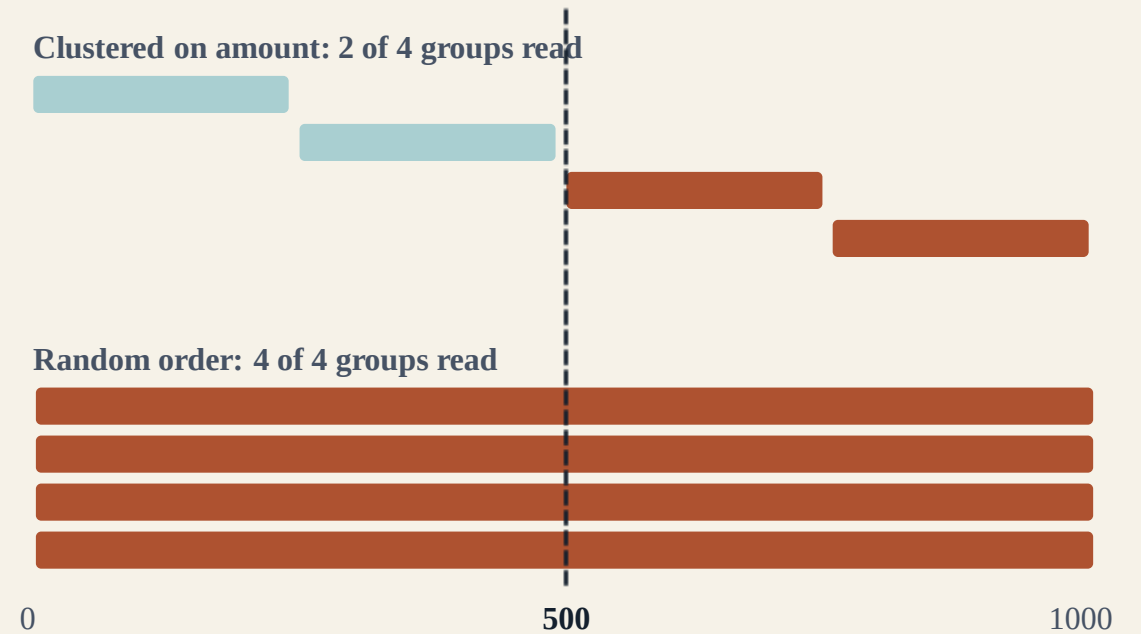
Key idea. Sort or cluster on low-cardinality columns first: runs get longer, encodings shrink, and scans read fewer bytes.

Predicate pushdown: min/max statistics skip row groups

Query: WHERE amount > 500



Value ranges per row group against the filter (amount = 500)



Key idea. Min/max skipping works only when values are clustered. The same file, written in random order, skips nothing.

File formats compared

Format	Layout	Schema	Best for	Watch out
CSV	Row, text	None	Simple interchange	No types, quoting bugs, slow scans
JSON / JSONL	Row, text	Implicit	APIs, logs, raw landing	Verbose, type drift, slow parsing
Avro	Row, binary	Embedded	Kafka events, evolving schemas	Reads all columns of a row
ORC	Columnar	Embedded	Hive and Trino estates	Narrower tool support than Parquet
Parquet	Columnar	Embedded	Analytics default everywhere	Small updates rewrite whole files
Arrow IPC	Columnar, in memory	Embedded	Zero-copy transfer between engines	Not a long-term storage format

Key idea. Use Avro or Protobuf on the wire, Parquet at rest, Arrow between processes, and JSON only at the edges.

Partitioning: directory layout and pruning

sales/

dt=2026-09-01/

part-0001.parquet

part-0002.parquet

dt=2026-09-02/

part-0001.parquet

dt=2026-09-03/

part-0001.parquet

Query

WHERE dt = '2026-09-02'

Only one directory is listed and read. The other days are pruned without opening a file.

Sizing a partition scheme for a 5 TB table

By day

365 partitions × 13.7 GB each. Good: large enough to scan efficiently.

By hour

8,760 partitions × 0.57 GB each. Borderline: watch file counts.

By user_id, 10 million users

10,000,000 partitions × 0.5 MB each. Bad: listing and open costs dominate.

Rule of thumb: at least 1 GB per partition and files of 128 MB to 1 GB.

Key idea. Partition on a low-cardinality column that queries filter on, usually time. Cluster within partitions for everything else.

The small-files problem, in numbers

Modelled scan time for 1 TB as file count grows (seconds)



Sources of small files: streaming micro-batches, over-partitioning, frequent tiny appends.

Model

$$T = n \times t_{\text{open}} / P + S / \text{BW}$$

$S = 1 \text{ TB}$, $\text{BW} = 10 \text{ GB/s}$, $P = 100$ readers, $t_{\text{open}} = 50 \text{ ms}$ per file.

Read the chart

Data time stays at 100 s. Only per-file overhead grows, and it dominates from about 1 MB files.

Key idea. Small files turn a throughput problem into a latency and request-count problem. Compact to 128 MB to 1 GB.

Clustering and Z-order: skipping on more than one column

A. Sorted by x only

f0	f1	f2	f3
f0	f1	f2	f3
f0	f1	f2	f3
f0	f1	f2	f3

B. Z-order on (x, y)

f0	f0	f1	f1
f0	f0	f1	f1
f2	f2	f3	f3
f2	f2	f3	f3

Filter	Sort by x	Z-order
x in {0,1}	2 of 4	2 of 4
y in {0,1}	4 of 4	2 of 4
x and y corner	2 of 4	1 of 4

Files read, out of 4

Each cell holds rows for one (x, y); f = data file. Dashed frame = query on the 2 × 2 corner.

Z-order and Hilbert curves

Interleave column bits so nearby points share files. Skipping stays useful on several columns.

Trade-off

Every added column dilutes each one. Two to four columns is typical. Rewrites cost compute.

Key idea. A single sort order favours one column. Space-filling curves spread skipping across several filter columns.

Bloom filters: skipping on point lookups

Bloom filter: 16 bits, 3 hash functions



Insert a → bits 2, 7, 11
Insert b → bits 5, 7, 14

Lookup c → bits 2, 9, 11
Bit 9 is 0: definitely absent

Lookup d → bits 5, 11, 14
All are 1: maybe present (false positive)

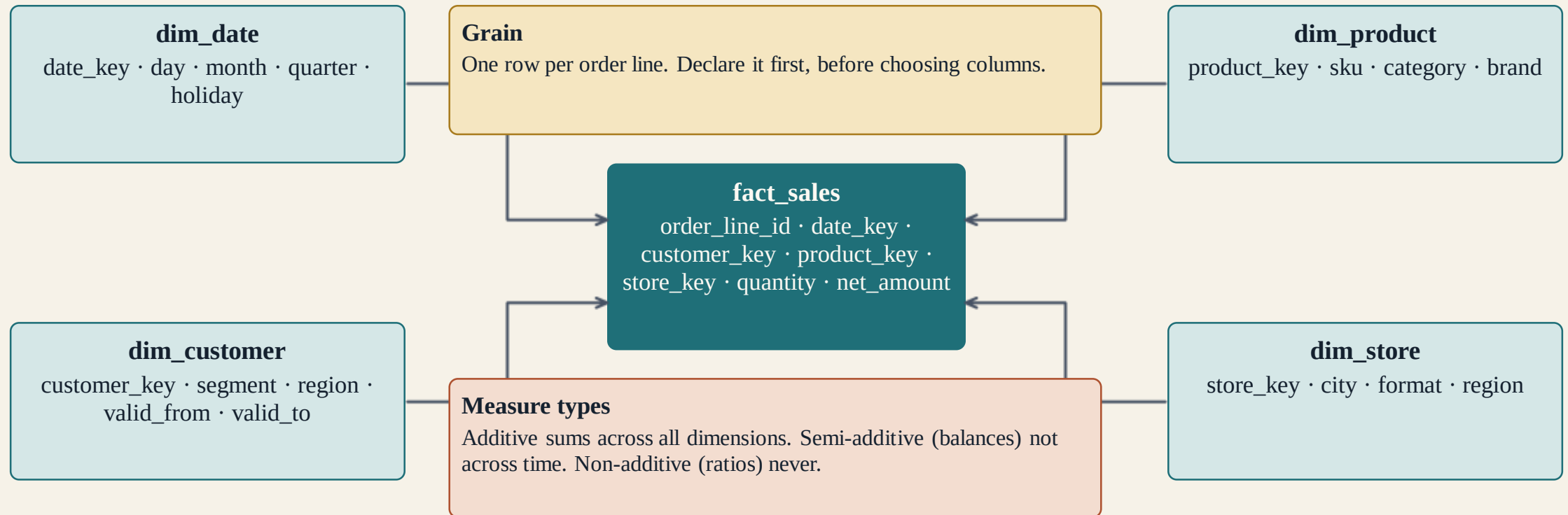
False positive rate $p \approx (1 - e^{(-kn/m)})^k$ Optimal $k = (m/n) \ln 2$ Bits needed $m = -n \ln p / (\ln 2)^2$

Target p	Bits per key	Hashes k	Size for 1M keys
10%	4.8	3	0.6 MB
1%	9.6	7	1.2 MB
0.1%	14.4	10	1.8 MB

Where used
Parquet and ORC per-column blooms for equality filters on high-cardinality columns such as user_id. HBase, Cassandra and LSM engines use them to skip SSTables.

Key idea. Bloom filters answer no with certainty and yes with a tunable error. About 10 bits per key buys a 1% false-positive rate.

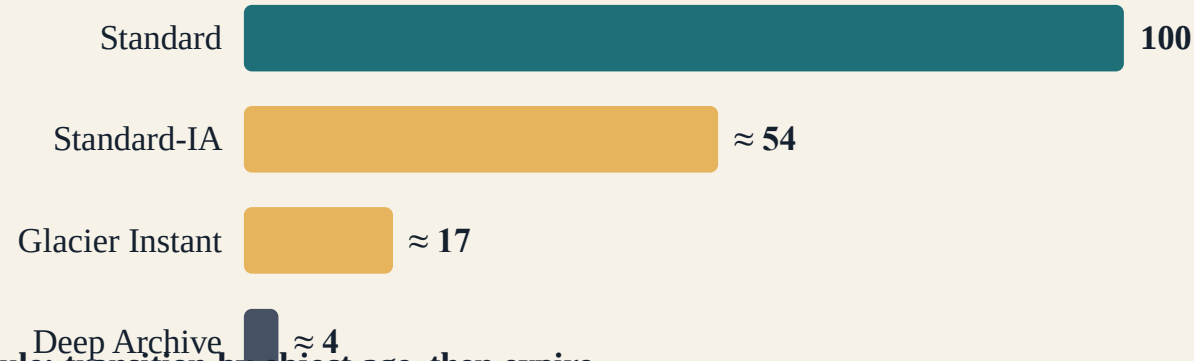
Dimensional modeling: the star schema



Key idea. Star schemas trade storage for simple joins. Surrogate keys and slowly changing dimensions keep history correct.

Lifecycle and storage tiers

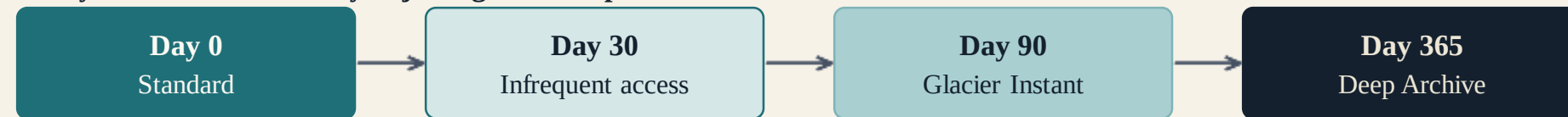
Relative storage price per GB-month (Standard = 100, approximate list prices)



Break-even

Move a byte to a colder tier if monthly saving exceeds expected retrieval cost: $(p_{\text{hot}} - p_{\text{cold}}) > \text{accesses/month} \times \text{retrieval fee}$.

Lifecycle rule: transition by object age, then expire



Cold tiers add minimum storage durations and retrieval delays of minutes to hours. Verify current terms and prices with your provider.

Key idea. Tier by access pattern, not age alone. Cold storage that is read often costs more than staying hot.

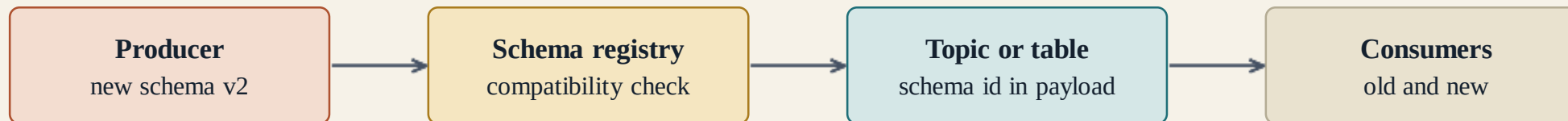
Schema evolution and compatibility rules

Which reader can handle which data? Backward: new reader, old data. Forward: old reader, new data.

Change	Backward	Forward	Full
Add field with default	Yes	Yes	Yes
Remove field that had a default	Yes	Yes	Yes
Add required field, no default	No	Yes	No
Remove required field	Yes	No	No
Rename field (no alias)	No	No	No
Widen int to long	Yes	No	No

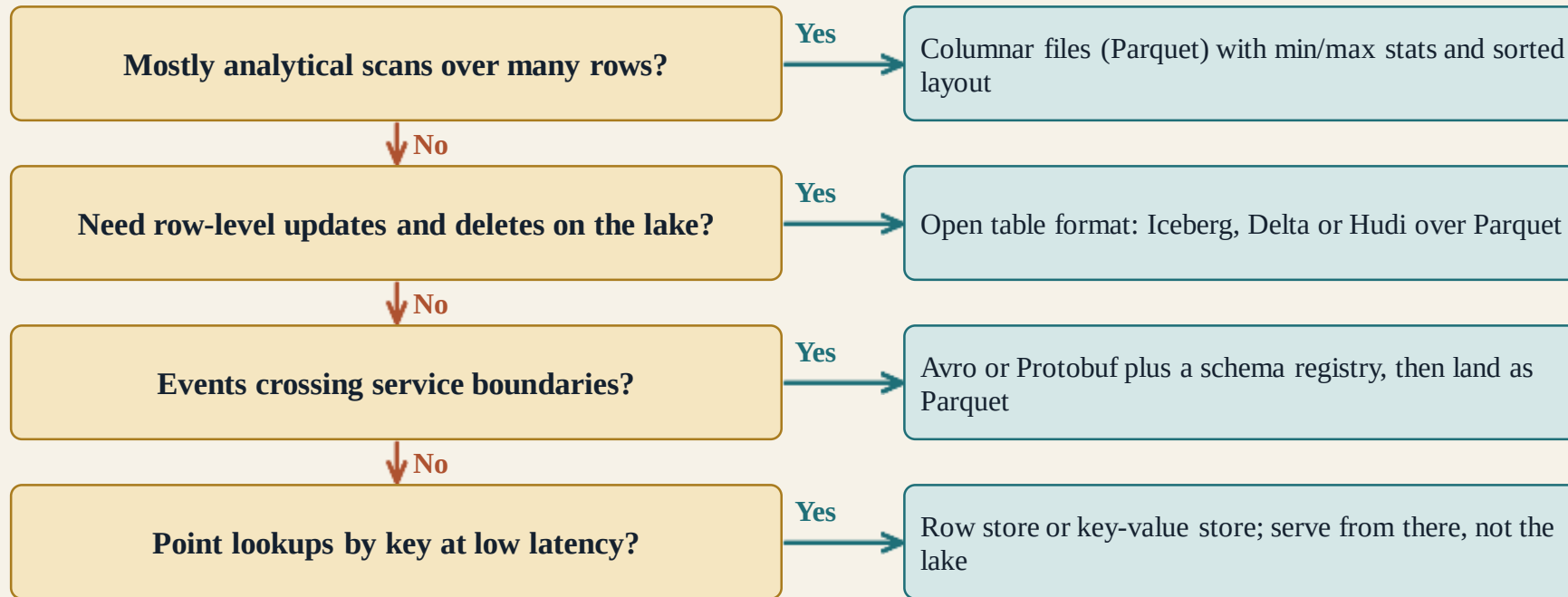
Registry modes

BACKWARD: new reads old.
FORWARD: old reads new. FULL:
both. NONE: unchecked. Pick per
subject.



Key idea. Evolve additively with defaults, and enforce compatibility in CI. Renames and type narrowing are breaking changes.

Choosing formats and layout: a decision path



Then set layout

Partition by time. Cluster by top filter columns. Target files of 128 MB to 1 GB. Compact on a schedule. Set lifecycle tiers.

Key idea. Choose format by access pattern first, then layout by query filters. Both are cheaper to get right before data volume grows.

PART 3 OF 9

Batch processing

Pipeline anatomy, orchestration, Spark execution, shuffles, skew and cost

- Idempotent, incremental pipelines
- Spark execution, shuffles and skew
- Sizing, critical paths and cost

DEPTH LADDER · THIS PART IS LEVEL 3

L1 · Concepts

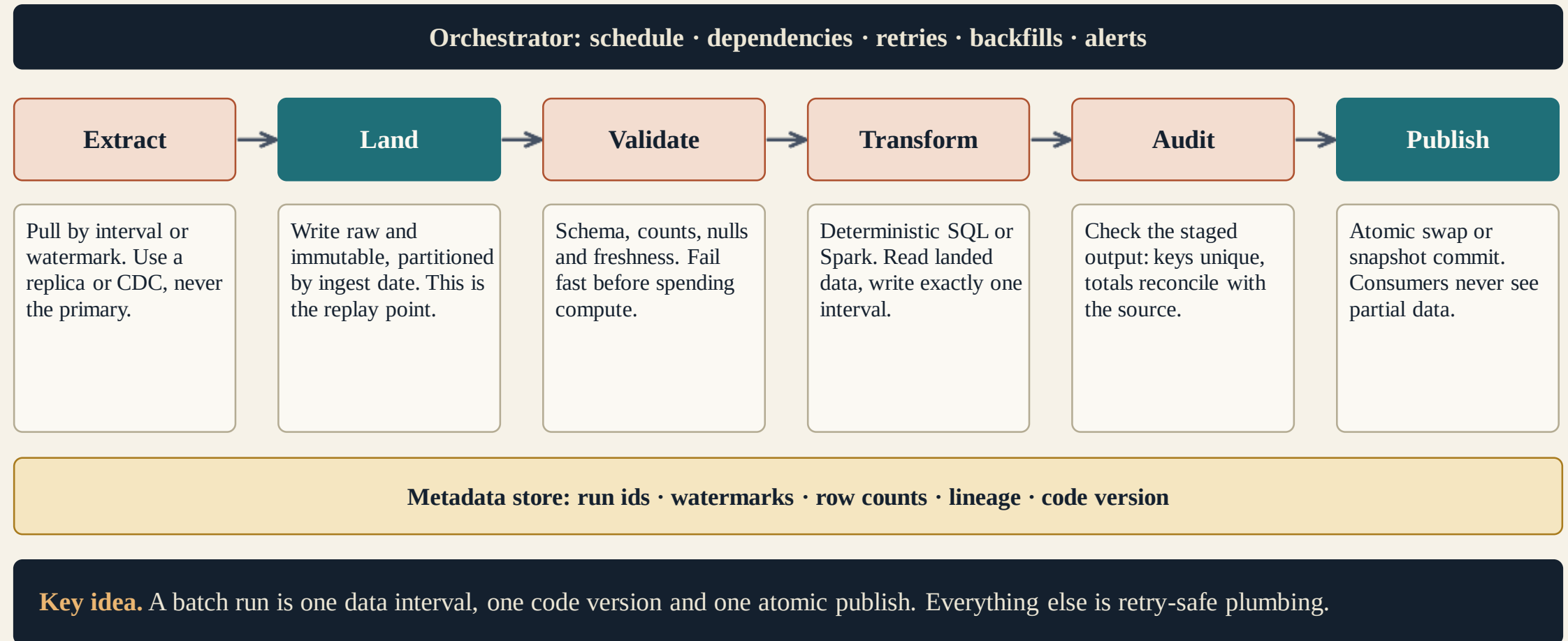
L2 · Structures

L3 · Mechanisms

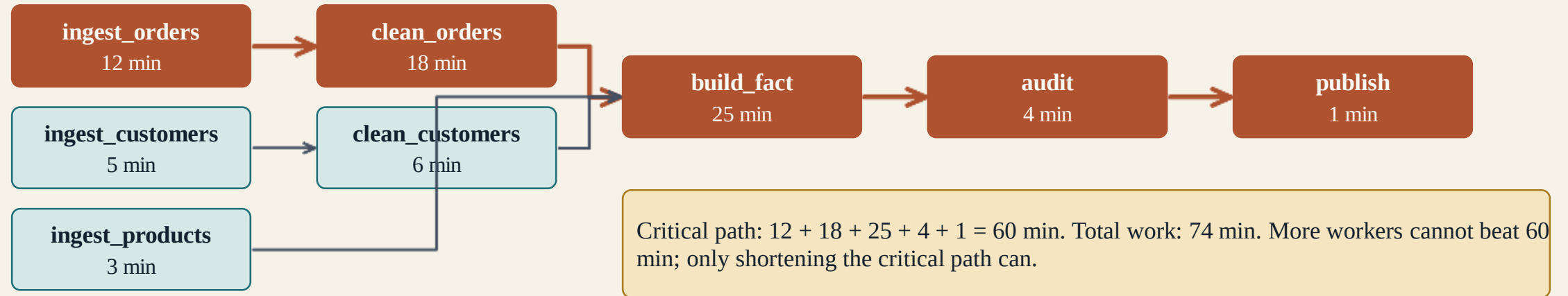
L4 · Internals

L5 · Quantitative systems

Anatomy of a batch pipeline



Orchestration: DAGs, intervals and the critical path



Data interval

The run for 2026-09-01 processes the half-open window [00:00, 24:00), whenever it actually starts.

Retries and backoff

Retry transient failures with exponential backoff.
Retry only idempotent tasks.

Sensors

Wait for upstream data or a partition to exist, not for a wall-clock time.

Key idea. Model pipelines as DAGs of idempotent tasks. Rust boxes are the critical path: optimise those first.

Idempotency: the property that makes batch safe

```
-- Unsafe: append
INSERT INTO sales_daily
  SELECT ds, store, SUM(amt)
  FROM raw_sales WHERE ds = '2026-09-01'
  GROUP BY ds, store;
-- Rerun after failure: duplicates
```

```
-- Safe: replace the interval
INSERT OVERWRITE sales_daily PARTITION (ds='2026-09-01')
  SELECT store, SUM(amt)
  FROM raw_sales WHERE ds = '2026-09-01'
  GROUP BY store;
-- Rerun: identical result
```

Deterministic

No now(), random() or unordered LIMIT inside transforms. Pass the interval in.

Replace, do not append

Overwrite the partition, or MERGE on a natural key.

Immutable inputs

Read the landed snapshot for the interval, never a moving latest.

Atomic publish

Swap partitions or commit one snapshot, so readers see old or new.

Key idea. Idempotent means running twice equals running once. It is what makes retries, reruns and backfills safe.

Incremental loading strategies

Strategy	How it works	Sees deletes	Late updates	Cost and risk
Full refresh	Reload everything each run	Yes	Yes	High: cost grows with table
High-watermark	Read rows with updated_at above last max	No	Only with lookback	Low, but blind to deletes
Log-based CDC	Read the database change log	Yes	Yes, ordered	Medium: connector operations
Snapshot diff	Compare snapshots by key or hash	Yes	Yes	High reads, simple logic
Partition replace	Reload the last N partitions	Within N	Within N	Medium, predictable

Key idea. Pick the cheapest strategy that still sees deletes and late updates. Watermarks alone silently miss both.

Late-arriving data and backfills

Event-date partitions, and daily runs that reprocess a 4-day lookback window



Choosing the lookback L

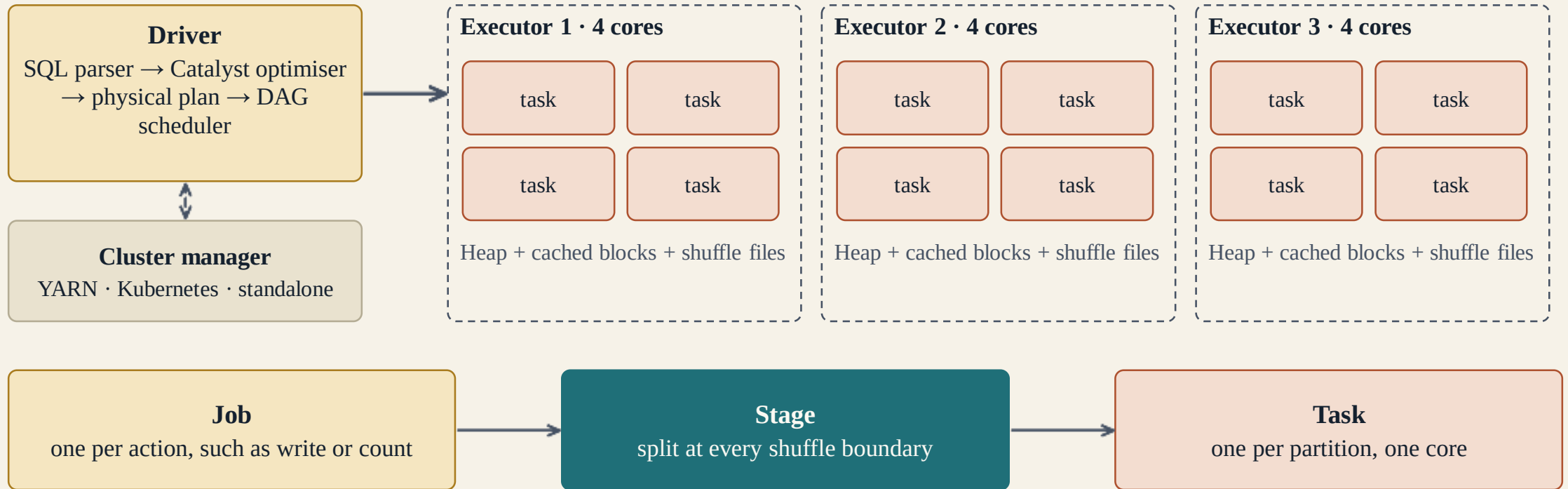
Measure lateness of events. If 99.9% arrive within 3 days, use $L = 4$ partitions. Cost per run rises with L times partition size.

Backfills

Rerun the same idempotent tasks for historical intervals with the code version of that time. Throttle to protect sources and downstream.

Key idea. Partition by event time, reprocess a lookback window, and treat backfill as a normal run over an older interval.

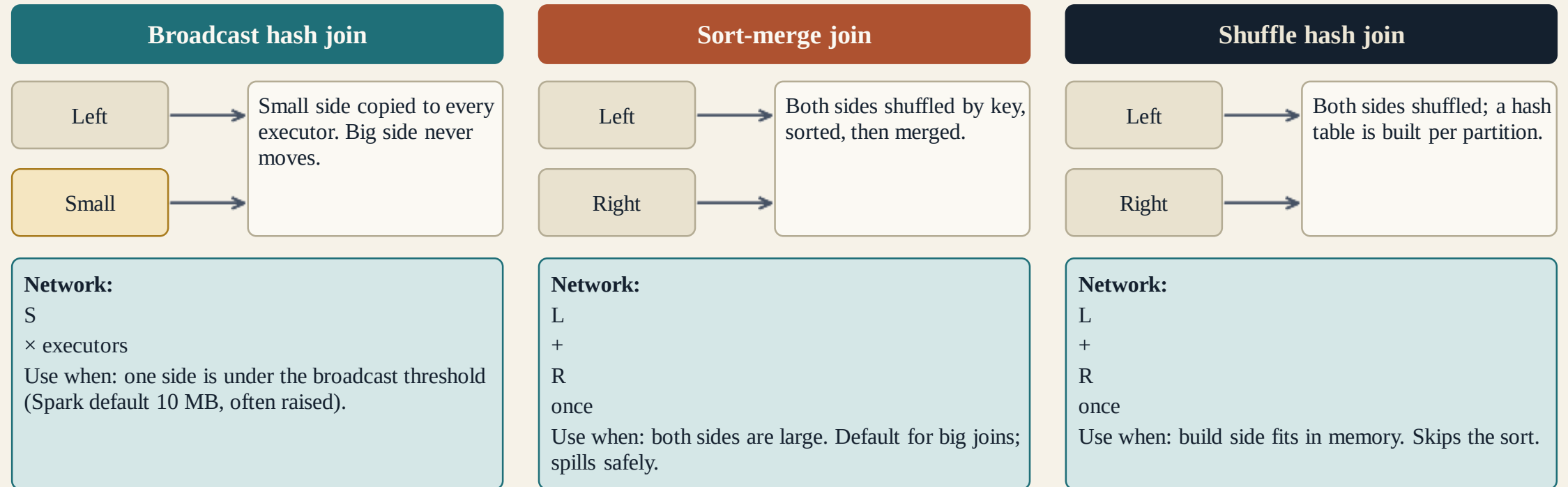
Spark execution model: driver, executors, stages and tasks



Parallelism = total cores. Tasks per stage = partitions. Waves = $\text{ceil}(\text{tasks} / \text{cores})$.

Key idea. Understand jobs, stages and tasks and you can read any Spark UI: shuffles split stages, partitions set the task count.

Join strategies and the cost of a shuffle

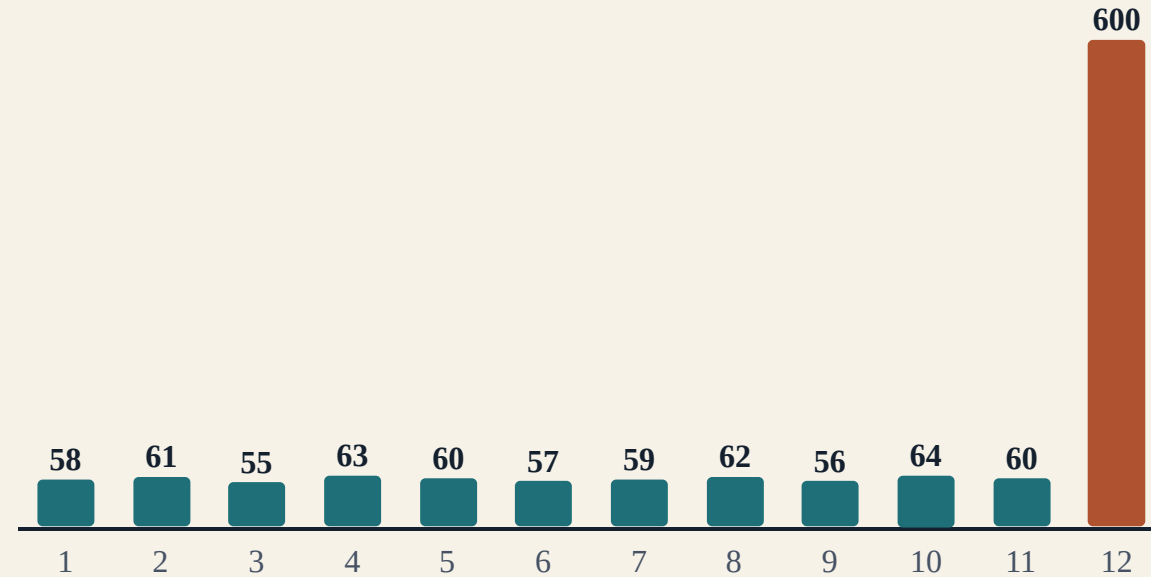


Example: 1 TB fact joined to a 500 MB dimension on 100 executors. Broadcast moves 50 GB; sort-merge moves about 1 TB, 20 times more.

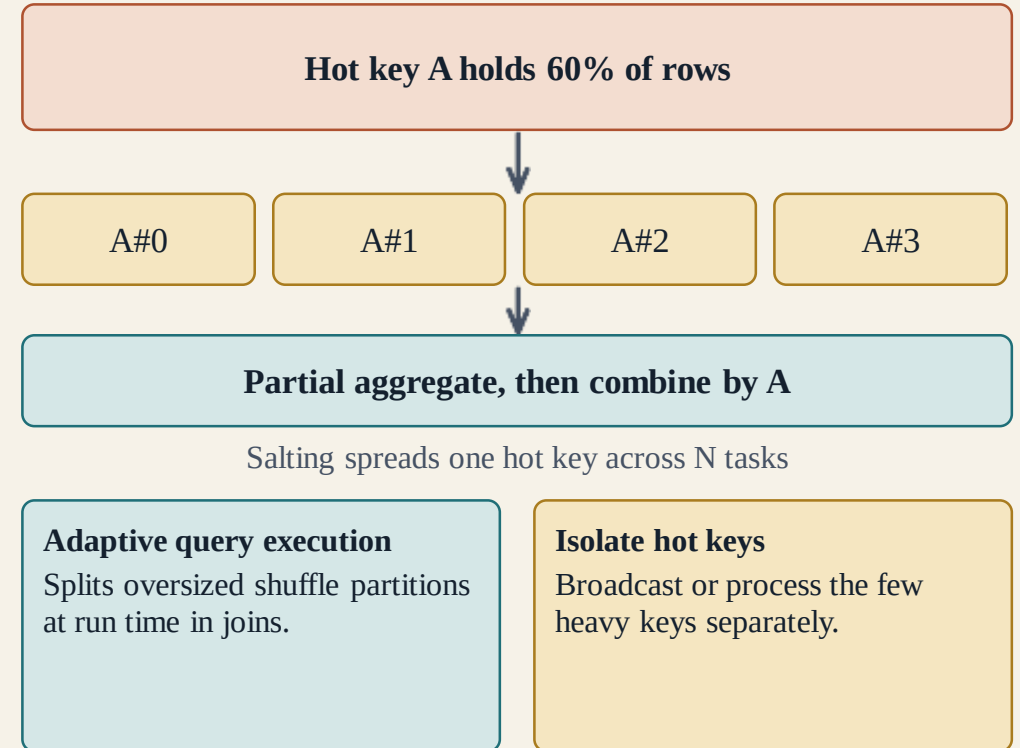
Key idea. Shuffles dominate cost. Prefer broadcast for small dimensions and reduce data before any large join.

Data skew: one slow task holds the stage

Task duration in one stage (seconds)



Stage time = slowest task: 600 s, not the 60 s typical



Key idea. Detect skew by comparing max to median task time. Fix it by splitting hot keys, not by adding machines.

Partition sizing: the arithmetic behind tuning

Shuffle partitions

$\approx \text{shuffle bytes} \div \text{target size}$. Aim for 128 to 256 MB. Example: $2 \text{ TB} \div 200 \text{ MB} = 10,000$.

Waves

$= \text{ceil}(\text{tasks} \div \text{total cores})$. 10,000 tasks on 500 cores = 20 waves.

Memory per task

$\approx \text{usable executor memory} \div \text{cores}$. A 32 GB, 4-core executor gives about 6 to 8 GB.

Task duration

Aim for one to a few seconds. Below about 200 ms, scheduling overhead dominates.

2 TB shuffle, 100 MB/s per core

Partition size	Tasks	Task time	Verdict
20 MB	100,000	0.2 s	Overhead
200 MB	10,000	2 s	Good
2 GB	1,000	20 s	Spill risk

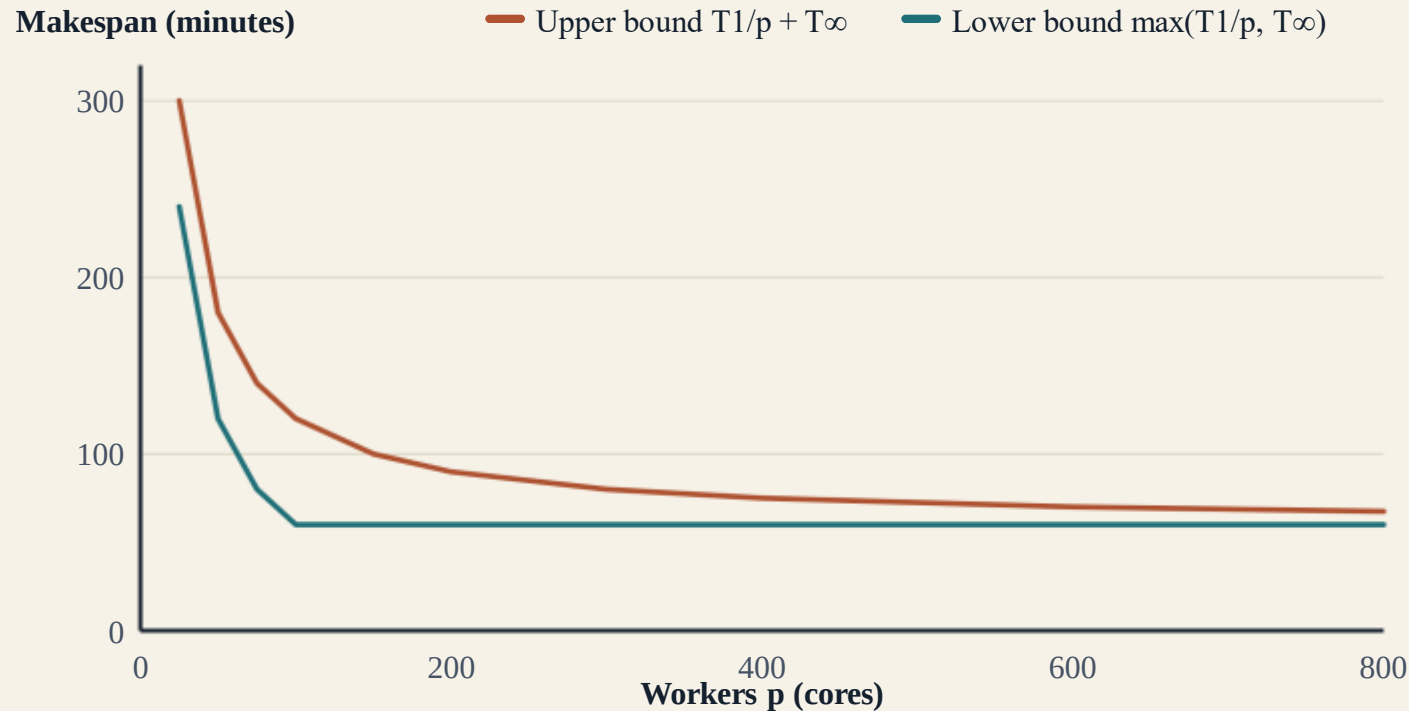
Adaptive execution

Spark can coalesce small shuffle partitions after each stage using runtime statistics, so a modest static setting is safe.

Rule: partitions of 100 to 250 MB, at least 2 to 3 times the core count.

Key idea. Partition size is the master knob: too big spills, too small drowns in overhead. Size from bytes, not from habit.

Critical path and cluster sizing



Work and span

$T1$ = total work = 6,000 core-minutes. T_{∞} = critical path = 60 minutes.

Graham bound

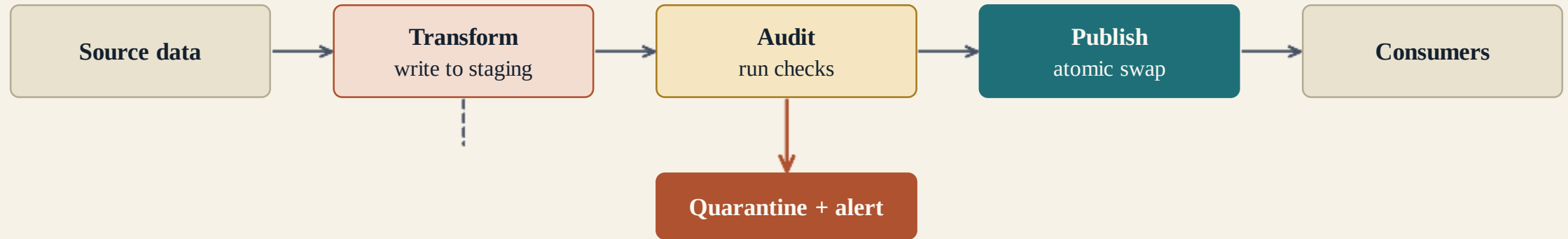
$T_p \leq T1/p + T_{\infty}$ for greedy scheduling. Also $T_p \geq \max(T1/p, T_{\infty})$.

Knee at $p = T1/T_{\infty} = 100$

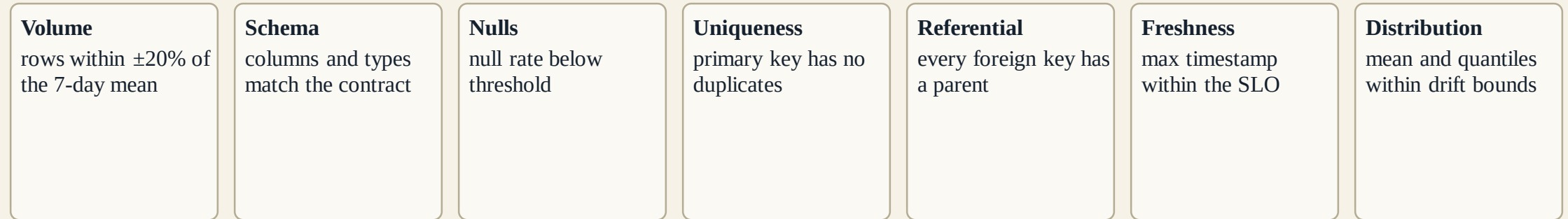
Past 100 cores the lower bound stays at 60 minutes.

Key idea. Extra workers help only until total work divided by workers falls to the critical path. Then restructure the DAG.

Write, audit, publish: quality gates in batch

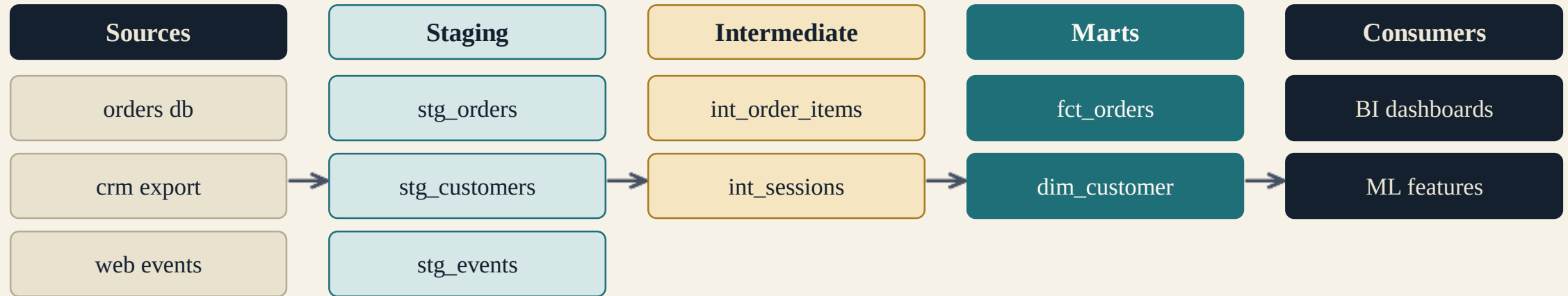


Seven automated checks in the audit step



Key idea. Write to staging, audit, then publish. A failed audit blocks publication and leaves the last good data serving.

Transformation layers: staging, intermediate and marts



Staging

One to one with a source. Rename, cast, deduplicate. No joins. Materialise as views.

Intermediate

Joins and business rules in small, reusable steps. Often ephemeral or tables.

Marts

Dimensional models at one grain each, tested and documented. Materialise incrementally.

Key idea. Layered models keep lineage readable, tests local and blast radius small. Each layer has one job.

The economics of batch clusters

Job: 400 core-hours of work, 0.5 h serial part, \$0.05 per core-hour

Workers p	Runtime	Core-hours	Cost
50	8.5 h	425	\$21.25
100	4.5 h	450	\$22.50
200	2.5 h	500	\$25.00
400	1.5 h	600	\$30.00
800	1.0 h	800	\$40.00

Cost = price × (p × t_{serial} + W). Speed is bought at price × t_{serial} per extra worker.

Cost in dollars



Spot or preemptible capacity is often far cheaper than on-demand and suits idempotent batch.

Key idea. Runtime falls as 1/p but cost rises linearly with the serial part. Pick the cheapest cluster that meets the deadline.

Batch failure modes: a diagnostic runbook

Symptom	Likely cause	First check	Fix
Job slower every week	Data growth or small files	Input size and file count trend	Compact, repartition, prune
One task never finishes	Key skew	Max versus median task time	Salt or isolate keys, adaptive execution
Executor out of memory	Huge partitions, collect, wide rows	Spill and partition size metrics	More partitions, avoid collect
Duplicates after retry	Non-idempotent append	Row counts per run	Overwrite interval or MERGE
Yesterday's data missing	Late upstream, wrong interval	Upstream freshness and run window	Sensors plus lookback
Sudden cost spike	Full refresh, bad join, retry loop	Query plan and run counts	Incremental logic, join keys, retry caps

Key idea. Diagnose from symptom to metric to fix. Most batch incidents repeat: growth, skew, memory, duplicates, lateness and cost.

PART 4 OF 9

Streaming

Logs, partitions, event time, watermarks, state, checkpoints and exactly-once delivery

- The log, partitions and replication
- Event time, windows and watermarks
- State, checkpoints, lag and delivery guarantees

DEPTH LADDER · THIS PART IS LEVEL 3

L1 · Concepts

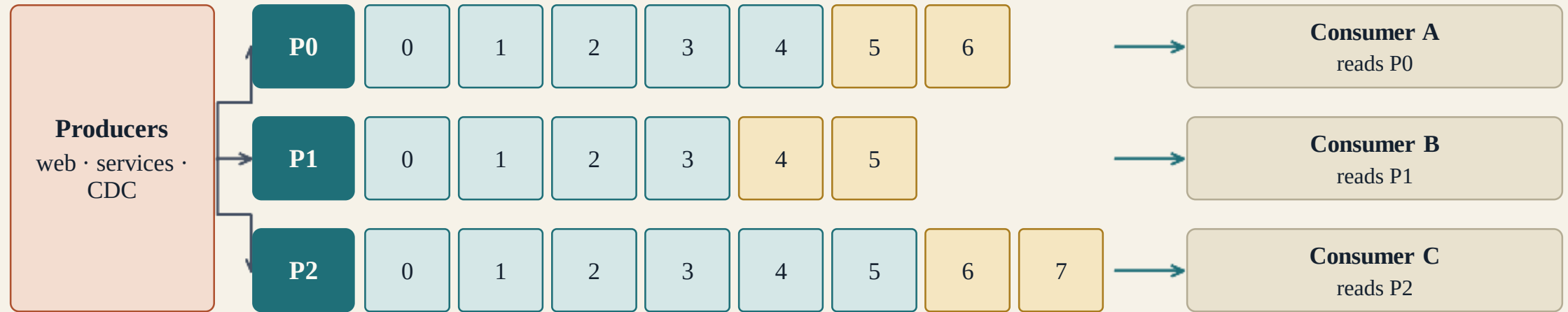
L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

The log abstraction: topics, partitions and offsets



Teal offsets are consumed and committed. Gold offsets are waiting.

Append-only

Records are added at the end. Retention is by time or size, and reading never deletes.

Offsets

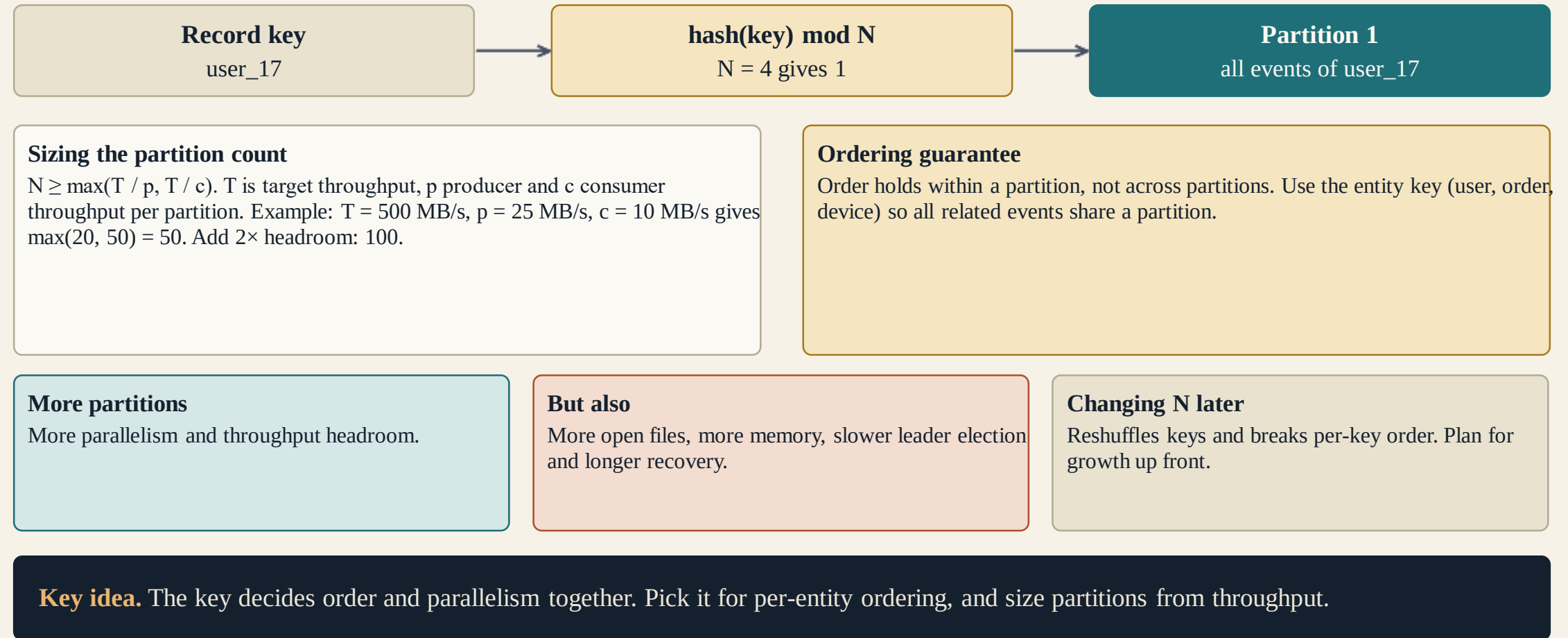
A position inside one partition. Each consumer group tracks its own.

Consumer groups

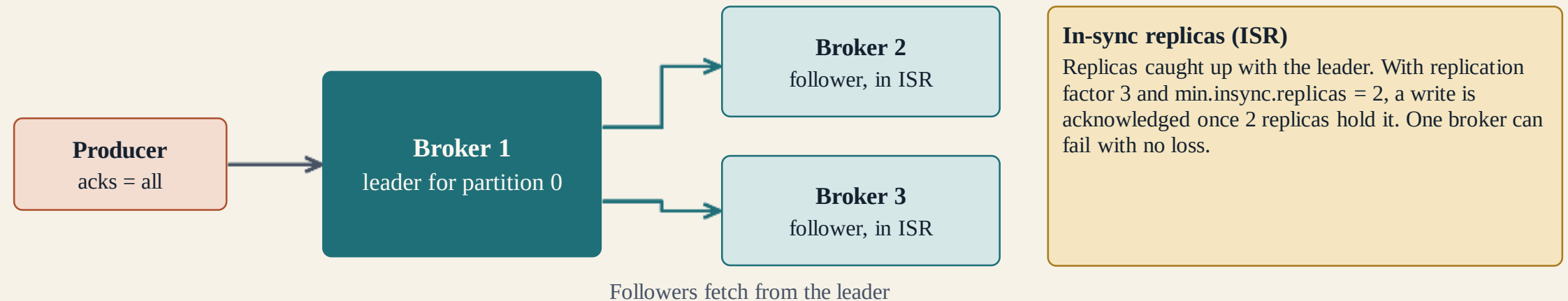
Each partition is read by one member of a group. Groups are independent.

Key idea. The log turns messaging into storage: durable, replayable and shareable by many consumers at their own pace.

Partitioning: keys, ordering and how many partitions



Replication and durability: leaders, followers and the ISR



Setting	Meaning	Risk
acks = 0	Fire and forget	Loss on any failure
acks = 1	Leader wrote it	Loss if leader dies first
acks = all	All in-sync replicas wrote it	Higher latency

Availability

Writes continue while at least `min.insync.replicas` replicas are in sync. Otherwise producers get errors, by design.

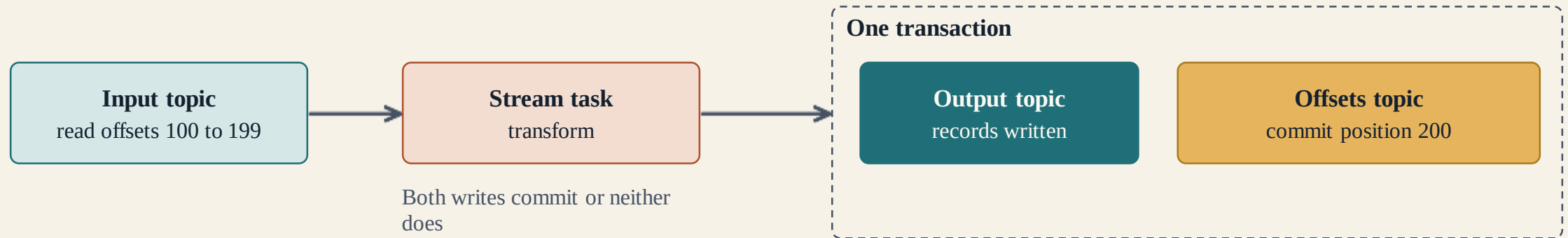
Key idea. Durability is a contract: replication factor, ISR size and acks together define what an acknowledgement means.

Delivery semantics: what a crash does

Semantics	Order of steps	Failure result	Use when
At most once	Commit offset, then process	Loss if crash after commit	Metrics where loss is tolerable
At least once	Process, then commit offset	Duplicates if crash before commit	Default; pair with idempotent sinks
Exactly-once effect	Process and commit atomically, or dedupe at sink	None visible, higher cost	Payments, counters, billing

Key idea. Exactly-once is an effect built from at-least-once delivery plus idempotence or atomic commits. Choose it per pipeline.

Idempotent producers and transactions



Idempotent producer

Producer id plus a sequence number per partition. The broker discards duplicates caused by retries.

Transactions

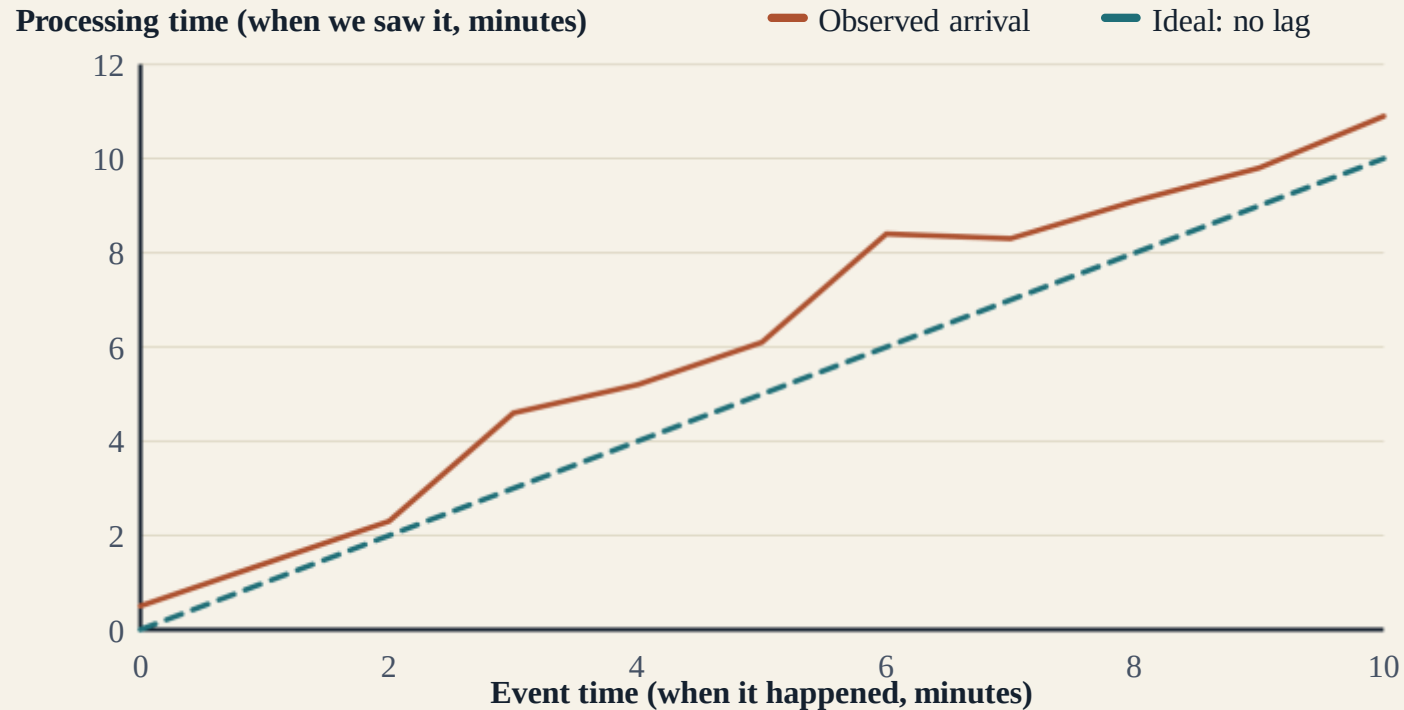
Atomic writes across partitions plus the offset commit, tracked by a transaction coordinator.

read_committed

Consumers skip aborted and open transactions. Visibility waits for the commit.

Key idea. Retries stop creating duplicates, and read-process-write loops become atomic. The price is commit latency.

Event time versus processing time



Event time

When the thing happened, stamped by the source device or service.

Processing time

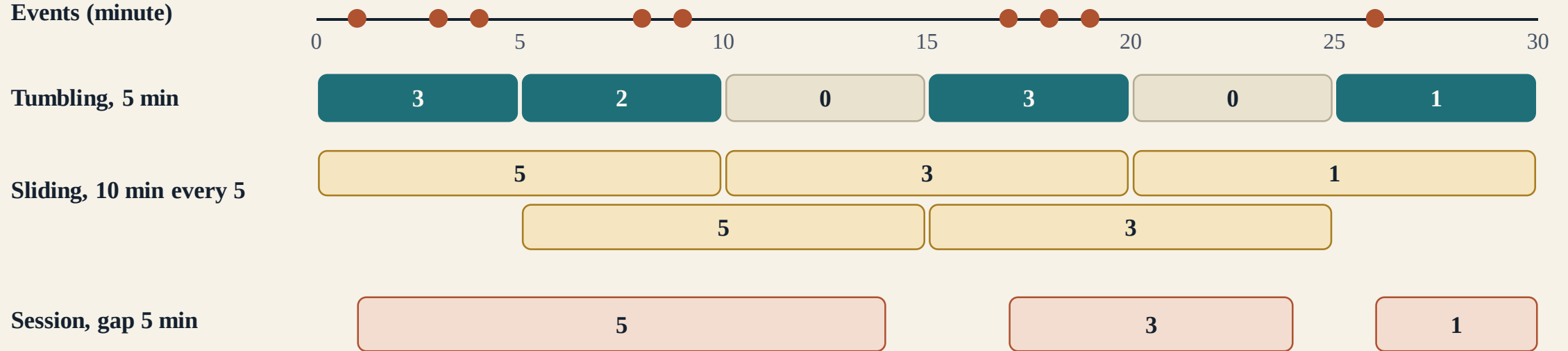
When the pipeline observed it. Easy to use, wrong for correctness.

Skew = vertical gap

Causes: network delay, offline mobile devices, backlogs, retries. It varies, so windows on processing time give unstable answers.

Key idea. Compute on event time, and accept that you must decide when a result is complete enough to emit.

Windows: tumbling, sliding and session



Fixed windows

Tumbling: each event in exactly one window. Sliding: in size $\div$ slide windows, so counts overlap.

Data-driven windows

Session windows close after a gap with no events, so length varies per key. They merge as late events bridge gaps.

Key idea. Choose the window for the question: tumbling for reports, sliding for trends, session for behaviour.

Watermarks and late data, step by step

Watermark = max event time seen – 2 minutes (bounded out-of-orderness)

Arrival	Event time	Max seen	Watermark	Result
e1	10:01	10:01	09:59	Buffered
e2	10:04	10:04	10:02	Buffered
e3	10:03	10:04	10:02	Buffered (out of order)
e4	10:06	10:06	10:04	Next window
e5	10:07	10:07	10:05	Fire [10:00, 10:05): 3 events
e6	10:02	10:07	10:05	Late: below watermark

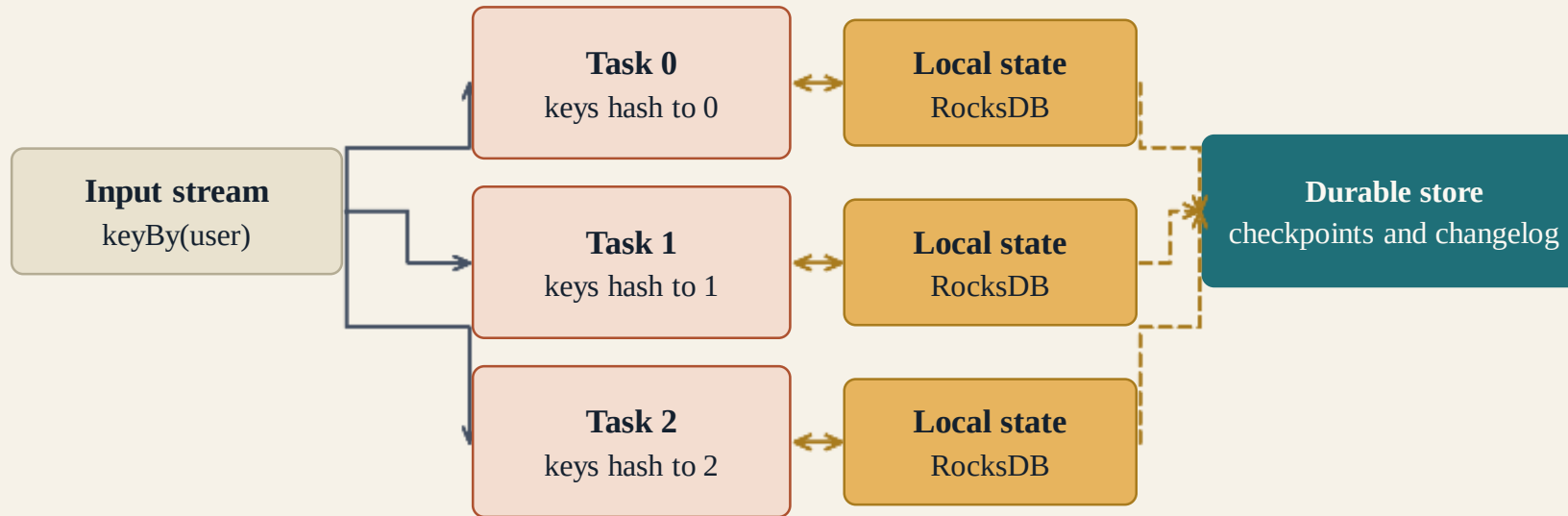
Larger lateness gives more complete answers and higher latency.

Key idea. A watermark is a promise about completeness. Tune it between latency and accuracy, and always plan for late events.

Watermark meaning
No event older than this is expected. A window fires when the watermark passes its end.

Late data options
Drop it, send it to a side output, or allow lateness and emit corrected results.

State management: keyed, local and check-pointed



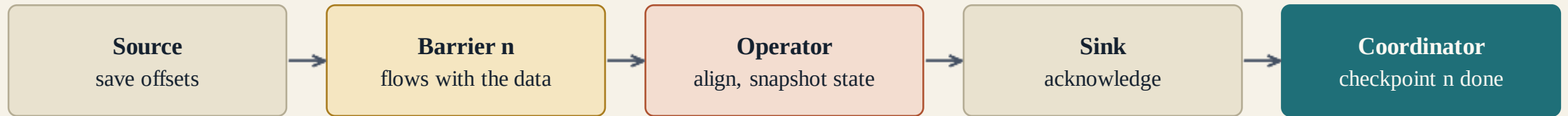
Sizing state

Size $\approx$ distinct keys $\times$ bytes per key $\times$ overhead. Example: 50 M users $\times$ 200 B $\times$ 1.5 = 15 GB. Across 30 tasks about 0.5 GB each.

State types: value, list, map. Set a TTL so idle keys expire, or state grows without bound.

Key idea. State makes streaming powerful and dangerous. Size it, expire it, and keep it local, checkpointed and rebuildable.

Checkpointing with barriers, and recovery time



Consistent snapshot

Barriers split the stream into before and after. Snapshotting every operator at its barrier gives a cut equal to a moment in the past, without stopping the job (Chandy–Lamport).

Recovery

Restart from the last complete checkpoint: restore state, rewind sources to saved offsets, replay. Output after that point is recomputed.

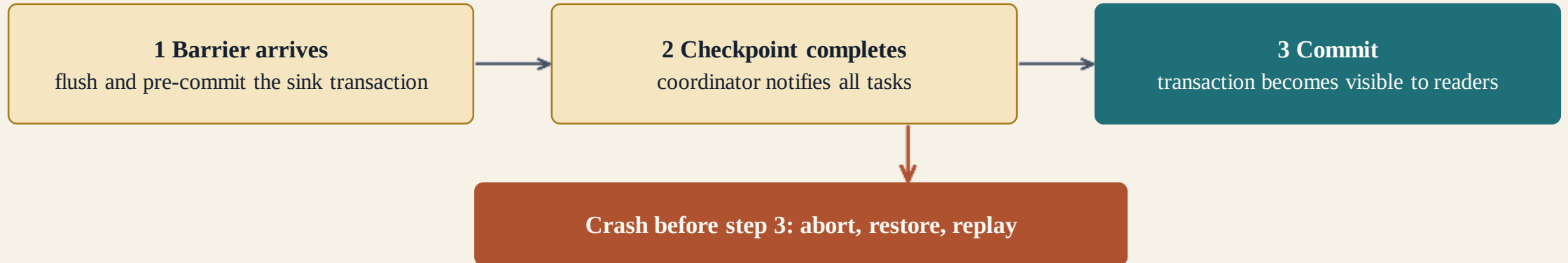
Worst-case recovery $\approx$ restore + replay. 20 GB state at 500 MB/s restores in 40 s. A 5-minute interval means up to 300 s of input to replay. If capacity is 2 $\times$ arrival, catch-up takes 300 s, so about 340 s in total. Shorter intervals cut replay but add snapshot overhead.

Key idea. Checkpoints buy exactly-once state and fast recovery. Interval is the dial between overhead and replay time.

Exactly-once end to end



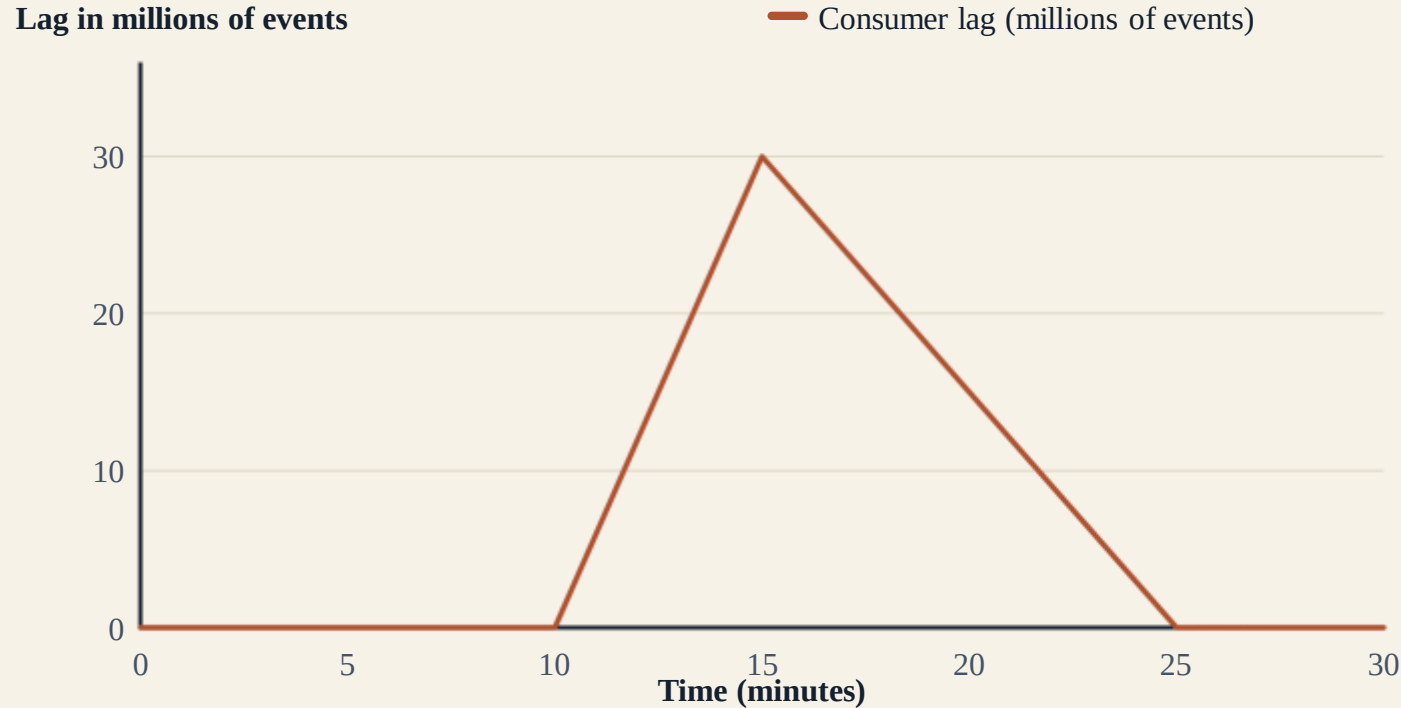
Two-phase commit at the sink, driven by checkpoints



Alternative: idempotent writes by primary key make replays harmless.

Key idea. End-to-end exactly-once needs three parts working together: replayable input, checkpointed state and an atomic or idempotent output.

Consumer lag, backpressure and drain time



Worked example

Arrival $\lambda = 100\text{k/s}$, capacity $\mu = 150\text{k/s}$. Outage 5 min:
lag = $100\text{k} \times 300\text{ s} = 30\text{ M}$. Drain time = lag $\div (\mu - \lambda) = 30\text{ M} \div 50\text{k/s} = 600\text{ s}$.

Headroom

Keep utilisation $\rho = \lambda/\mu$ at or below about 70%.

Backpressure

Bounded buffers pass slowness upstream instead of dropping data.

Key idea. Lag is the streaming health metric. Drain time depends on spare capacity, so run with headroom.

Stream joins and their state cost

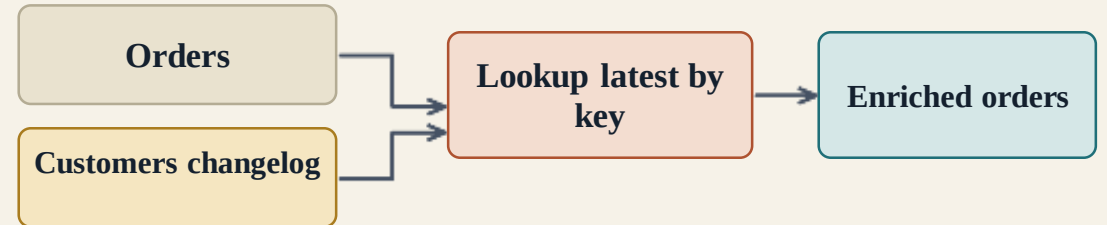
Stream-stream windowed join



State cost

Both sides are buffered for the window. $10\text{k events/s} \times 1,800\text{ s} \times 500\text{ B} = 9\text{ GB}$. State grows with rate and window length. Watermarks clean it up.

Stream-table enrichment



State cost

Latest row per key from a compacted changelog. $20\text{ M customers} \times 1\text{ KB} = 20\text{ GB}$, partitioned by key or replicated to each task. Late dimension updates cause temporal inconsistency.

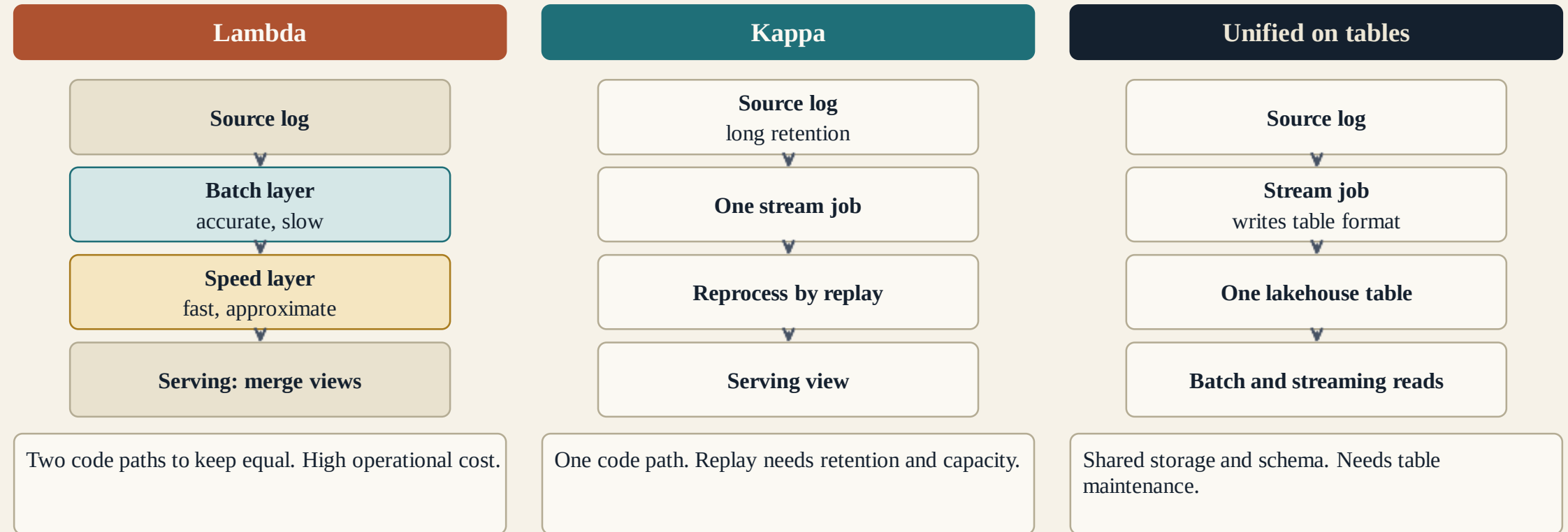
Key idea. Joins turn streams into state. Bound the window, key both sides identically, and budget the memory.

Stream engines compared

Engine	Model	Typical latency	Best for
Apache Flink	True streaming, event time, keyed state, checkpoints	Milliseconds to seconds	Stateful low-latency pipelines
Kafka Streams	Library inside your service, state stores backed by changelog topics	Milliseconds	Kafka-to-Kafka microservices
Spark Structured Streaming	Micro-batches by default, same API as batch	Hundreds of ms to seconds	Unify batch and stream code
Apache Beam	Portable model with runners such as Dataflow and Flink	Depends on runner	Portability across engines

Key idea. Match the engine to the team and the latency need: libraries for services, Flink for heavy state, Spark for batch parity.

Lambda, Kappa and unified architectures



Key idea. Modern platforms lean on unified tables: one copy, one schema, and both batch and streaming readers.

PART 5 OF 9

Lakehouse

How open table formats give warehouse guarantees on cheap object storage

- Table-format internals: Iceberg, Delta, Hudi
- Transactions, concurrency and maintenance
- Modelling, performance and choosing an architecture

DEPTH LADDER · THIS PART IS LEVEL 4

L1 · Concepts

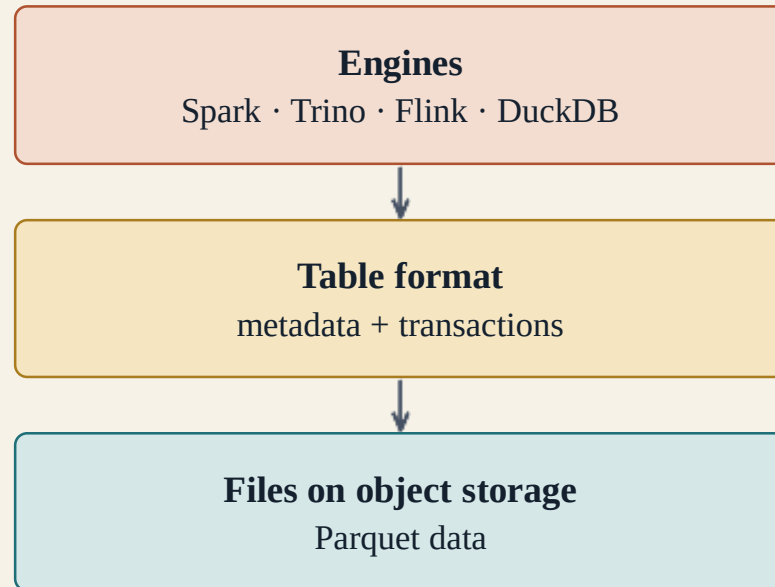
L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

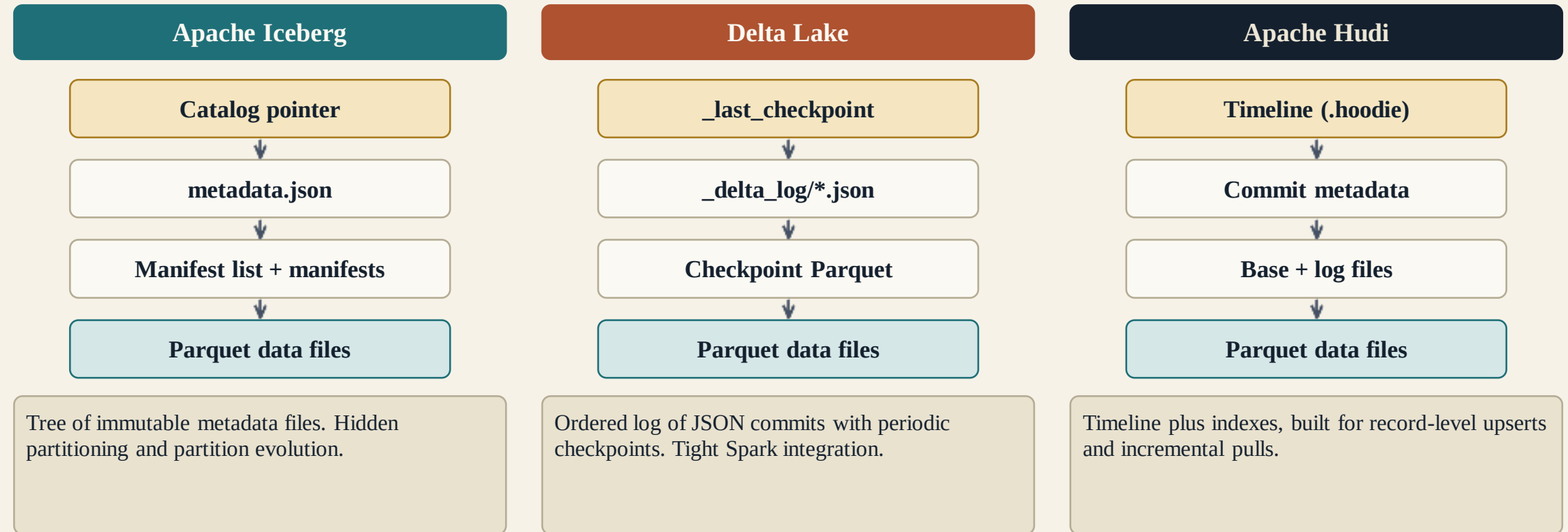
Why a lakehouse: adding a table layer to files



Need	Raw Parquet files	Open table format
Atomic commit	Readers see partial writes	One metadata swap
Schema control	By convention	Enforced and versioned
Row-level change	Rewrite files by hand	UPDATE, DELETE, MERGE
Concurrent writers	Unsafe	Optimistic concurrency
Time travel	None	Snapshots
File listing	Slow LIST calls	Manifest metadata

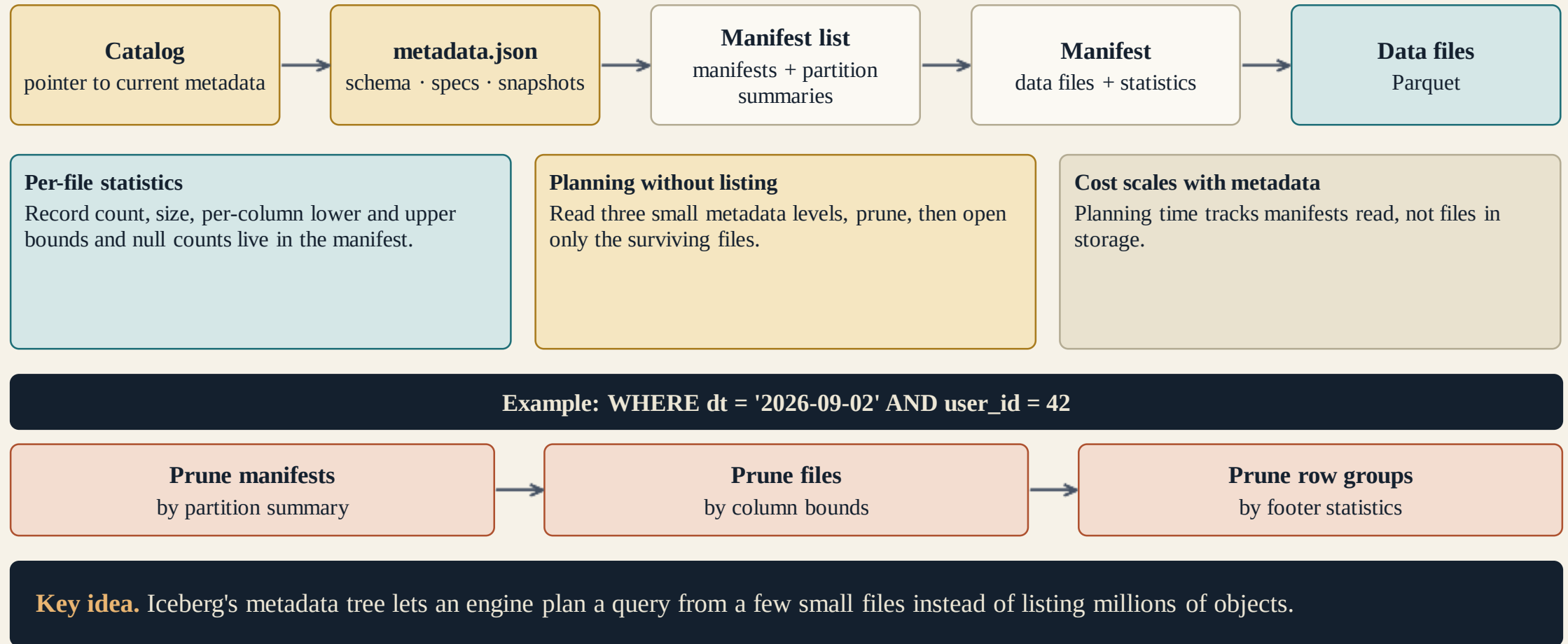
Key idea. A lakehouse is warehouse behaviour on open files: a metadata layer adds transactions, schemas and history to object storage.

Three open table formats side by side

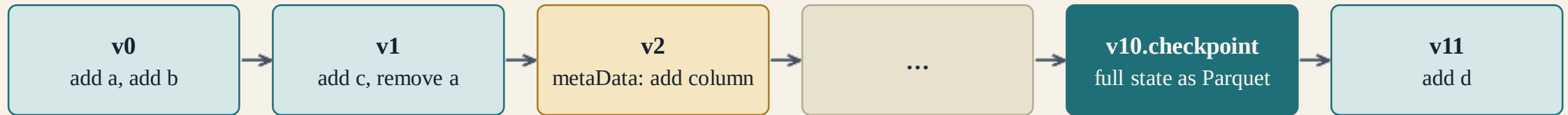


Key idea. All three give ACID commits over Parquet. They differ in metadata layout, update strategy and ecosystem gravity.

Iceberg internals: the metadata tree



Delta Lake internals: the transaction log



Rebuilding state

Table state = latest checkpoint + later JSON commits. Readers consult `_last_checkpoint`, list newer log files and replay the actions.

Atomic commit

A commit is the creation of the next numbered file, using put-if-absent. The first writer to create version N wins.

Actions

add, remove, metaData, protocol, commitInfo, txn.

Checkpoints

Written every 10 commits by default, so replays stay short.

Removal is logical

Removed files stay until VACUUM after a retention period.

Key idea. Delta records every change as an ordered log. Checkpoints keep the cost of reconstructing the current state small.

ACID transactions on an object store

Writer 1



Writer 2



Atomicity

Data files are invisible until the metadata pointer moves in one step.

Isolation

Readers pin an immutable snapshot, so writers never disturb them.

Durability

Inherited from the object store, with replicated metadata.

Needs

Atomic put-if-absent or compare-and-swap, from the store or a catalog.

Key idea. ACID on object stores comes from immutable files plus one atomic pointer update, with retry after conflict.

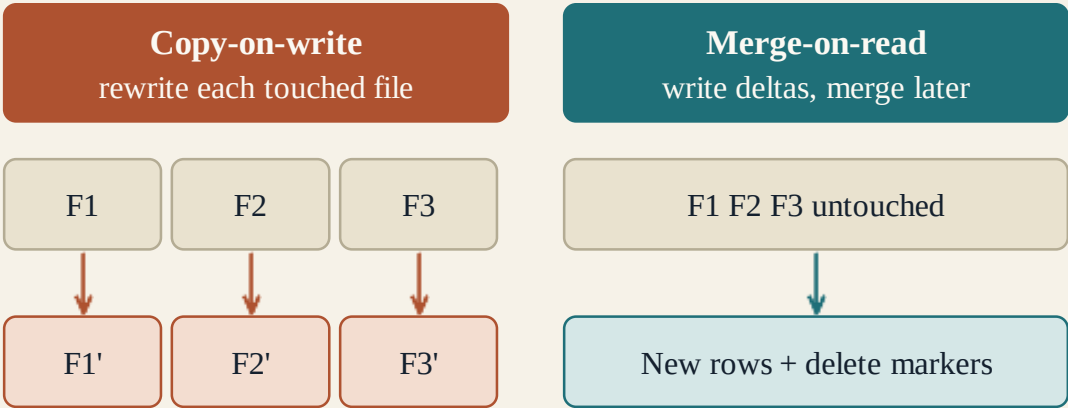
Concurrent writers: what conflicts and what does not

Writer A	Writer B	Outcome	Reason
Append	Append	No conflict	Different new files
Append	Update or delete	Usually none	New files are not read by the update
Update	Update, same files	Conflict	Both rewrite the same file
Update	Update, different partitions	No conflict	Disjoint file sets
Compaction	Append	No conflict	Compaction rewrites only old files
Compaction	Update, same files	Conflict	Rewritten file is gone

Key idea. Design jobs to write disjoint partitions or files and retry on conflict. Isolation level sets how strict validation is.

Copy-on-write versus merge-on-read

Update 1% of rows scattered over 1,000 files of 128 MB

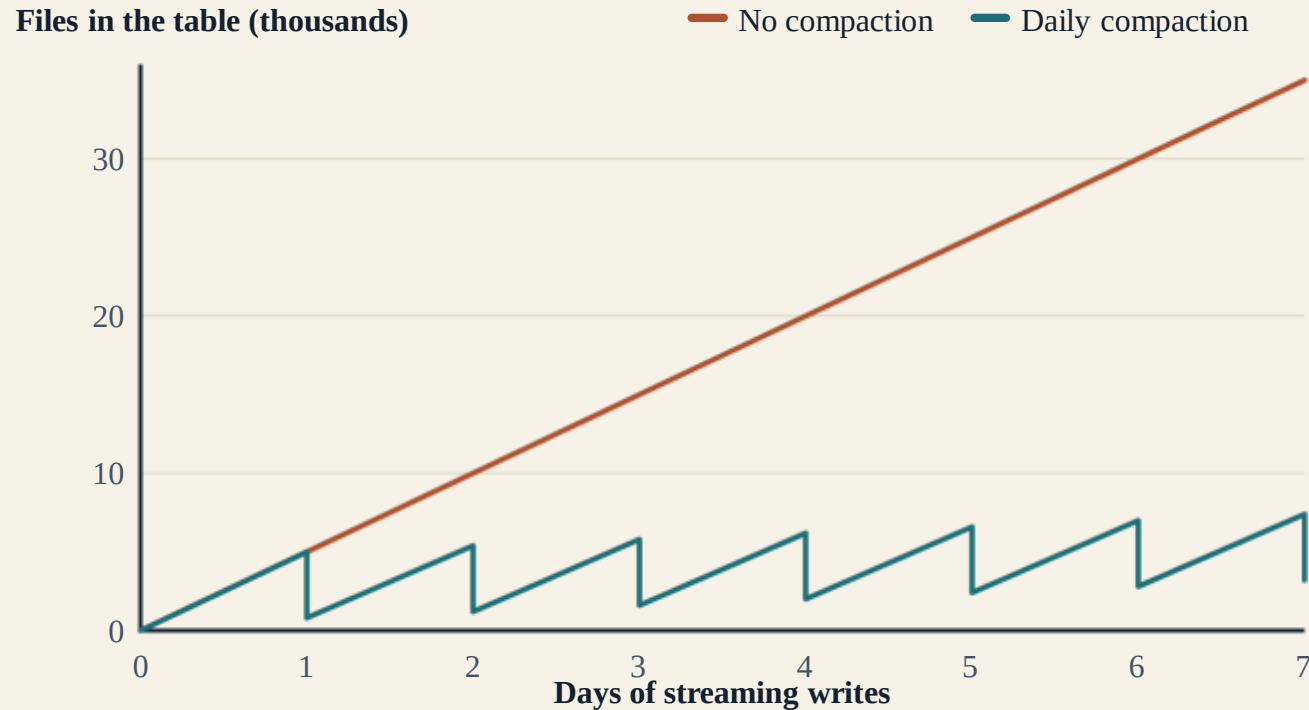


****	Copy-on-write	Merge-on-read
Bytes written	128 GB	≈ 1.3 GB
Write amplification	≈ 100×	≈ 1×
Read cost	Plain scans	Apply deletes
Best for	Read-heavy, rare updates	Frequent updates

Write amplification = bytes rewritten ÷ bytes changed. Here 128 GB ÷ 1.28 GB = 100 for copy-on-write. Merge-on-read stores position or equality deletes (Iceberg) or deletion vectors (Delta) and merges them at read time or during compaction.

Key idea. Copy-on-write pays at write time, merge-on-read pays at read time. Compaction settles the debt.

Table maintenance: compaction, expiry and cleanup



Compact

bin-pack to 128 MB to 1 GB

Expire snapshots

drop old history

Remove orphans

older than longest write

Rewrite manifests

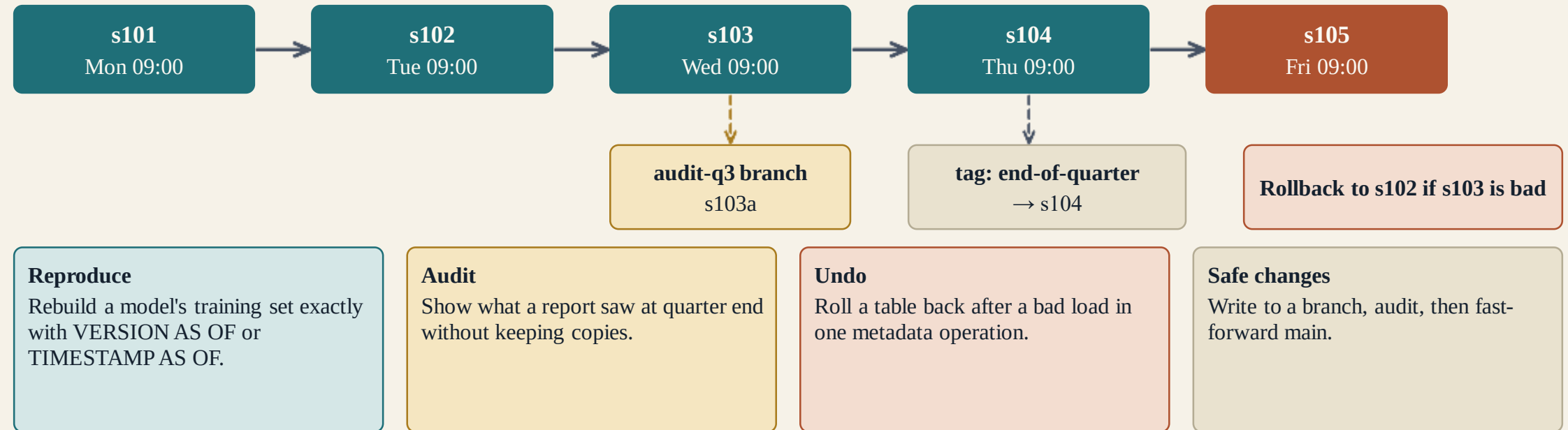
keep planning fast

Run on a schedule and monitor file counts.

Key idea. Streaming and frequent commits create small files and metadata. Scheduled maintenance keeps reads and planning fast.

Time travel, branches and tags

main branch



Expiring snapshots removes history. Set retention to cover audits and privacy deletion duties.

Key idea. Snapshots make tables versioned data: reproducible, auditable and reversible until retention removes the history.

Schema and partition evolution

Spec 1: months(ts), files from January to June

M1

M2

M3

M4

M5

M6

Spec 2: days(ts), files from July onward

D1

D2

D3

D4

D5

D6

One query: planner evaluates each partition spec separately. No rewrite of old files, and no change to user SQL.

Hidden partitioning

Partition by a transform of a column, such as day(ts). Users filter on ts, not on a derived column.

Column ids

Columns are tracked by id, not name or position. Rename and reorder are metadata-only.

Safe changes

Add, drop, rename and widen types without rewriting data files.

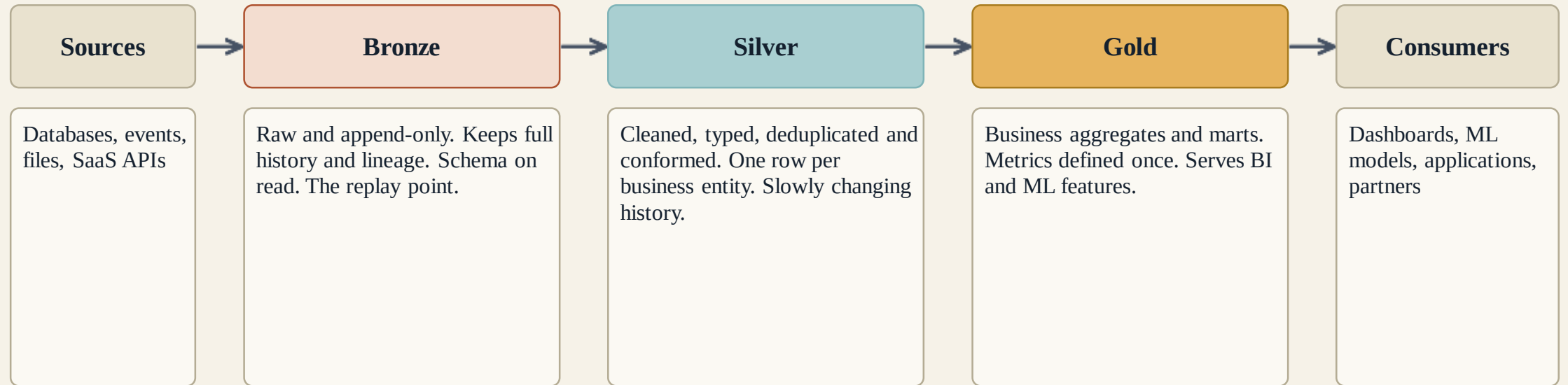
Key idea. Evolution as metadata: change partitioning and schema going forward and leave historical files where they are.

Catalogs: the commit point and the security boundary

Catalog	Role	Strength	Watch out
Hive Metastore	Legacy table registry	Universal engine support	Weak commit semantics, locks
AWS Glue Data Catalog	Managed registry on AWS	Integrates with AWS analytics	AWS-centred
Iceberg REST catalog	Open HTTP spec for catalogs	Engine-neutral, vended credentials	Needs a server implementation
Unity Catalog	Governance and catalog	Fine-grained access and lineage	Feature depth varies by host
Nessie	Git-like catalog	Branches and multi-table commits	Extra service to run

Key idea. The catalog decides who can commit and read. Prefer an open REST interface so engines stay swappable.

Medallion architecture: bronze, silver, gold



Quality gates and contracts sit at each boundary. Bronze accepts anything; silver enforces schema and keys; gold enforces business rules. Failures quarantine, not overwrite.

Key idea. Medallion layers separate replay, cleaning and serving, so each step can be rebuilt from the one before it.

How a lakehouse query avoids reading data



Read fraction = $0.0192 \times 0.25 \times 0.5 \times 0.1 = 0.00024$, about 4,000× less. At \$5 per TB scanned: \$5,000 versus \$1.20.

Key idea. Speed comes from not reading data. Each layer of metadata pruning multiplies savings, so layout is a performance feature.

Lakehouse performance levers

Lever	Effect	Cost or risk
Sort or cluster	Tighter min/max ranges, fewer files read	Rewrite cost, choose columns carefully
Compaction	Fewer requests, larger sequential reads	Compute and commit conflicts
Column statistics	Better join plans and pruning	Collection cost, staleness
Local caching	Lower latency on hot data	Cache capacity and warm-up
Materialised views	Precomputed aggregates	Freshness lag, storage
Deletion vectors or MoR	Cheaper row-level updates	Extra read work until compaction

Key idea. Optimise in order: layout and compaction first, statistics next, caching and precomputation last.

Warehouse, lake or lakehouse: choosing

Criterion	Warehouse	Raw data lake	Lakehouse
Data types	Structured, some semi-structured	Anything	Anything
Interactive SQL	Excellent	Slow without engines	Good with tuning
Governance	Mature, built in	Manual	Catalog-driven, maturing
Cost at scale	Higher for storage and compute	Lowest storage	Low storage, elastic compute
Openness	Often proprietary formats	Fully open	Open formats and engines
ML and data science	Awkward export	Native file access	Native file access

Key idea. Pick a warehouse for simplicity, a lakehouse for openness and ML reach, and treat a raw lake as an ingestion zone only.

PART 6 OF 9

Data mesh

Ownership, data products, contracts, platform and governance, with honest trade-offs

- Why a central team becomes the bottleneck
- Data products, contracts and the platform
- Costs, limits and when not to adopt a mesh

DEPTH LADDER · THIS PART IS LEVEL 4

L1 · Concepts

L2 · Structures

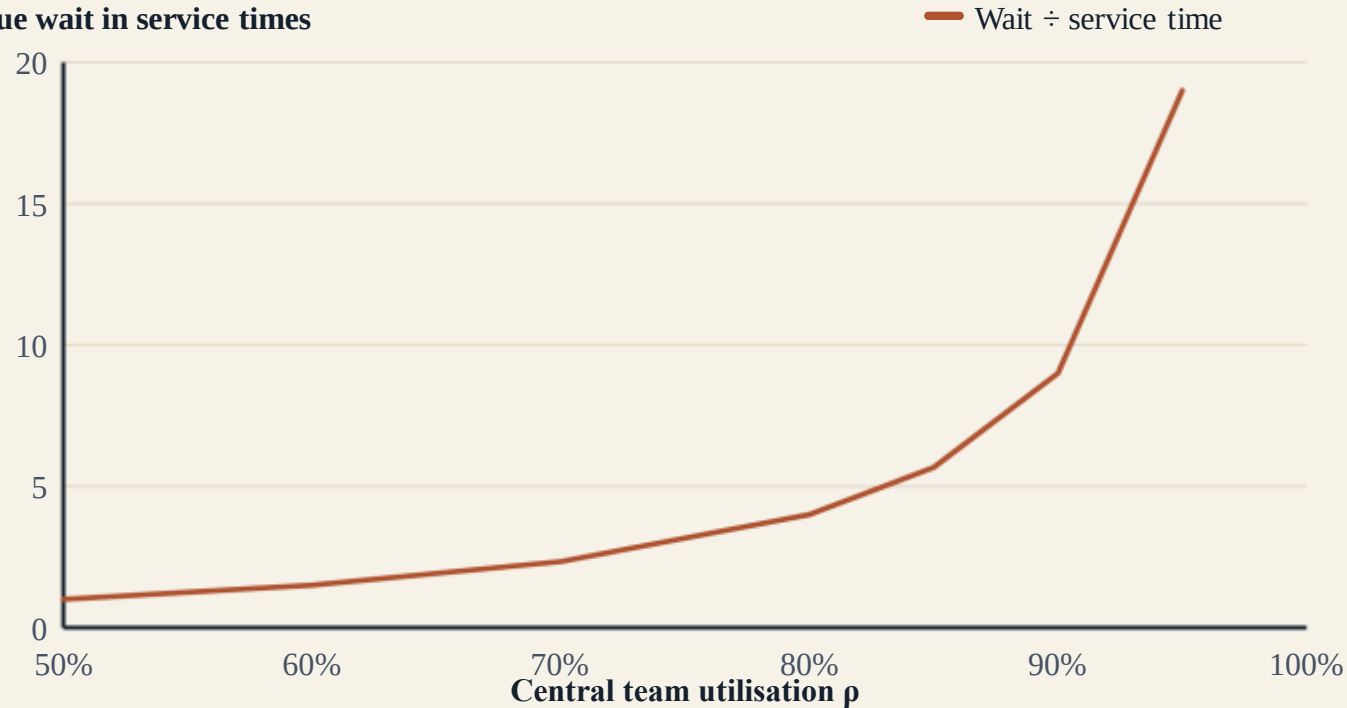
L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

The central bottleneck, quantified

Queue wait in service times



Queueing law

Mean wait in an M/M/1 queue = $\rho / (1 - \rho) \times$ service time. At $\rho = 0.9$ the wait is 9 service times.

Example

A team completes 10 requests a week, demand is 9: $\rho = 0.9$. If a request takes a week of effort, it waits about 9 weeks.

Point

Waiting, not work, is the cost. Splitting queues by domain cuts ρ per queue.

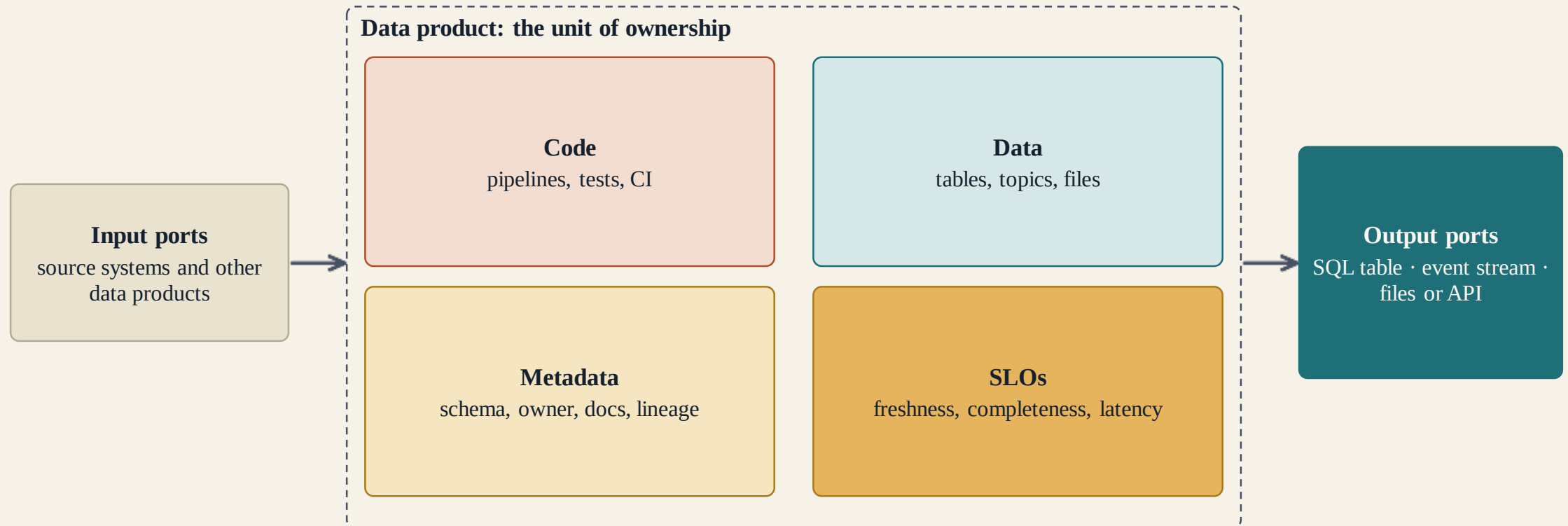
Key idea. A single central team is a shared queue. Near full utilisation, small demand increases cause huge delays.

The four principles of data mesh

Domain ownership	Data as a product	Self-serve platform	Federated governance
Teams that produce data own its quality, meaning and serving.	Discoverable, documented, versioned outputs with an owner and SLOs.	Shared tooling makes building a product cheap and consistent.	Global rules enforced as code. Local choices stay local.
Without it	Without it	Without it	Without it
A central team guesses at meaning and fixes slowly.	Tables nobody can find or trust.	Every domain rebuilds pipelines its own way.	Chaos, or a central gate that recreates the bottleneck.

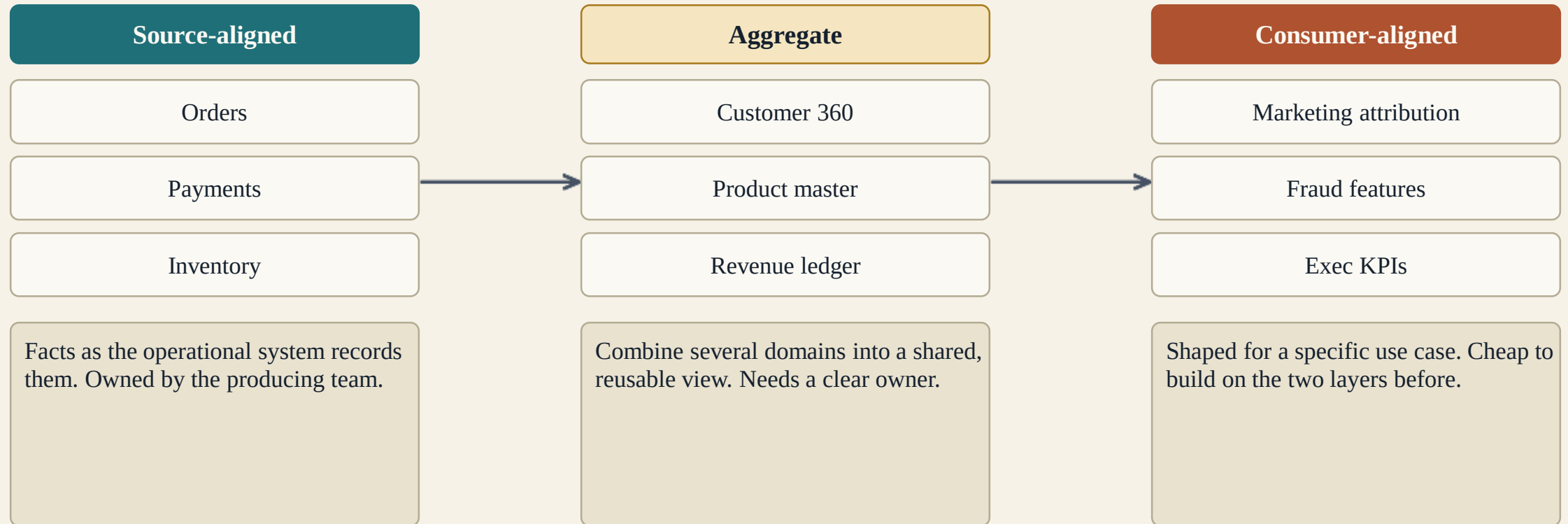
Key idea. The four principles interlock: each one covers the failure mode created by the others.

Anatomy of a data product



Key idea. A data product bundles code, data, metadata and SLOs behind stable ports, owned by one team.

Domain boundaries and three kinds of data product



Key idea. Model domains as source-aligned, aggregate and consumer-aligned layers. Ownership follows the business boundary.

Data contracts: example and anatomy

```
dataset: orders.order_completed
owner: team-checkout
version: 2.3.0
schema:
  order_id: string, required, unique
  amount: decimal(12,2), required
  completed_at: timestamp, UTC
semantics: amount in order currency, tax included
sla:
  freshness: 15 minutes
  completeness: 99.9 percent
evolution: additive only, 90 days notice for breaking
pii: [customer_email]
```

Schema

names, types, keys and nullability

Semantics

meaning, units and grain

SLAs

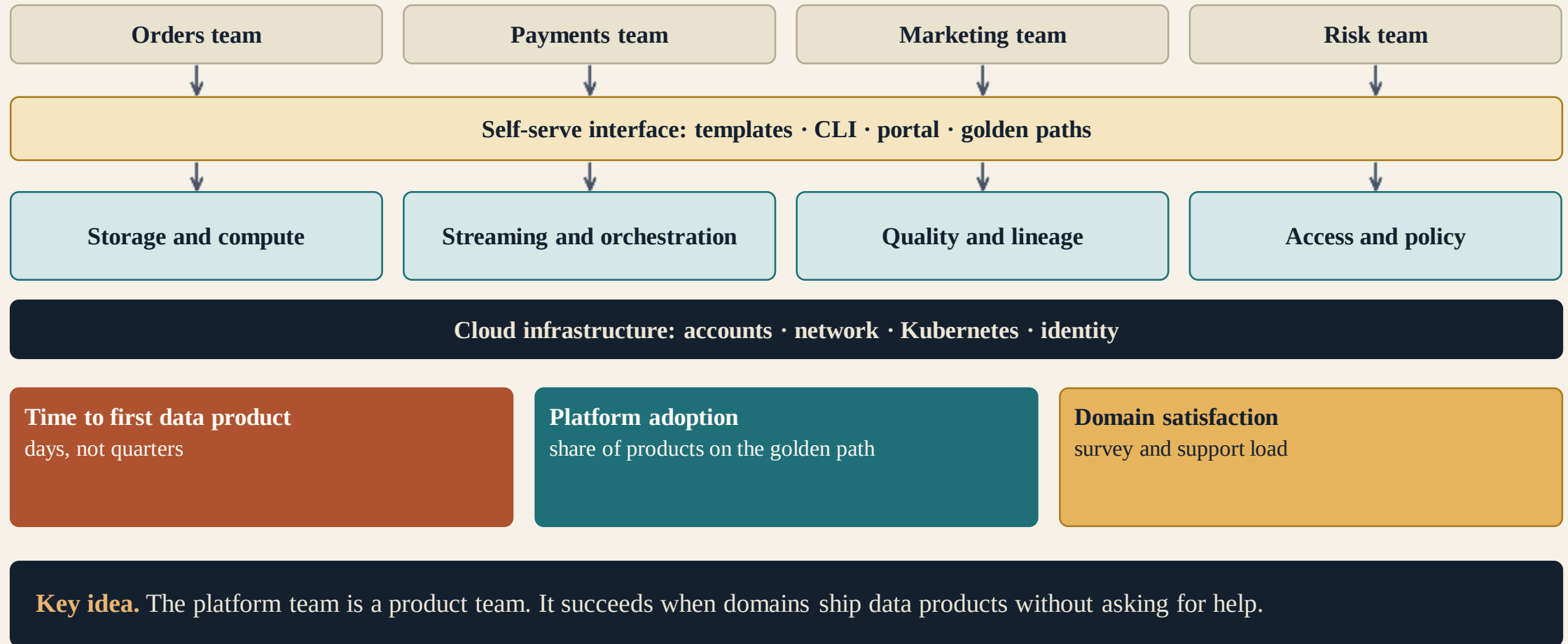
freshness, completeness, availability

Evolution

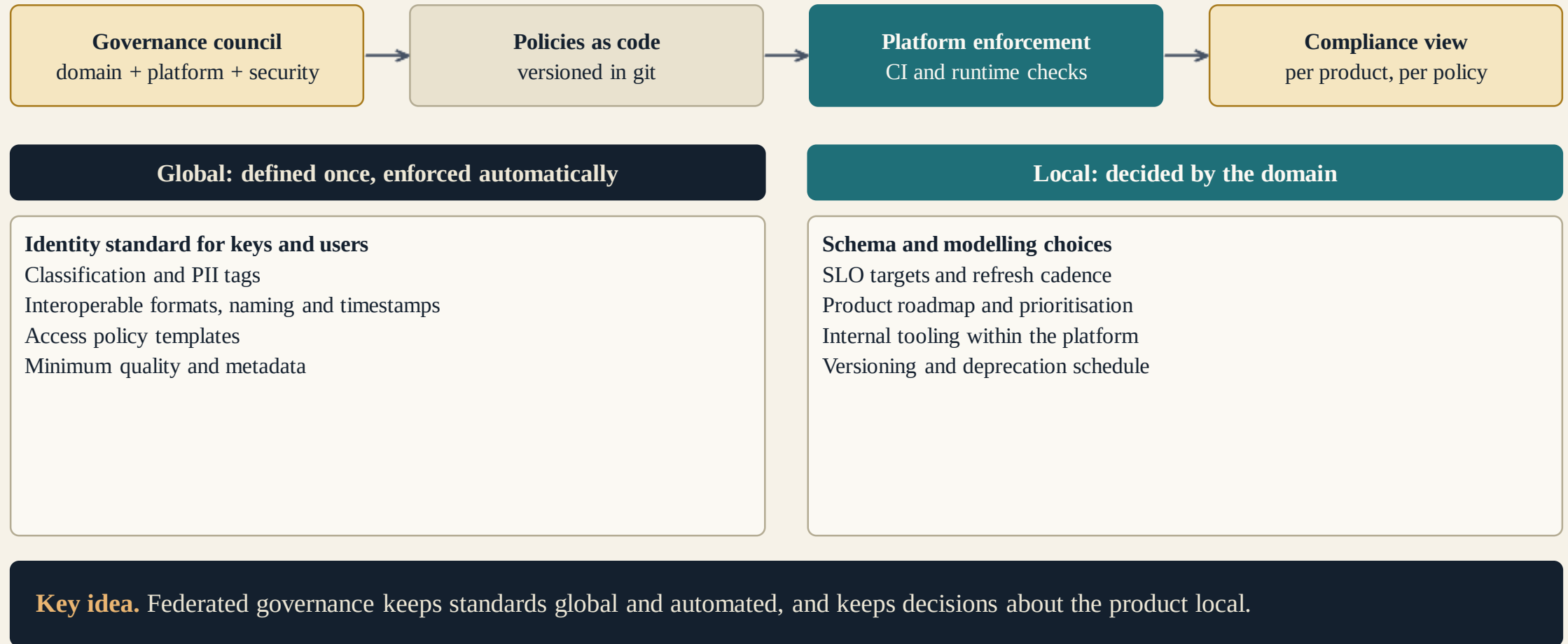
versioning and deprecation rules

Key idea. A contract turns implicit assumptions into a versioned, testable interface between producer and consumer.

The self-serve data platform



Federated computational governance



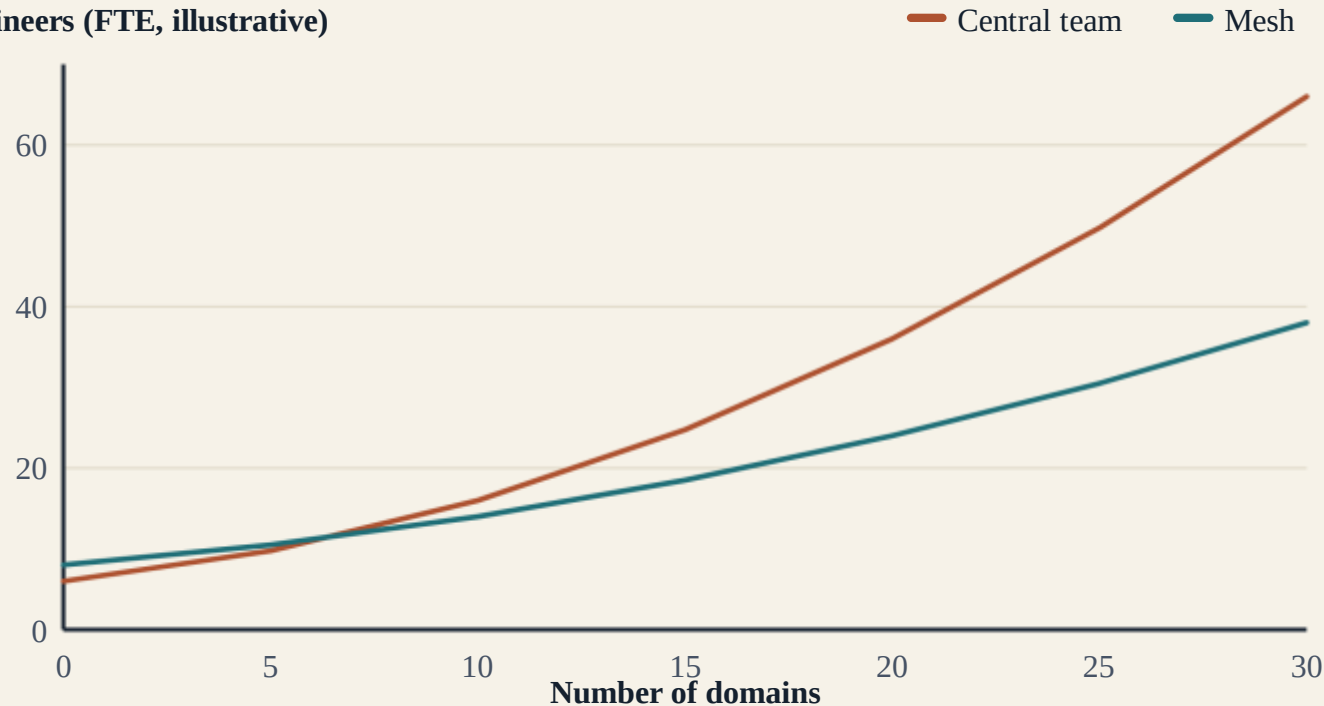
Team roles in a mesh and where they go wrong

Team	Purpose	Interaction	Typical failure
Domain (stream-aligned)	Owns products end to end	Consumes the platform as a service	Skills gap, duplicated effort
Platform	Builds self-serve capabilities	Treats domains as customers	Becomes a new bottleneck
Enabling	Coaches domains on modelling and quality	Time-boxed pairing	Advice without ownership
Governance guild	Sets and evolves global policies	Council with domain reps	Slow or ignored standards

Key idea. Conway's law applies: the org chart becomes the architecture. Design the teams and their interfaces on purpose.

A cost model: when does a mesh break even?

Engineers (FTE, illustrative)



Model assumptions

Central = $6 + 0.5N + 0.05N^2$. Mesh = $8 + 0.4N + 0.02N^2$. Fixed platform cost is higher; coordination growth is lower.

Crossover near $N = 7$

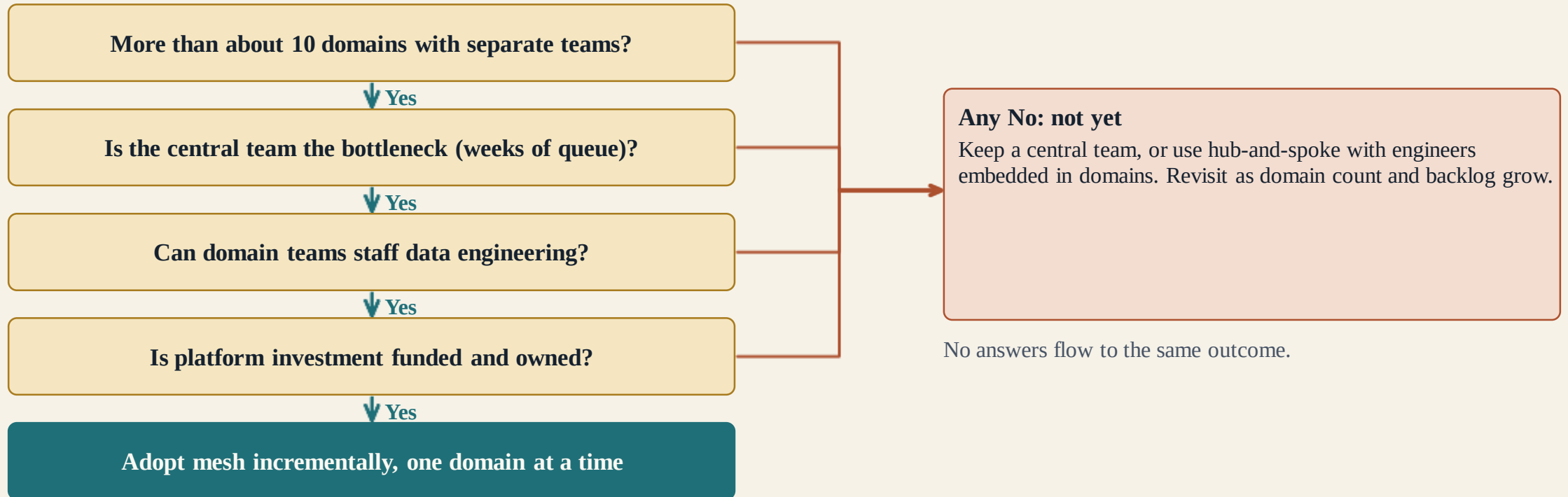
At 10 domains: 16 versus 14. At 30: 66 versus 38.

Caveat

Coefficients are assumptions. Measure your own.

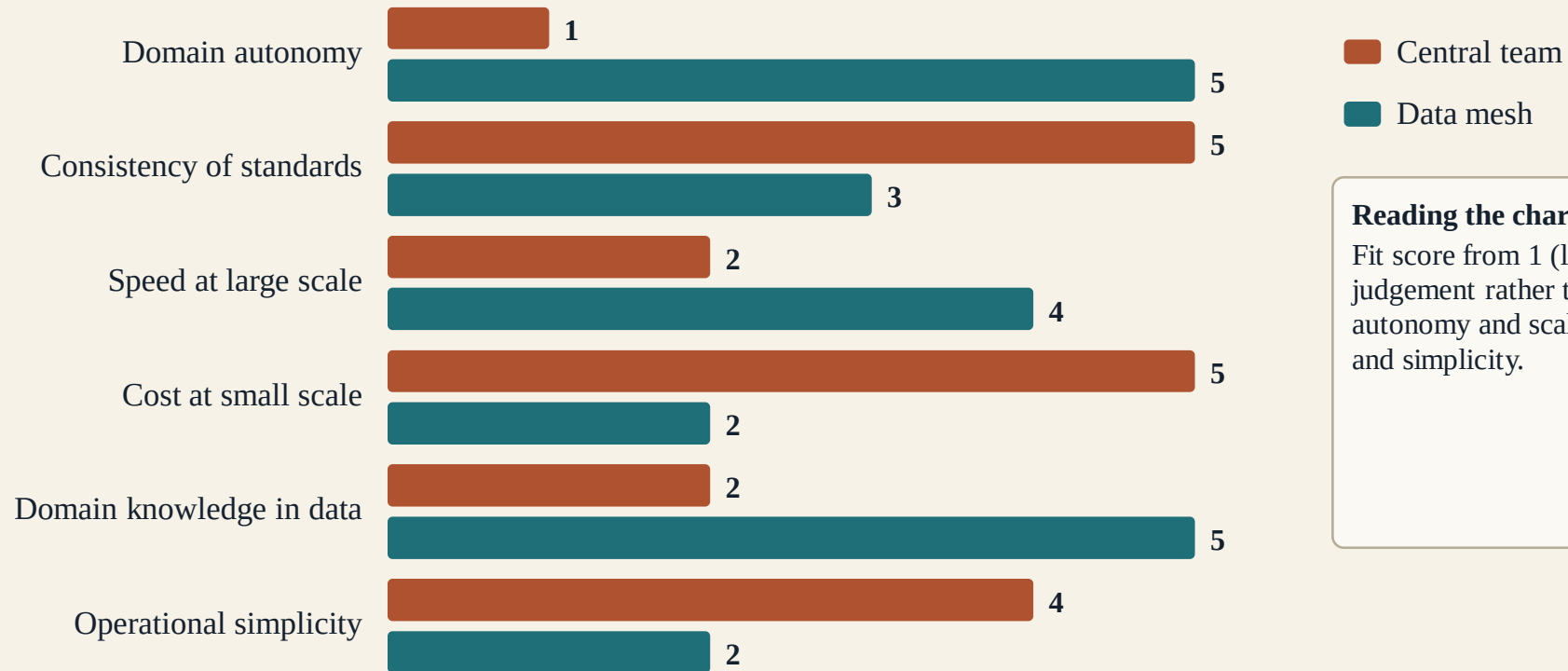
Key idea. A mesh has a fixed entry cost. It pays back only when central coordination overhead grows faster than domain count.

When a data mesh is the wrong answer



Key idea. Mesh solves a scale-of-organisation problem. Small organisations rarely have it and pay the costs without the benefits.

Trade-off scorecard: central team versus mesh

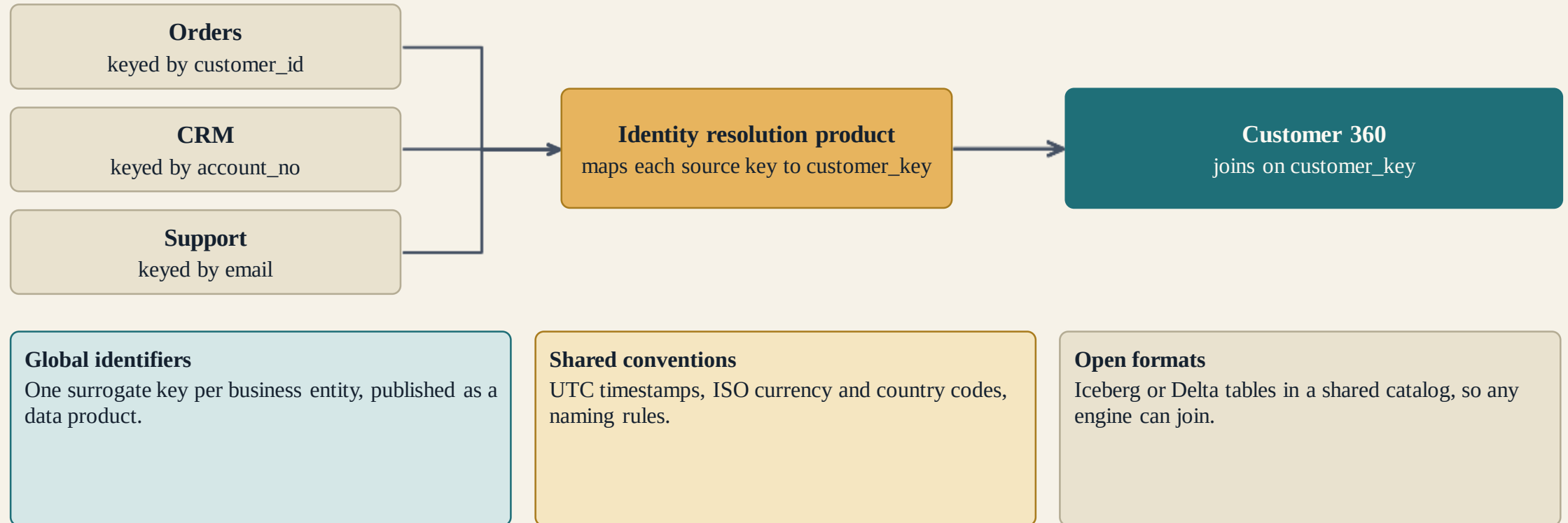


Reading the chart

Fit score from 1 (low) to 5 (high), a qualitative judgement rather than a measurement. Mesh buys autonomy and scale at the price of consistency and simplicity.

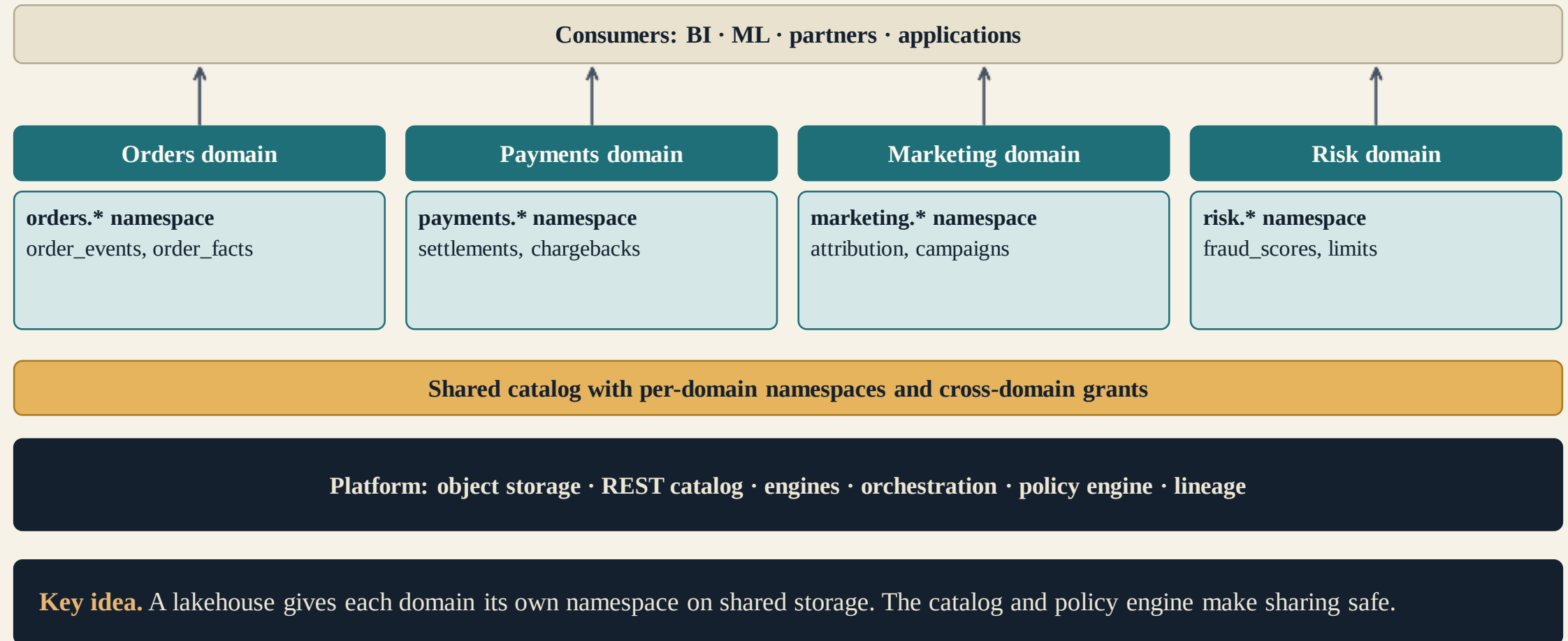
Key idea. Neither model wins on every axis. Choose the trade-off that matches your organisation's size and pain.

Interoperability: joining across domains



Key idea. Autonomy without interoperability recreates silos. Agree on identifiers and conventions globally, and publish them as products.

Implementing a mesh on a lakehouse



PART 7 OF 9

Scaling and capacity

The mathematics of speedup, queues, tail latency, sharding, replication and cost

- Amdahl, USL, Little's law and queueing
- Tail latency, sharding and quorums
- Capacity plans, cost models and availability

DEPTH LADDER · THIS PART IS LEVEL 5

L1 · Concepts

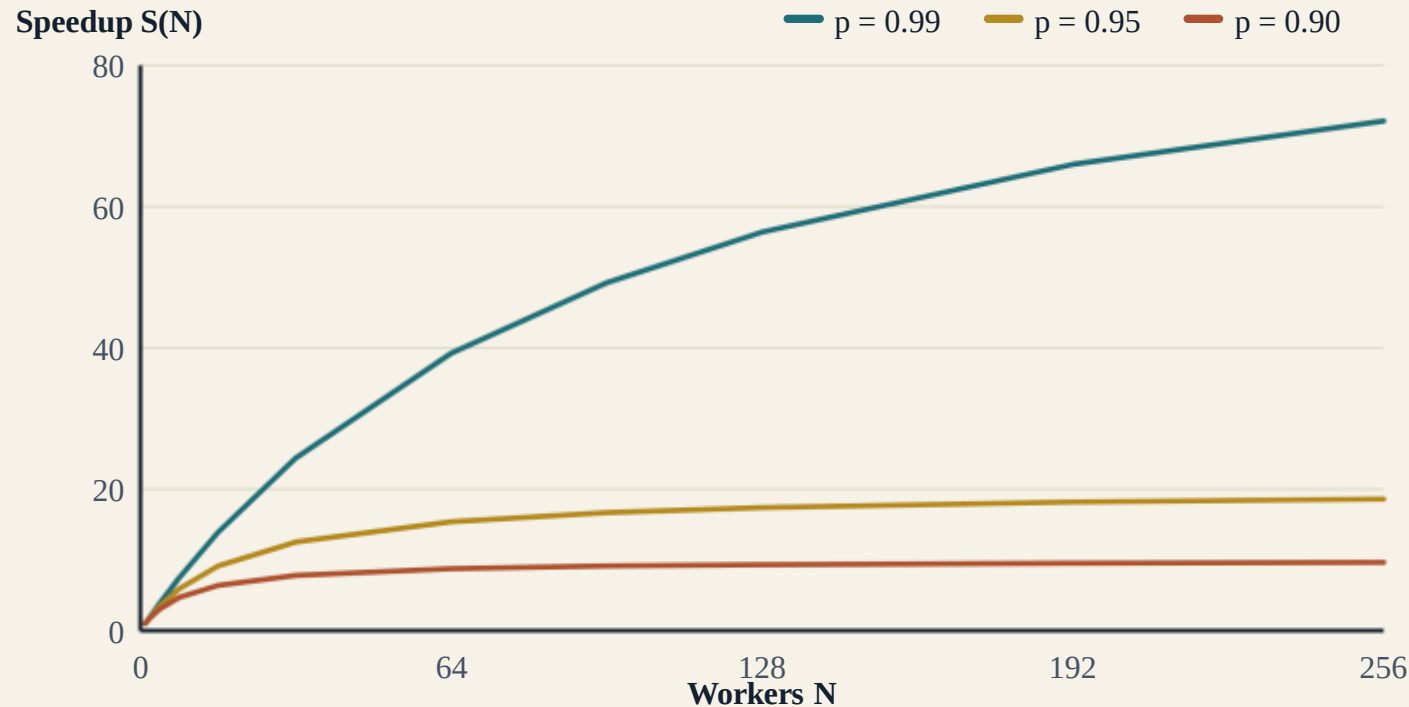
L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

Amdahl's law: the ceiling on parallel speedup



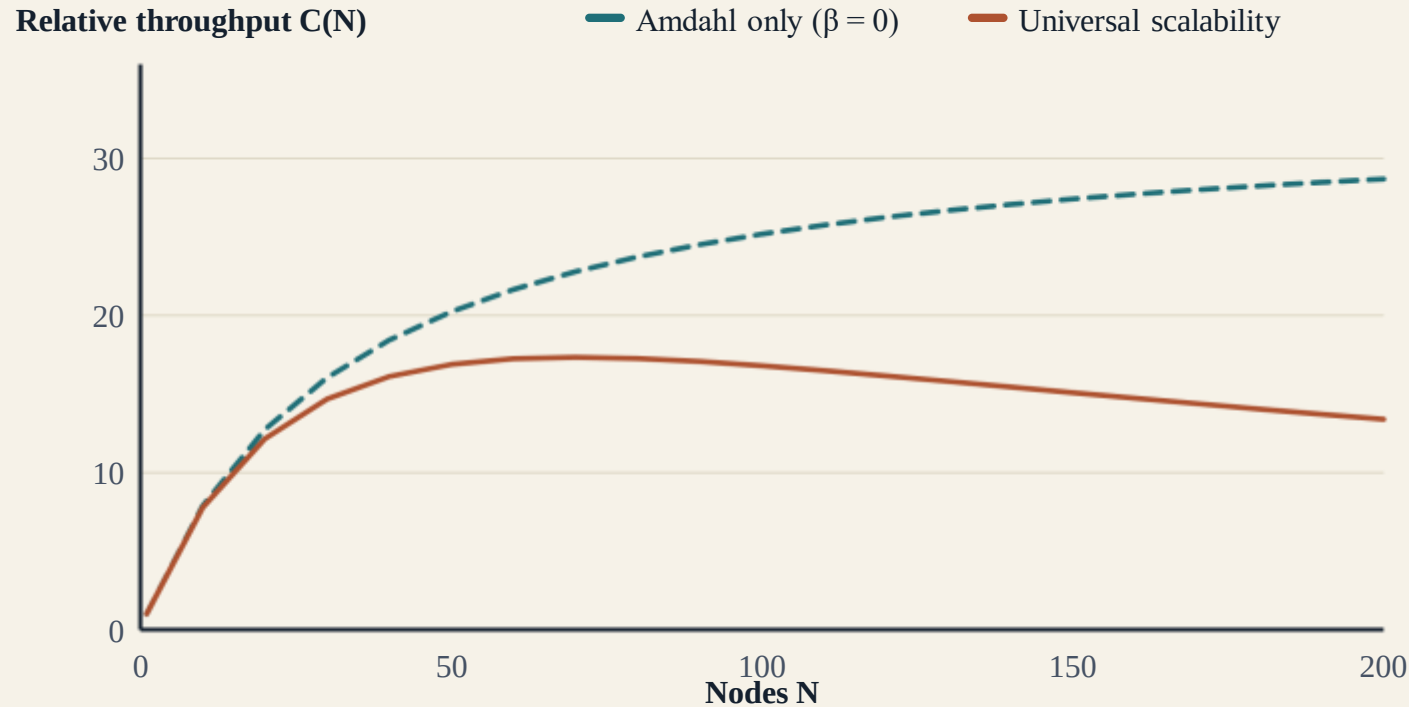
$S(N) = 1 / ((1 - p) + p/N)$
 p is the parallel fraction of the work.

Ceiling = $1 / (1 - p)$
 $p = 0.90$ caps at $10\times$, $p = 0.95$ at $20\times$, $p = 0.99$ at $100\times$.
At $N = 256$: $9.6\times$, $18.6\times$ and $72\times$.

In data systems
The serial part is commits, final merges, driver work and the slowest task.

Key idea. A 5% serial fraction limits any cluster to $20\times$ speedup. Find and shrink the serial part before buying nodes.

Universal Scalability Law: contention plus coherency



$$C(N) = N / (1 + \alpha(N-1) + \beta N(N-1))$$

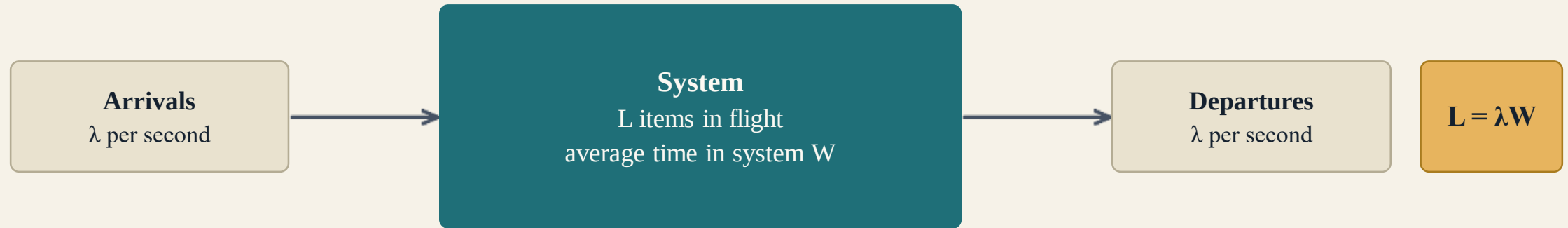
α contention, β coherency cost.

Peak at $N^* = \sqrt{(1-\alpha)/\beta}$
 $\alpha = 0.03$, $\beta = 0.0002$ gives $N^* \approx 70$ and $C \approx 17$. Beyond it throughput falls.

β source
Pairwise coordination: locks, gossip, all-to-all shuffles.

Key idea. Adding nodes can reduce throughput once coordination cost grows with the square of cluster size.

Little's law: sizing concurrency

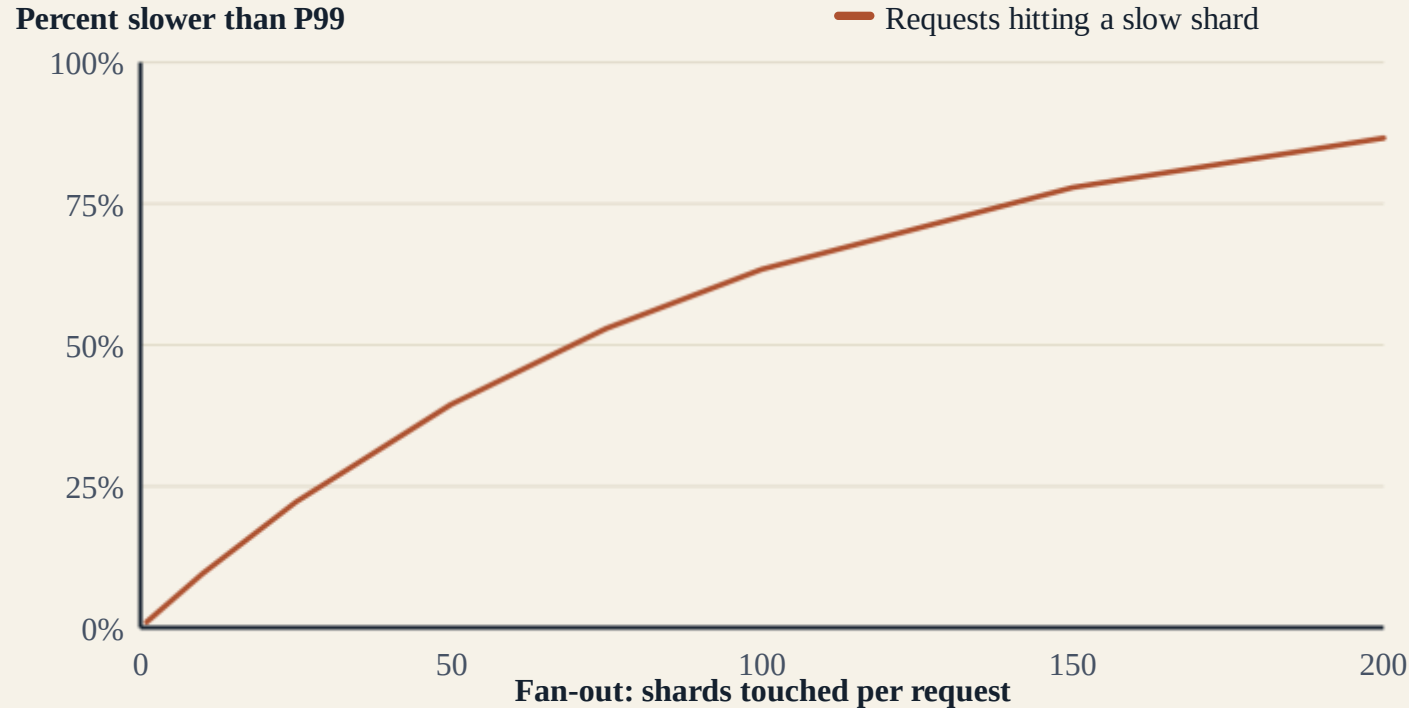


Scenario	λ	W	In flight L	Slots at 70% load
Query service	5,000/s	0.2 s	1,000	1,429
Stream job	100,000/s	20 ms	2,000	2,857

Slots = $\lambda \times W \div$ target utilisation. It holds for any stable system, whatever the arrival distribution.

Key idea. Little's law converts rate and latency into concurrency. Use it to size thread pools, connections and consumer counts.

Tail latency and fan-out



Probability

$P(\text{slow}) = 1 - (1 - q)^n$. With $q = 1\%$ and $n = 100$: 63.4%.

Mitigations

Reduce fan-out. Hedge requests after a short delay. Set per-shard timeouts and return partial results. Replicate hot shards. Keep utilisation low.

Key idea. At scale the tail is the typical experience. A slow one-in-a-hundred shard slows most requests that fan out to a hundred.

Sharding strategies: hash, range and directory

Hash sharding

shard = $\text{hash}(\text{key}) \bmod N$

S0	S1	S2	S3
1, 5, 9	2, 6, 10	3, 7, 11	4, 8, 12

Even load. Point lookups are fast. Range scans hit every shard.

Range sharding

shard = which key range holds key

S0	S1	S2	S3
1 to 3	4 to 6	7 to 9	10 to 12

Range scans are local. Monotonic keys such as timestamps create a hot last shard.

Directory sharding

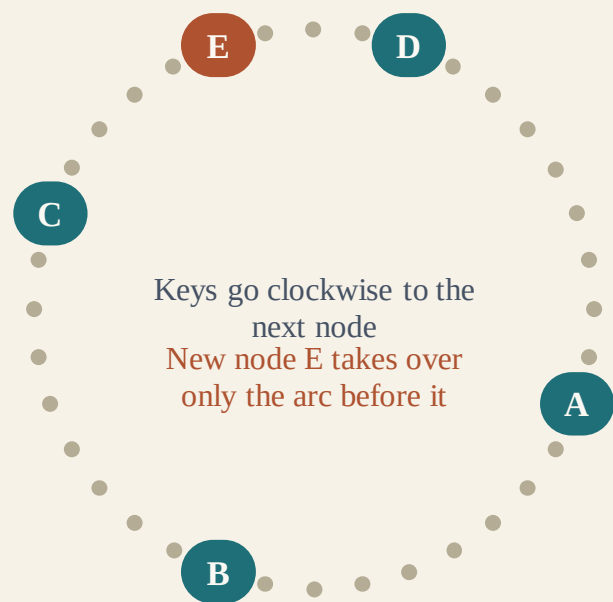
lookup service maps key to shard

S0	S1	S2	S3
1, 6, 11	2, 4, 9	3, 8, 12	5, 7, 10

Flexible placement and easy moves. The directory must be fast and highly available.

Key idea. The sharding function is a promise about which queries stay local. Choose it from the dominant query, not from convenience.

Consistent hashing and rebalancing



Nodes N → N+1	hash mod N moves	Ring moves
4 → 5	80%	20%
9 → 10	90%	10%
19 → 20	95%	5%
99 → 100	99%	1%

Virtual nodes
Give each server many ring positions. Load imbalance falls roughly as $1/\sqrt{v}$, so 100 virtual nodes leave about 10% standard deviation and 1,000 about 3%.

Key idea. Consistent hashing moves about $1/(N+1)$ of the keys when a node joins. Modulo hashing moves almost all of them.

Replication and quorums

N = 3 replicas. Write set W = 2, read set R = 2

Replica 1
write acked

Replica 2
write acked, read

Replica 3
read

Any read set overlaps any write set in at least one replica when $R + W > N$.

PACELC
If Partitioned: choose Availability or Consistency. Else: choose Latency or Consistency. Quorum settings move you along the curve.

W	R	$R + W > 3?$	Effect
3	1	Yes	Fast reads, writes need all replicas
2	2	Yes	Balanced, tolerates one failure
1	3	Yes	Fast writes, reads need all
1	1	No	Fast, eventually consistent

Key idea. Choose R and W so that they overlap for the reads that must be fresh. Otherwise accept eventual consistency and design for it.

A worked capacity plan for a streaming platform

Step	Result
Ingest: 1M events/s × 1 KB	1 GB/s raw
After 4× compression	250 MB/s
Disk writes, replication factor 3	750 MB/s
Consumer egress, 3 groups	750 MB/s
Storage, 7 days × RF 3	453.6 TB
Brokers by throughput (150 MB/s each)	5
Brokers by storage (20 TB × 70% fill)	33

Retention decides

Throughput needs 5 brokers, storage needs 33. Retention and replication, not speed, set the cluster size.

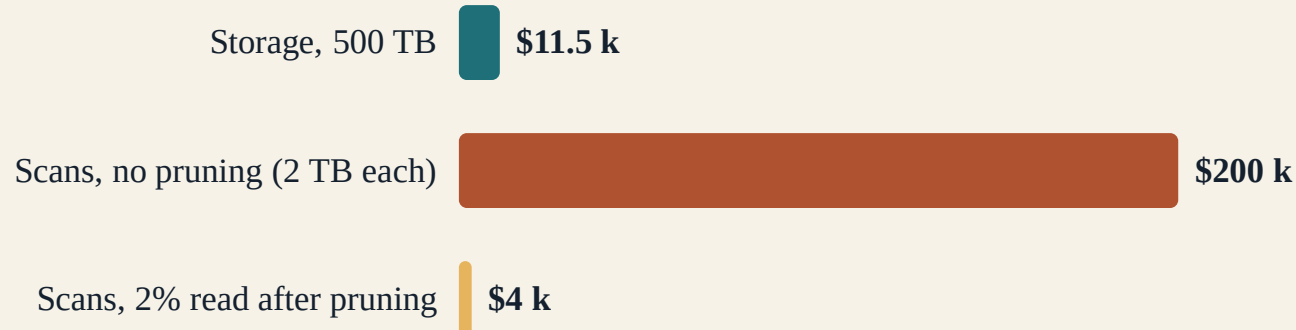
Levers

Shorten retention. Use tiered storage to move old segments to object storage. Compress harder. Add fewer, denser brokers.

Key idea. Do the arithmetic before choosing hardware. In streaming platforms, storage often dominates throughput.

Cost model: storage versus scan cost

Monthly cost for a 500 TB lake, 20,000 queries, \$23/TB-month storage, \$5/TB scanned



Monthly cost = $S \times p_{\text{storage}} + Q \times B \times p_{\text{scan}}$, where B is bytes actually scanned per query.

Storage

Cheap and linear. Tiering and compression trim it.

Scans

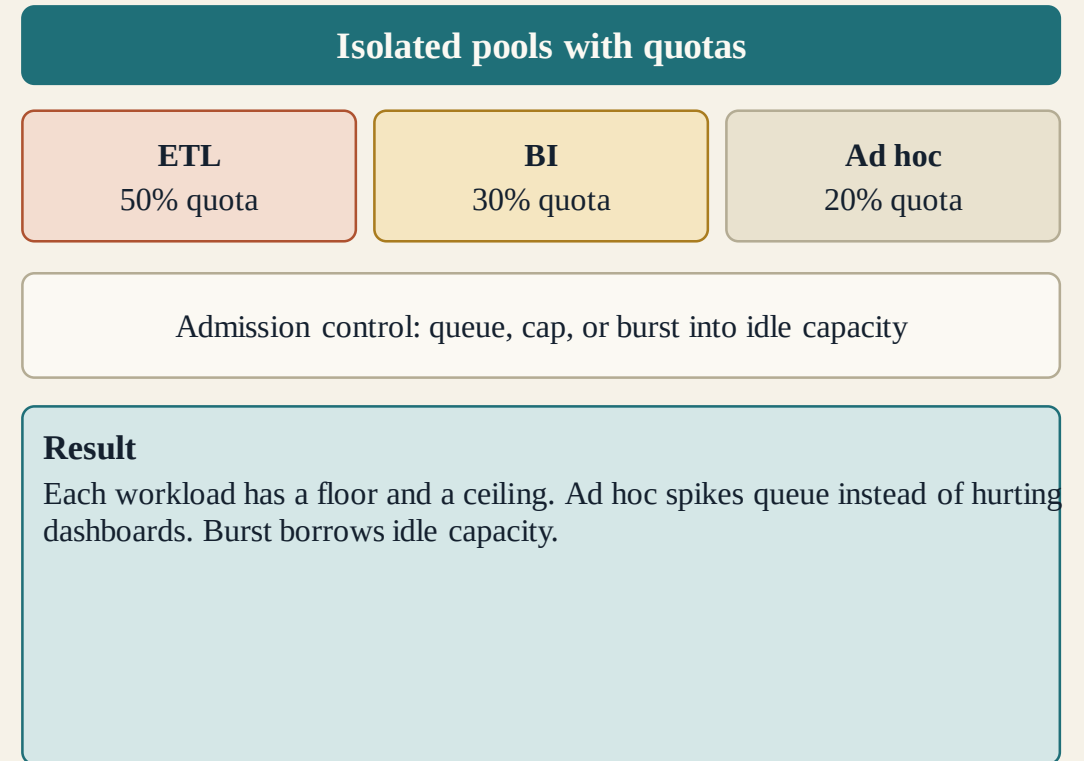
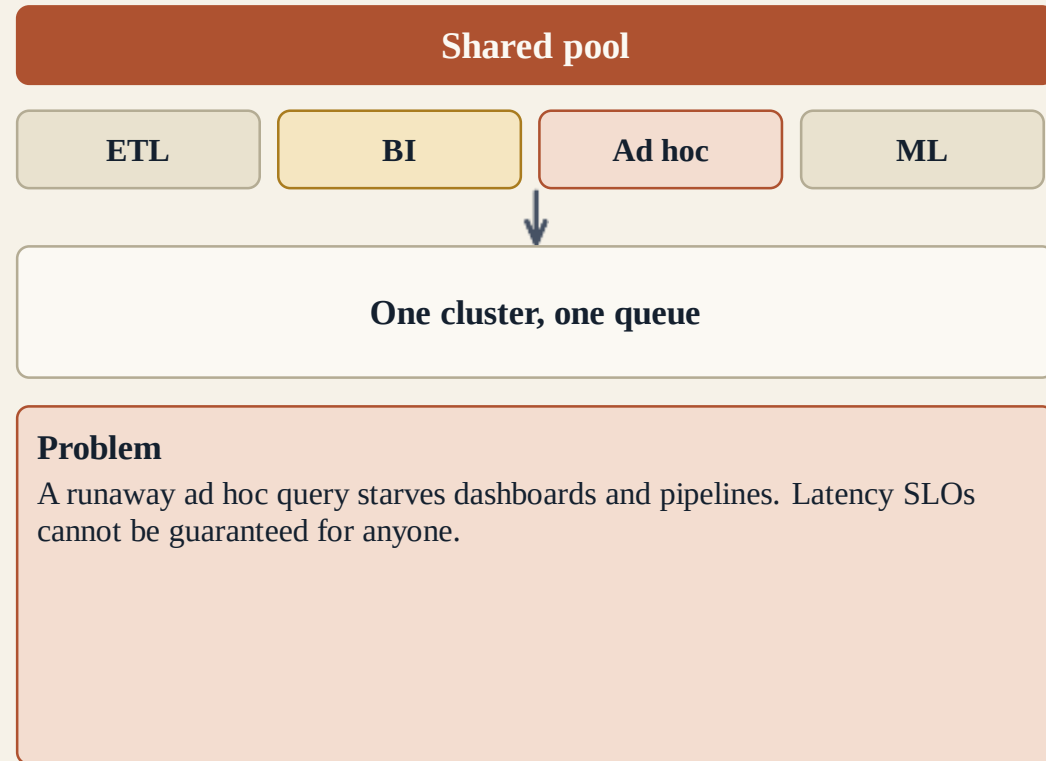
Dominates at high query volume. Pruning cuts them by orders of magnitude.

Compute

Elastic. Idle time is the waste to remove.

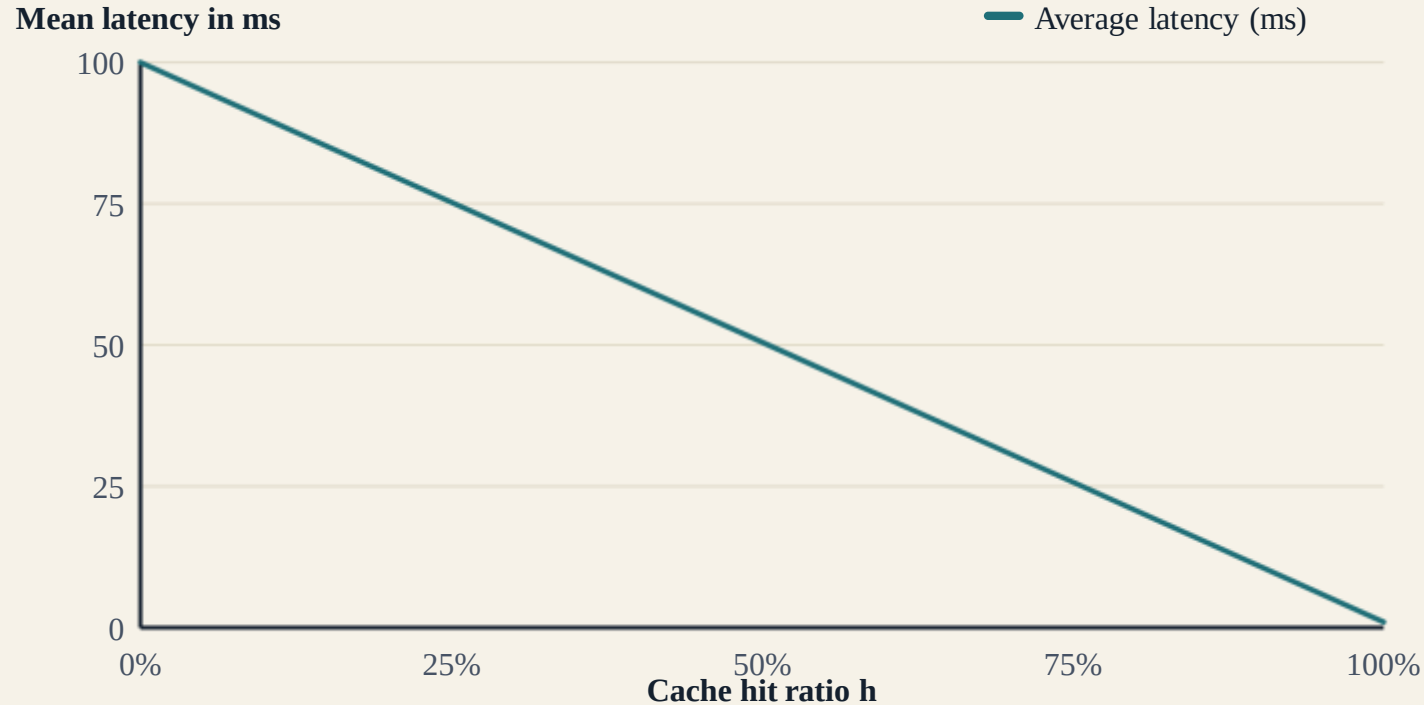
Key idea. At scale, cost follows bytes scanned. Layout and pruning are the largest cost levers, not the storage price.

Concurrency and workload isolation



Key idea. Isolation converts a shared risk into a per-workload budget. Set quotas from SLOs and use admission control at the edge.

Caching hierarchy and the hit-ratio payoff



$$T = h \cdot t_{\text{cache}} + (1 - h) \cdot t_{\text{miss}}$$

With $t_{\text{cache}} = 1$ ms and $t_{\text{miss}} = 100$ ms.

Payoff is steep at the top

From 90% to 99% hits, mean latency drops from 10.9 ms to 2.0 ms, 5.5× lower.

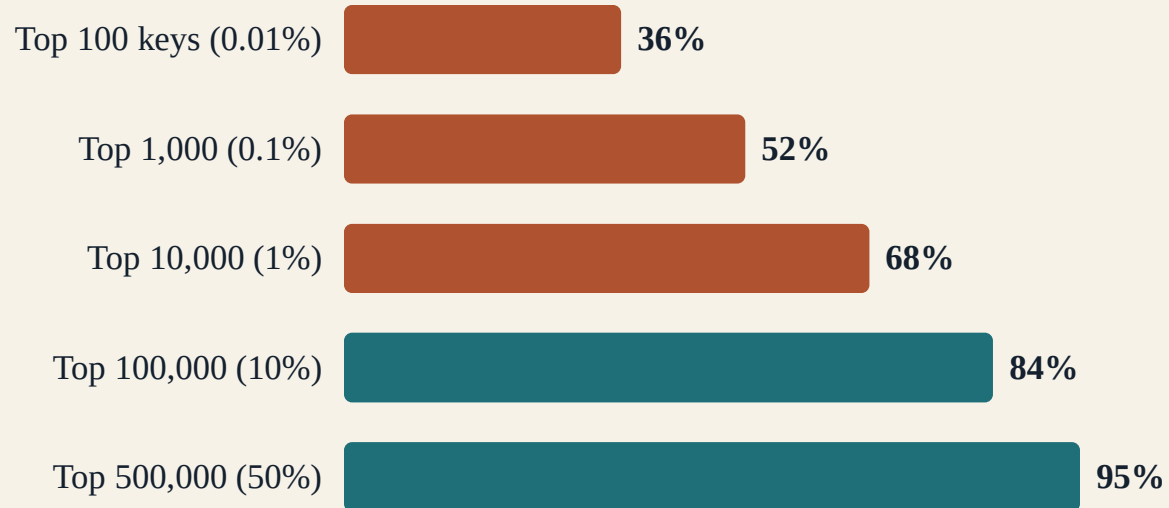
Layers

Client, CDN, service, engine NVMe, object store.

Key idea. Hit ratio dominates. The last few points of hit rate buy the most, and immutable table files make caching safe.

Skew at scale: Zipf and hot keys

Share of requests for 1 million keys under Zipf($s = 1$)



Zipf($s = 1$)

The k -th most popular key gets a share $\propto 1/k$. Share of top m of $N \approx (\ln m + 0.577) / (\ln N + 0.577)$.

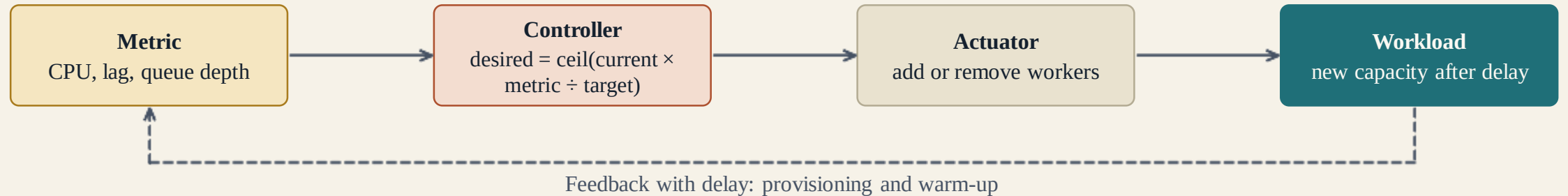
Consequence

One partition holding a few hot keys saturates first. Adding nodes does not help that shard.

Fixes: cache hot keys, split them across sub-keys, replicate them, route by load.

Key idea. Real workloads are skewed. Design for the hot 1% of keys, since they carry most of the traffic.

Elastic scaling as a control loop



Worked example

10 workers at 90% CPU, target 60%: $\text{desired} = \text{ceil}(10 \times 0.9 / 0.6) = 15$. Add 5.

Stability

Scale up fast, scale down slowly. Use cooldowns and a deadband so delay does not cause oscillation.

Signals

Lag and queue depth predict overload better than CPU for streams.

Limits

Stateful jobs need rebalancing time, so scale in steps.

Guardrails

Set a minimum, a maximum and a cost cap.

Key idea. Autoscaling is a control loop with delay. Tune it for stability first, then for cost.

Availability arithmetic and failure domains

Availability	Downtime per year	Per 30 days
99%	87.7 hours	7.2 hours
99.9%	8.8 hours	43.2 minutes
99.99%	52.6 minutes	4.3 minutes
99.999%	5.3 minutes	26 seconds



Series: $A = A1 \times A2 \times \dots \times An$
Five 99.9% parts in a chain give $99.9\%^5 = 99.50\%$, about 44 hours a year.

Parallel: $A = 1 - (1 - A1)(1 - A2)$
Two independent 99% replicas give $1 - 0.01^2 = 99.99\%$.

Chain above: $0.999 \times 0.999 \times 0.9999 \times 0.999 = 99.69\%$. The weakest components and their count set the pipeline's availability, so add redundancy where it is cheapest to remove risk.

Key idea. Availability multiplies down a chain and improves with independent redundancy. Count the components on the critical path.

Scaling playbook: bottleneck, symptom, lever, ceiling

Bottleneck	Symptom	First lever	Ceiling
CPU-bound compute	High CPU, near-linear speedup	Add workers, vectorise, prune	Amdahl ceiling
Shuffle and network	Long stages, large shuffle bytes	Broadcast, pre-aggregate, colocate	Bisection bandwidth
Storage requests	Many small reads, throttling	Compact, larger files, spread prefixes	Per-prefix request limits
Skew or hot key	One slow task or partition	Salt, split, cache, replicate	Key distribution
Coordination and metadata	Slow commits and planning	Fewer commits, rewrite manifests	USL coherency term
Memory and state	OOM, spill, RocksDB stalls	More partitions, state TTL	Node memory

Key idea. Name the bottleneck before scaling. Each has a symptom, a first lever and a mathematical ceiling.

PART 8 OF 9

Operations and governance

Observability, quality, security, privacy, cost, delivery, recovery and the operating model

- Observing and alerting on data health
- Security, privacy and access at scale
- Cost, delivery, recovery and maturity

DEPTH LADDER · THIS PART IS LEVEL 5

L1 · Concepts

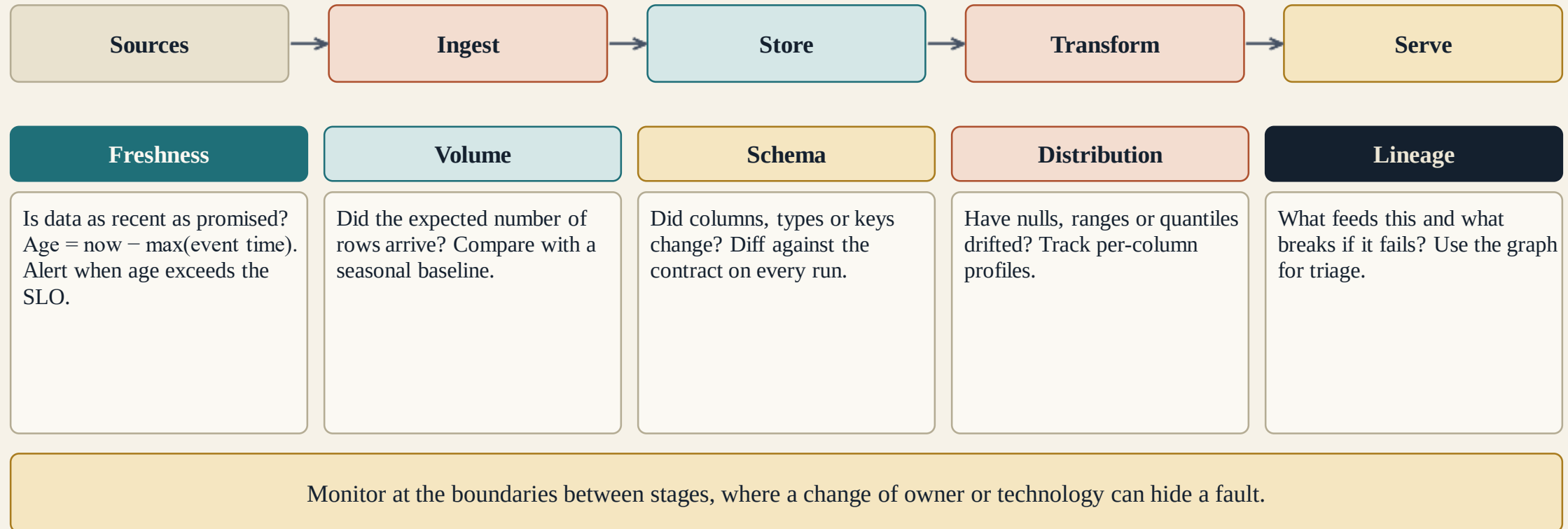
L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

Data observability: five pillars



Key idea. Data observability watches the data, not just the machines: freshness, volume, schema, distribution and lineage.

SLO burn-rate alerting

Burn rate = error rate ÷ (1 – SLO)

Rate 1 spends the budget exactly over the SLO period.

Budget spent = burn rate × window ÷ period

Example: 14.4 × 1 h ÷ 720 h = 2% of a 30-day budget.

Long window	Short window	Burn rate	Budget spent	Action
1 hour	5 minutes	14.4×	2%	Page now
6 hours	30 minutes	6×	5%	Page
3 days	6 hours	1×	10%	Open a ticket

Alert only when both windows exceed the rate. The long window filters noise, and the short window stops the alert soon after recovery. Applies to freshness and correctness SLOs as well as availability.

Key idea. Burn-rate alerts page on the speed of budget loss, so urgent problems wake people and slow leaks become tickets.

Lineage and impact analysis



Rust path: a change to `raw.orders.amount` reaches `fct_orders`, the revenue dashboard and churn features.

Table and column level

Table lineage shows dependencies. Column lineage shows which fields feed a metric.

Open standard

OpenLineage events carry job, run, inputs and outputs from Spark, Airflow, dbt and others.

Uses

Impact analysis before changes, root-cause search during incidents, and evidence for audits.

Key idea. Lineage turns 'who uses this?' and 'why is this wrong?' from meetings into a graph query.

Data quality testing pyramid

Statistical monitors

drift, seasonality, anomalies

Reconciliation

totals match the source

Row constraints

not null, unique, ranges, references

Schema and contract tests

types, keys, required columns

Cheap, deterministic, run every time at the base. Costly, probabilistic at the top.

Anomaly check

$z = (x - \mu) / \sigma$ over a trailing window. Flag

z

> 3 . Example: mean 1.00 M rows, σ 0.03 M; today 0.88 M gives $z = -4$.

Practice

Model weekly seasonality. Route alerts to owners. Tune thresholds so that alerts are rare and always actionable.

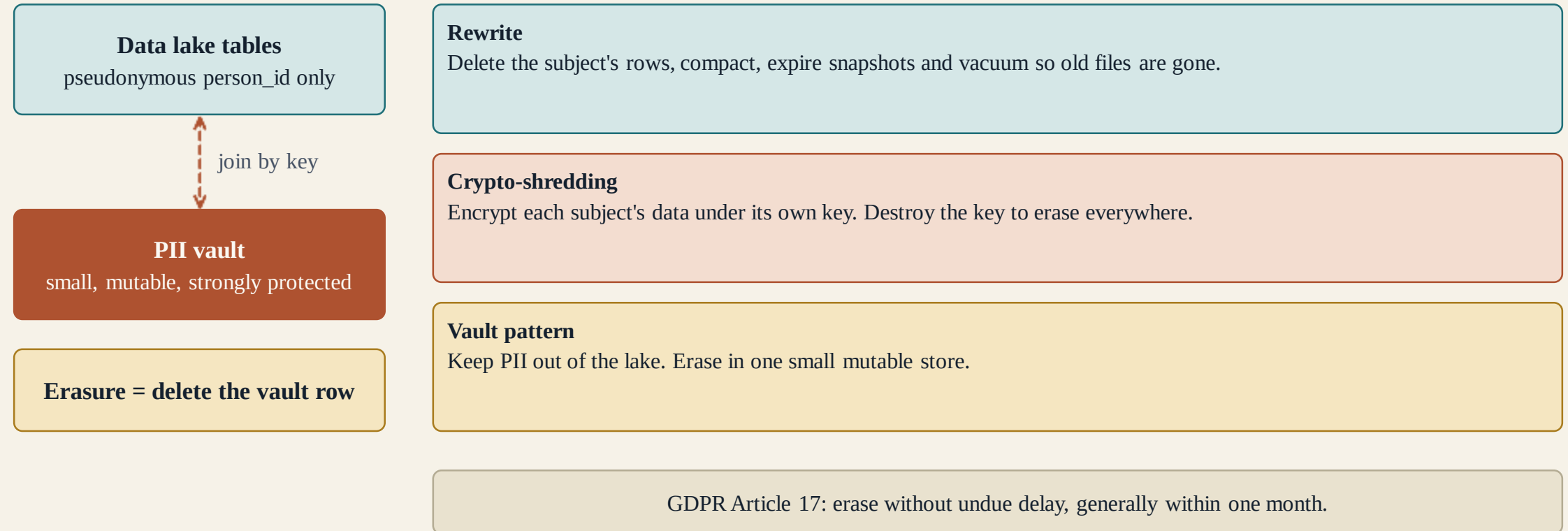
Key idea. Layer the tests: many cheap deterministic checks, fewer statistical ones, all owned and all actionable.

Security architecture: five layers

Identity	Single sign-on, MFA, workload identities for jobs. No shared accounts or long-lived keys.
Authorisation	Roles and attributes decide access. Row filters and column masks enforce it in the engine.
Network	Private endpoints, no public buckets, egress controls between accounts and regions.
Data protection	Encryption at rest with managed keys, TLS in transit, tokenisation for the most sensitive fields.
Audit	Immutable access logs, alerts on unusual reads, regular access reviews.

Key idea. Defence in depth: every layer assumes the one before it can fail, and everything is logged.

Privacy engineering and the right to erasure



Key idea. Immutable files and privacy erasure conflict. Design for erasure early: keep PII out of the lake or make it shreddable.

Access control models and policy as code

```
-- classify once, protect everywhere
CREATE MASKING POLICY mask_email
  (val STRING, role STRING) RETURNS STRING ->
  CASE WHEN role = 'privacy' THEN val
  ELSE sha2(val, 256) END;
APPLY mask_email TO COLUMNS TAGGED 'pii.email';
CREATE ROW POLICY region_rows ON orders
  USING (region = current_user_region());
```

Policies as code are versioned, reviewed and tested, then enforced by the engine or catalog at query time.

RBAC

Roles grant permissions. Simple, but roles multiply with every exception.

ABAC

Rules over user, resource and context attributes. Flexible and harder to audit.

Tag-based

Tag columns once. Policies follow the tags. Best fit for large catalogs.

Key idea. Tag sensitive columns once and let policy engines enforce masking and row filters for every tool.

FinOps for data platforms: unit economics

Unit metric	Formula	Use
Cost per TB scanned	$\text{compute cost} \div \text{TB scanned}$	Compare engines, catch bad queries
Cost per pipeline run	$\text{cluster cost} \div \text{runs}$	Find expensive jobs
Storage per TB-month	$\text{storage bill} \div \text{TB stored}$	Track retention effects
Cost per data product	tagged spend + platform share	Chargeback and pricing

Levers

Rightsize clusters. Use spot for idempotent jobs. Auto-suspend idle engines. Prune scans. Shorten retention. Tier cold data.

Tag everything

Owner, product and environment on every resource.

Budget per product

Alert at 80% of monthly budget.

Review monthly

Top ten costs, owners, trend.

Key idea. You cannot manage spend you cannot attribute. Tag resources, compute unit costs and give owners a budget.

CI/CD for data pipelines



Environments

Dev on sampled data, staging on a clone of production, then production. No manual edits.

Schema migrations

Expand first (add columns), migrate consumers, then contract (remove). Never break in one step.

Rollback

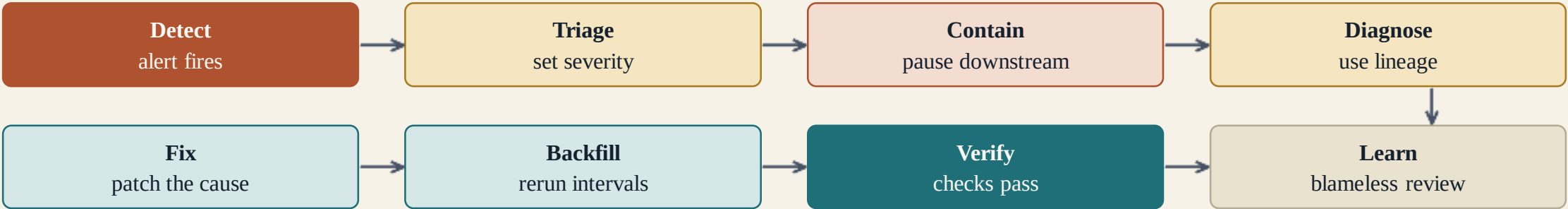
Revert the commit and restore the table snapshot. Both are one command.

Contracts in CI

Reject changes that break a published contract or a downstream test.

Key idea. Treat pipelines and models as software: version, test, review and deploy them automatically with an easy way back.

Incident response for data pipelines



Severity	Example	Response
1	Regulatory or customer-facing data wrong	Acknowledge in 15 minutes, all hands
2	Executive dashboard stale beyond its SLO	Acknowledge in 1 hour
3	Non-critical dataset delayed	Next business day

Key idea. Contain first, then diagnose. Lineage shows the blast radius, and idempotent backfills make the fix safe to replay.

Disaster recovery: RPO, RTO and strategies

Strategy	RPO (data loss)	RTO (time to recover)	Relative cost
Backup and restore	Hours	Hours to days	Lowest
Pilot light	Minutes	Tens of minutes to hours	Low
Warm standby	Seconds to minutes	Minutes	Medium
Active-active	Near zero	Near zero	Highest

Replication order

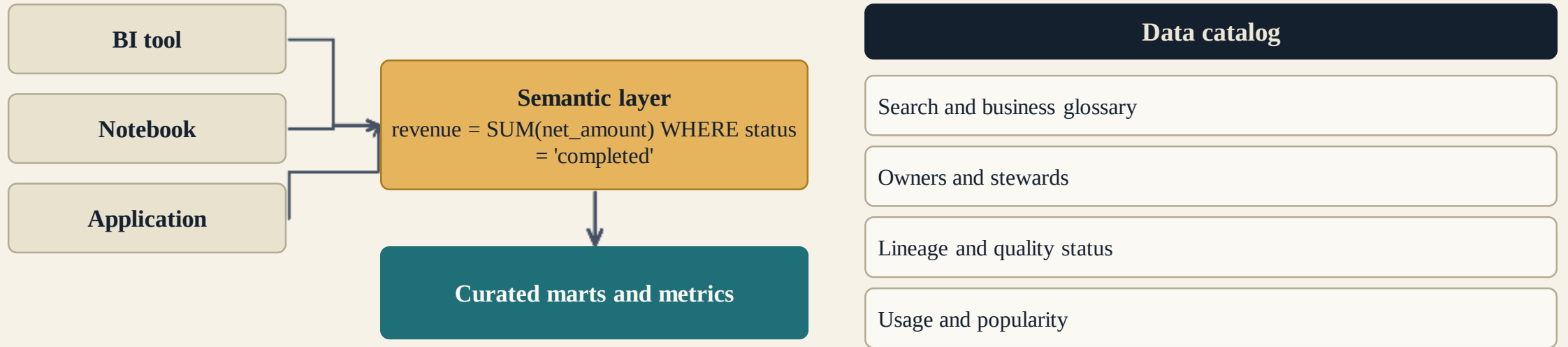
Copy data files first, then metadata. A replica must never point to files that have not arrived.

Practice

Run restore drills and time them. An untested backup is a hypothesis, not a plan.

Key idea. Set RPO and RTO from business impact, then buy the cheapest strategy that meets them, and test the restore.

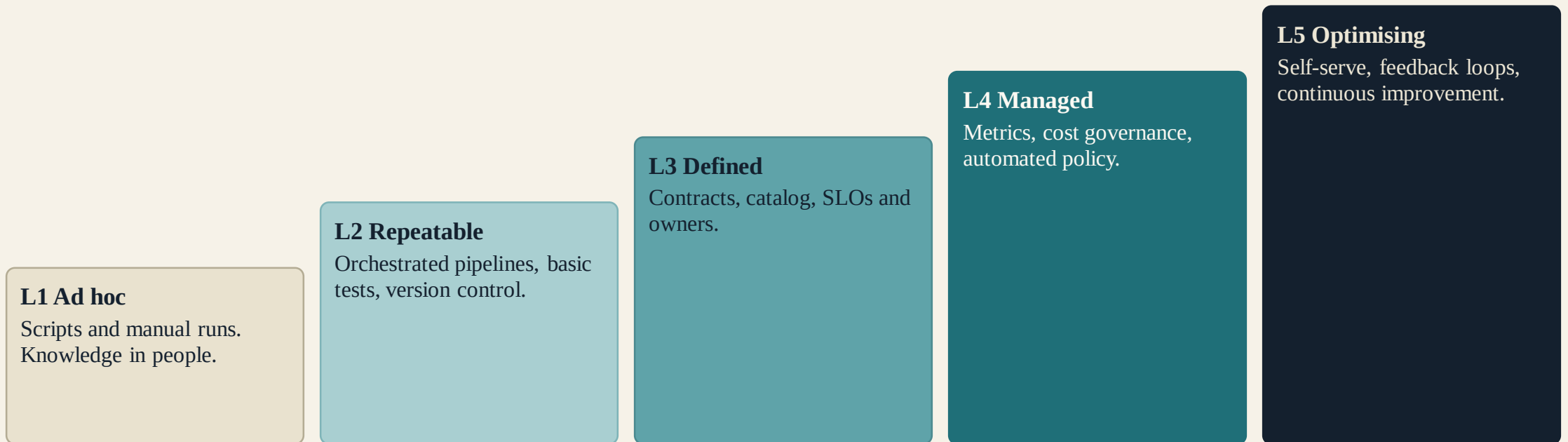
Semantic layer and catalog



One definition of revenue, computed once. Every dashboard, notebook and API asks the semantic layer, so numbers agree. The catalog makes products findable, and tells people who to ask and whether to trust them.

Key idea. Define metrics once and publish them with owners and quality signals, so people find data and trust it.

Platform maturity model



Key idea. Move up one level at a time. Skipping levels adds tooling that the organisation cannot yet operate.

PART 9 OF 9

Synthesis

Decision frameworks, case studies with numbers, anti-patterns and an architecture review kit

- From requirements to architecture
- Five worked case studies
- Anti-patterns, trade-offs and a review checklist

DEPTH LADDER · THIS PART IS LEVEL 5

L1 · Concepts

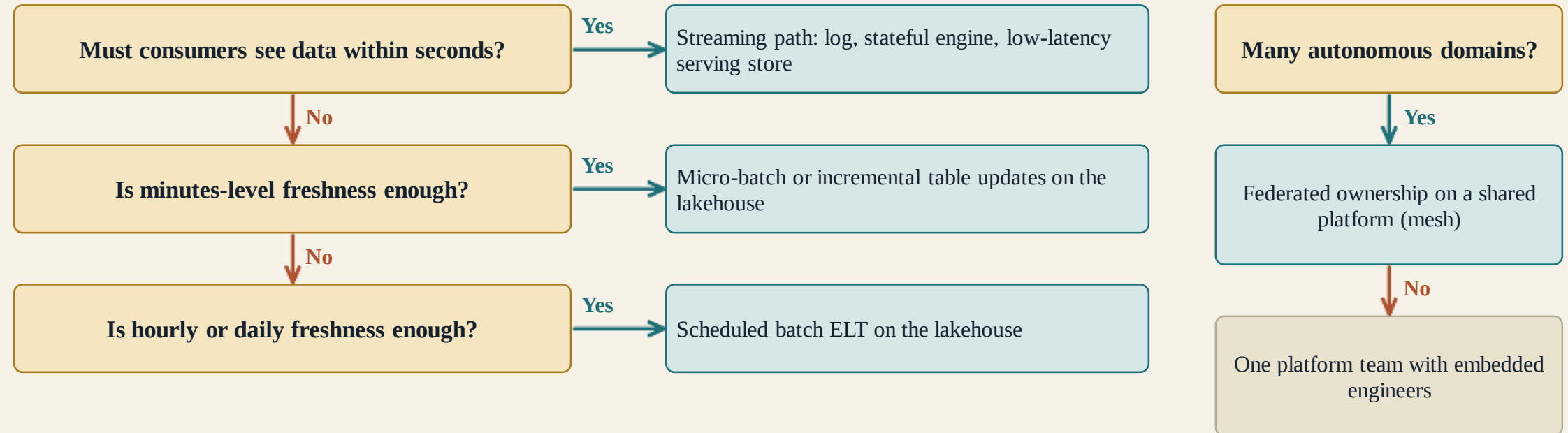
L2 · Structures

L3 · Mechanisms

L4 · Internals

L5 · Quantitative systems

From requirements to architecture: a decision path



Confirm with numbers: what does the consumer do differently with a fresher result? If nothing, choose the slower, cheaper path.

Key idea. Choose freshness from the consumer's decision, then choose ownership from your organisation's size. The two questions are independent.

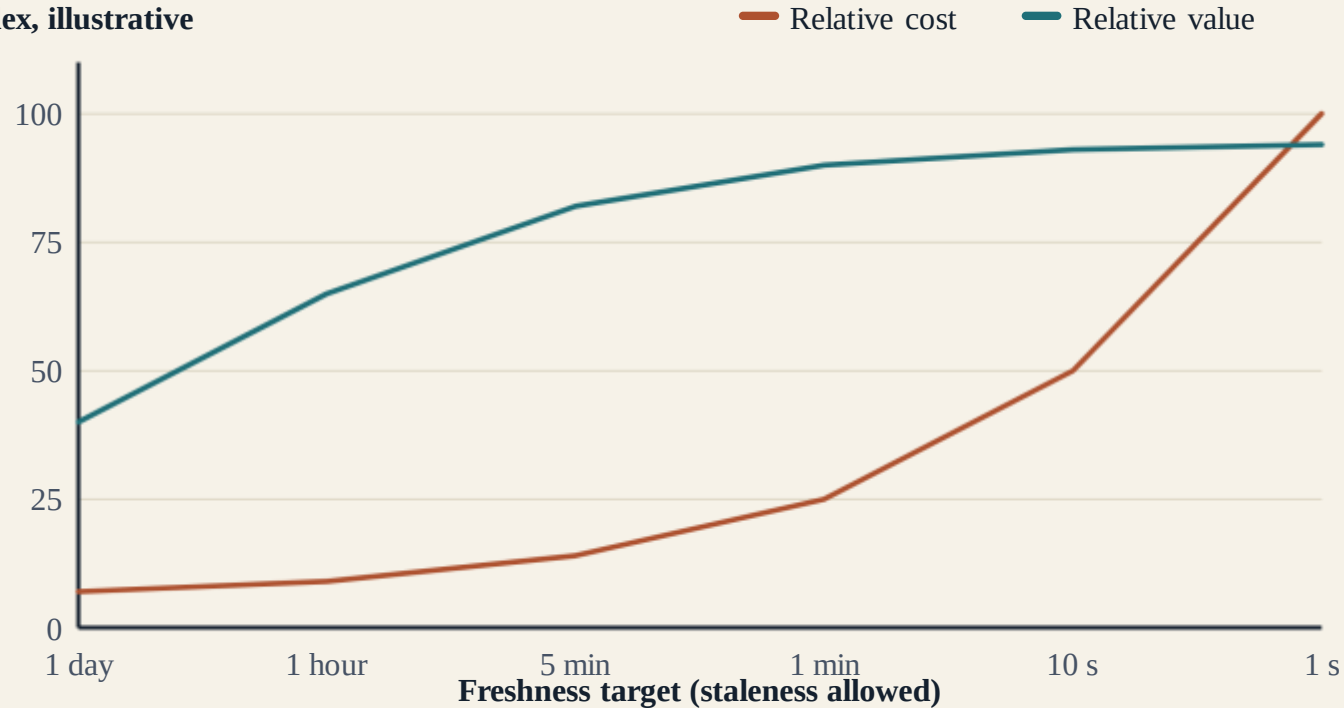
Batch versus streaming across seven dimensions

Dimension	Batch	Streaming
Latency	Minutes to hours	Milliseconds to seconds
Complexity	Lower: stateless jobs, simple retries	Higher: state, time, ordering, back-pressure
Cost profile	Pay per run, cheap at rest	Always-on capacity, pay for idle
Correctness	Complete inputs per interval	Watermarks and late-data policy
Reprocessing	Rerun an interval	Replay the log, rebuild state
Failure handling	Retry the task or run	Checkpoint restore, exactly-once care
Skills	SQL and orchestration	Distributed systems and event-time thinking

Key idea. Streaming buys latency at the price of complexity and idle cost. Adopt it where fresher data changes a decision.

The cost of freshness

Index, illustrative



Shape, not data

Both curves are illustrative. Cost typically rises steeply below minutes. Value saturates once the decision is fast enough.

The knee

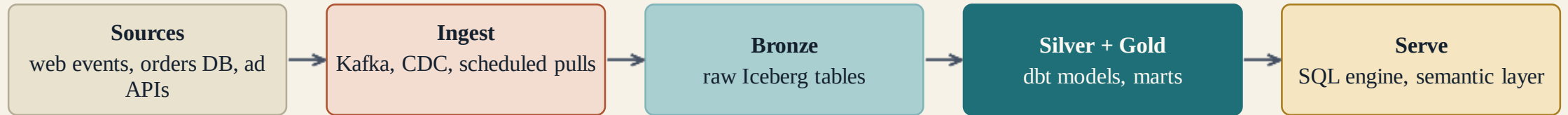
The widest gap between value and cost sits near minutes for many analytics uses.

Your job

Find the knee for each use case and stop there.

Key idea. Freshness has diminishing returns and increasing cost. Buy the least freshness that changes the decision.

Case 1: e-commerce analytics platform



Requirements: dashboards fresh within 15 min · finance close daily · 50 M events/day · 500 k orders/day

Streaming only for events

Web events flow through Kafka into Iceberg every 5 minutes. Orders use CDC.

Batch for everything else

Ad platforms are pulled hourly. Finance marts rebuild daily, with audits.

One table format and catalog

Iceberg with a REST catalog. Any engine can read every table.

Not chosen: a stateful stream processor, or a data mesh. The 15-minute target needs only micro-batch, and one team owns the platform.

Key idea. Case 1, e-commerce analytics: minutes-level freshness is met by micro-batch. Simpler than streaming, cheaper than always-on.

Case 1: capacity and cost arithmetic

Step	Arithmetic	Result
Average event rate	$50\text{ M} \div 86,400\text{ s}$	579 events/s
Peak rate (10× average)	579×10	5.8 k events/s
Raw volume per day	$50\text{ M} \times 1.2\text{ KB}$	60 GB
Parquet, 5× compression	$60\text{ GB} \div 5$	12 GB/day
One year retained	$12\text{ GB} \times 365$	4.4 TB
Storage cost at \$23/TB-month	$4.4 \times \$23$	≈ \$100/month

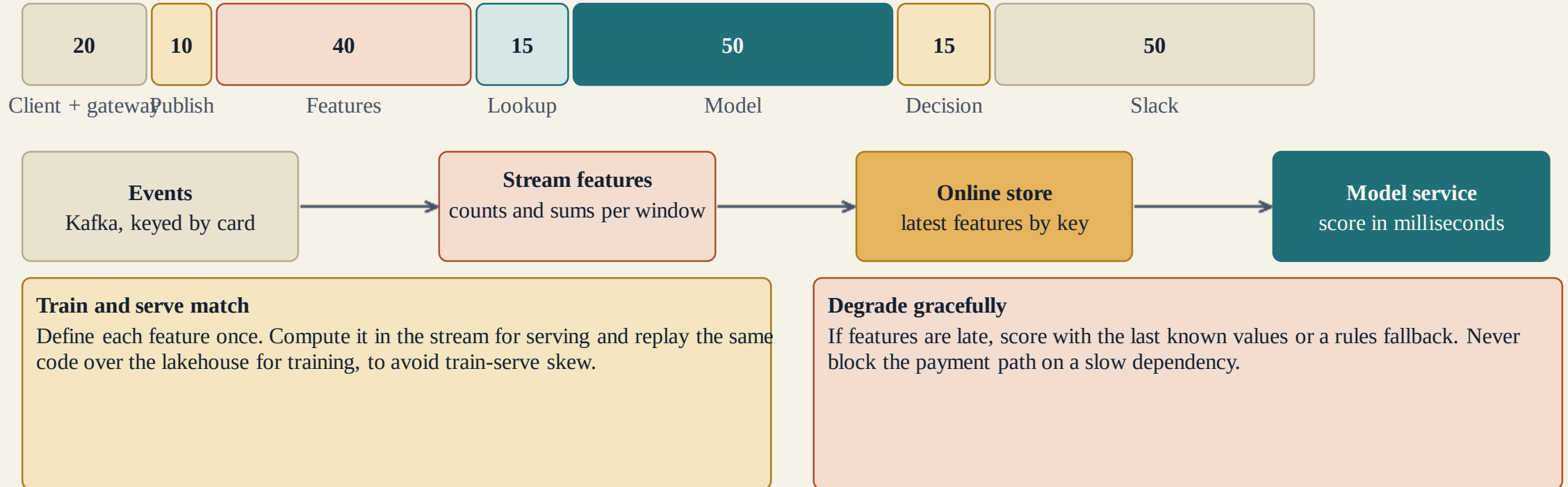
Insight

This is a small workload. Storage costs about \$100 a month, and 5.8 k events/s fits on one small Kafka cluster.

Key idea. Do the sizing first. Many platforms are over-built for data volumes that fit on a laptop-sized budget.

Case 2: real-time fraud scoring

End-to-end latency budget: 200 ms at P99 (each ms = 7 px)



Key idea. Case 2, fraud scoring: a latency budget forces streaming. Budget every hop and design the fallback first.

Case 3: IoT telemetry at one million devices

Step	Arithmetic	Result
Message rate	1 M devices ÷ 10 s	100 k msg/s
Ingest bandwidth	100 k × 200 B	20 MB/s
Raw per day	20 MB/s × 86,400	1.73 TB
Raw per year, 4× compressed	1.73 TB × 365 ÷ 4	158 TB
1-minute rollups per day	1 M × 1,440 × 100 B	144 GB
Rollups per year	144 GB × 365	52.6 TB

Devices
MQTT gateway

Kafka
100 k msg/s

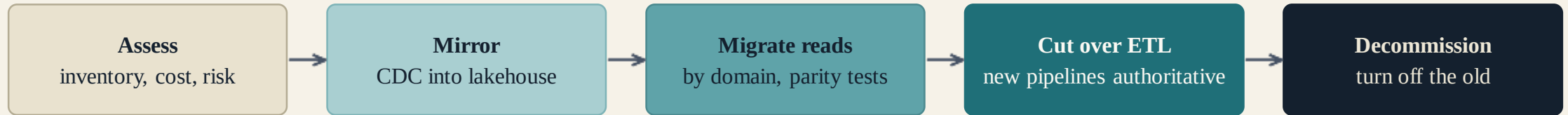
Stream downsample
10 s to 1 min

Lakehouse
raw 7 d + rollups 2 y

Tier by age
Keep raw data for a week for debugging. Keep rollups for trends. This cuts stored volume about 3× a year.

Key idea. Case 3, IoT telemetry: retention and rollups, not ingest rate, decide the cost. Downsample early and tier by age.

Case 4: migrating a warehouse to a lakehouse



Parity testing

Compare row counts, checksums and query results within tolerance before each cut over.

Dual-run cost

Both platforms run for months. Budget for it and keep the window short.

Semantic drift

SQL dialects and null or rounding rules differ. Test business metrics, not just tables.

Rollback path

Keep the old system readable until users confirm the new one.

Strangler pattern: move one domain at a time, so risk is small and value arrives early.

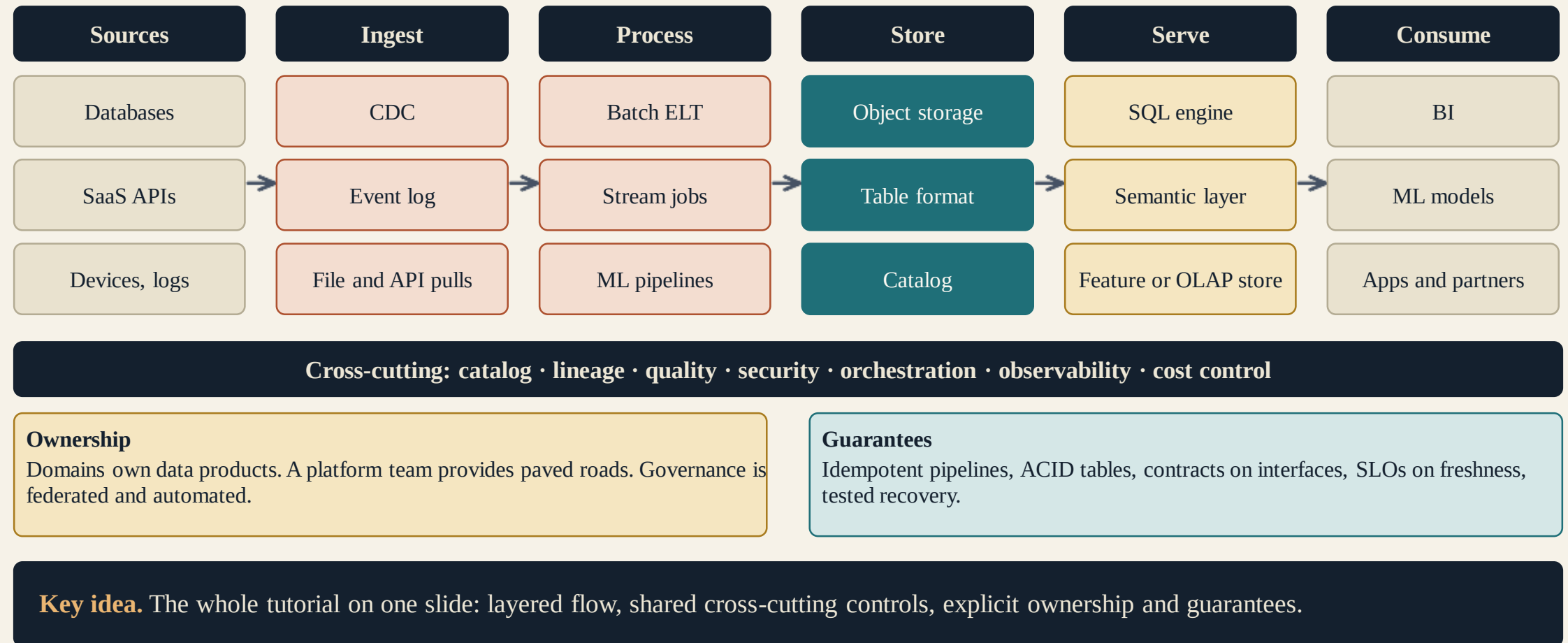
Key idea. Case 4, warehouse to lakehouse: migrate incrementally with parity tests. Big-bang cut-overs fail on semantic differences.

Case 5: introducing a mesh incrementally

	Q1	Q2	Q3	Q4
Platform	Thin platform: templates, catalog	Contracts and quality checks in CI		Cost chargeback
Governance	Council formed	Global policies as code		Policy review
Domains	2 pilot domains	5 data products live		6 domains
Metrics	Baseline	Time to first product		Incidents, cost, adoption
Gate each quarter on measured outcomes, not on the calendar.				

Key idea. Case 5, mesh rollout: start with two domains and a thin platform, and expand only when the numbers justify it.

The reference architecture, end to end



Anti-patterns and what to do instead

Anti-pattern	Why it hurts	Instead
Streaming everything	Always-on cost and complexity for data nobody reads fresh	Match freshness to the decision
Lake without table format or catalog	No transactions, no discovery: a swamp	Adopt an open table format and a catalog
One team for every request	Queueing delay grows steeply near capacity	Federate ownership or add capacity
Pipelines without contracts	Silent breaking changes downstream	Versioned contracts checked in CI
Partitioning by high-cardinality key	Millions of tiny files and slow planning	Time partitions plus clustering
Datasets without owners	No one fixes quality or answers questions	Owner, SLO and on-call for every product

Key idea. Most failures repeat: over-engineered freshness, missing ownership, missing contracts and bad layout.

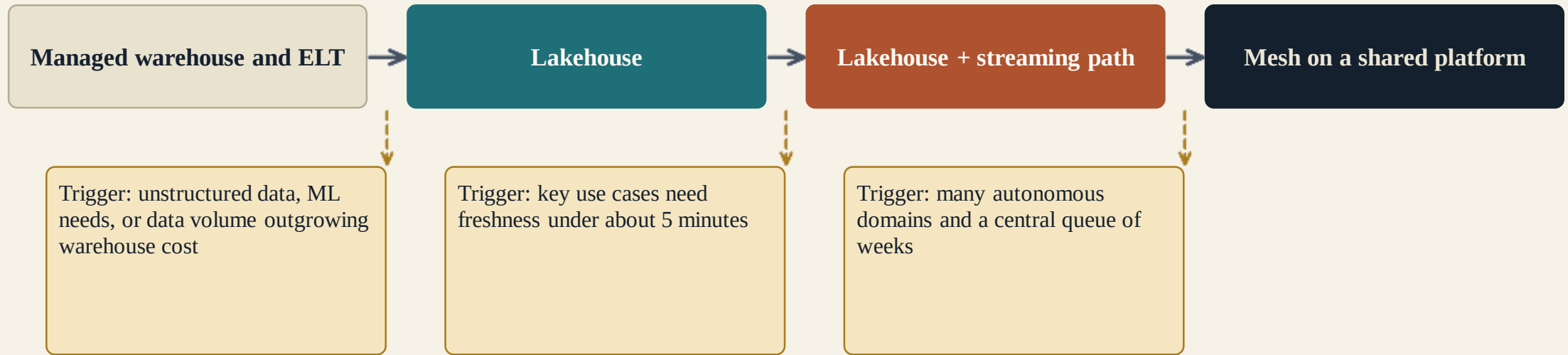
Trade-off matrix: architectures against quality attributes

Architecture	Freshness	Cost	Simplicity	Governance	Scale	Flexibility
Warehouse-centric	3 Fair	2 Low	5 Strong	5 Strong	3 Fair	2 Low
Raw data lake	2 Low	5 Strong	3 Fair	1 Weak	4 Good	5 Strong
Lakehouse	3 Fair	4 Good	3 Fair	4 Good	5 Strong	4 Good
Streaming-first	5 Strong	2 Low	2 Low	3 Fair	4 Good	3 Fair
Data mesh	3 Fair	3 Fair	1 Weak	4 Good	4 Good	5 Strong

Qualitative scores from 1 (weak) to 5 (strong), for a mid-sized organisation. Use them to start a discussion, not to end one.

Key idea. No architecture wins every column. Read the grid against your top two priorities and accept the weakest cell you can afford.

Evolution roadmap and the triggers that justify each step



Thresholds are illustrative. Each move adds operating cost, so trigger it on evidence: a missed SLO, a repeated incident, a queue that keeps growing. Do not adopt the next stage because it is fashionable.

Key idea. Evolve on triggers. Each stage should solve a pain you can measure and that the previous stage cannot.

Architecture review checklist

Sources and ingest

Who owns each source? Is there a contract? Is CDC used, not primary reads?

Storage and formats

Open format and catalog? Files 128 MB to 1 GB?
Partitions sized from bytes?

Processing

Idempotent and replayable? Skew handled? Late data policy defined?

Serving

Latency and concurrency budgets set? Workloads isolated? Metrics defined once?

Governance and ops

Owners, SLOs, lineage, masking, alerts, tested recovery in place?

Cost and scale

Unit costs tracked? Bottleneck named? Capacity arithmetic written down?

Key idea. Use the checklist in design reviews: unanswered questions are risks with owners and dates.

Formula cheat sheet

Volume

$$\text{bytes/day} = \text{rate} \times \text{size} \times 86,400$$

Partitions

$$N = \text{shuffle bytes} \div \text{target size}$$

Amdahl

$$S = 1 / ((1-p) + p/N)$$

USL

$$C = N / (1 + \alpha(N-1) + \beta N(N-1))$$

Little

$$L = \lambda W$$

M/M/1 wait

$$\rho / (1 - \rho) \text{ service times}$$

Tail

$$P(\text{slow}) = 1 - (1-q)^n$$

Cache

$$T = h \cdot t_c + (1-h) \cdot t_m$$

Bloom

$$m = -n \ln p / (\ln 2)^2$$

Quorum

$$R + W > N$$

Availability

$$\text{series } \prod A, \text{ parallel } 1 - \prod (1-A)$$

Burn rate

$$\text{error rate} \div (1 - \text{SLO})$$

Key idea. Twelve formulas cover most first-pass sizing. Write assumptions next to every result, and revisit them as data arrives.

Key takeaways and further reading

1 • Requirements first

Freshness, scale, correctness and cost targets choose the design.

2 • Layout is performance

Formats, partitions and clustering decide bytes scanned.

3 • Guarantees are built

Idempotence, ACID tables, checkpoints and contracts.

4 • Numbers beat opinions

Size it, model the limits, measure the tail.

5 • Ownership scales, tools change

Organise for it, and keep tools replaceable.

Further reading

Designing Data-Intensive Applications, Kleppmann

Fundamentals of Data Engineering, Reis and Housley

Data Mesh, Dehghani

The Dataflow Model, Akidau et al., 2015

Delta Lake, Armbrust et al., VLDB 2020

Apache Iceberg table specification

The Site Reliability Engineering Workbook, Google

Key idea. Thank you. Take one part, run its numbers on your own platform, and decide what to change.